Malware Analysis - Lab Notes

IMPORTANT WARNING

- These notes have been made starting from my notes taken during lessons and the power points published by the professor, then rearranged through the help of Claude to produce a narrative flow.
- All the content belongs to the Professor of the course.
- While I try to ensure everything is inside these notes, please don't use them as a "source of truth", and study from official sources as well.

CHAPTER 1: x86 Assembly

x86 Assembly for Malware Analysis — Reading Compiled C

Reverse engineering a malicious binary almost always begins with the same problem: the source code is gone, and all that remains is machine code. Before any payload, persistence mechanism, or network protocol can be understood, an analyst must be able to look at a stream of x86 instructions and reconstruct the high-level logic the compiler started from. The skill is pattern recognition. Compilers are deterministic translators, so every `if`, every `while`, and every function call leaves a recognizable fingerprint in the assembly. Learning those fingerprints on clean, well-understood code is what makes it possible to read hostile code later, where there are no comments, no symbols, and often deliberate attempts to mislead.

This chapter builds that foundation using a small 32-bit Windows program compiled from a set of ordinary C control-flow functions. The binary targets the 80386 instruction set — the first 32-bit Intel CPU and still the baseline for a huge amount of real-world malware — and is examined statically in a disassembler.

1. The Analysis Environment

Malware analysis is performed inside a disposable virtual machine, restored from a clean snapshot before each session. The guiding principle is isolation: the machine should contain nothing of value and should have no casual path to the outside world. A sample that is being dissected must never be given the opportunity to reach the host, the local network, or the internet, because even a sample intended only for study can execute, spread, or exfiltrate if handled carelessly. Keeping the VM behind a restrictive network configuration and treating every binary as live ordnance is the baseline discipline of the field.

The primary tool is a disassembler. A **disassembler** translates raw machine-code bytes back into human-readable assembly mnemonics; a full-featured one such as IDA goes further, performing automatic analysis to recover function boundaries, cross-references, and data types,

and it also embeds a **debugger** so the same binary can later be executed one instruction at a time. Static disassembly (reading the code without running it) and dynamic debugging (watching it run) are the two complementary halves of analysis. A free edition of IDA is sufficient for everything in this chapter.

1.1 Loading a Portable Executable

When a Windows binary is opened, the disassembler first identifies the file format. A 32-bit Windows program is reported as a *Portable Executable for 80386 (PE)*. Because nothing in the raw code declares how functions receive their arguments, the tool must assume a default **calling convention**, and for this platform that default is `__cdecl` — the convention used by virtually all C code compiled for 32-bit Windows. This assumption drives how the tool interprets stack accesses as function arguments, so it matters from the very first instruction.

1.2 Symbols: a luxury that real malware denies

A compiled binary can carry **debug information** — a side table mapping machine addresses back to original function names, variable names, and line numbers. The GCC/MinGW toolchain emits this information in the **DWARF** format (the standard used by Linux compilers, and by GCC even when it targets Windows). When such information is present, the disassembler offers to load it, and the result is dramatic: functions appear with their original names (`fun_if`, `fun_while`, `main`) and global variables appear as `A`, `B`, `C` instead of bare addresses.

This convenience is almost never available in practice. An author of malicious code has every incentive to **strip** the binary, removing all symbols so that functions appear only as generic labels like `sub_401460` and variables only as hexadecimal addresses. The same program can be built in both a symbol-rich form and a stripped form; comparing them is the fastest way to appreciate how much interpretive work symbols do, and how much of that work falls back onto the analyst when they are absent. For this reason, the patterns taught below must ultimately be recognizable *without* any names at all — the names are training wheels.

There is one recurring exception worth remembering: a careless author sometimes leaves behind the path to the program database (PDB) file that was generated at build time. That leftover string can reveal the directory structure, user name, and project layout of the machine on which the malware was compiled — a small intelligence gift from a sloppy adversary.

1.3 Annotations are not code

The disassembler decorates its output with inferred commentary: reconstructed function prototypes, argument names, stack-frame summaries, and structural hints such as `; __unwind {`. These lines are **analysis output, not instructions that exist in the binary**. One of the first discriminations an analyst must learn is separating what the CPU will actually execute from what the tool merely believes about it. A related discrimination is separating the handful of functions the author wrote from the large body of **compiler and runtime**

support code that every C program is linked against (startup routines, C library internals, thread-local-storage callbacks). Time is wasted analyzing library code; identifying and ignoring it is a skill in itself.

Finally, the disassembler offers two views of the same code. The **text view** is a linear listing; the **graph view** arranges the code into **basic blocks** — straight-line runs of instructions with a single entry and a single exit — connected by arrows that represent possible control flow. A conditional branch produces two outgoing edges, conventionally drawn so that the *condition-true* and *condition-false* successors are visually distinct. The graph view is the natural way to see the shape of an `if` or a loop at a glance, and it is where the constructs below become obvious.

2. Anatomy of a Function

Everything that follows depends on understanding how a function uses the **stack**, so this section establishes the model once.

2.1 Global variables

The program declares three global integers, `A`, `B`, and `C`. Globals live at fixed addresses in a data section for the entire lifetime of the process, so every reference to them is an absolute memory access. In symbol-rich disassembly these appear as `ds:A`, `ds:B`, `ds:C`; stripped, they would appear as `ds:dword_407074` and similar. The key recognition cue is the *absolute address* operand — a hallmark of a global, as opposed to the frame-relative operands used for locals.

2.2 The stack frame, prologue, and epilogue

Local variables and function arguments are not stored at fixed addresses; they live on the stack, in a region called the function's **stack frame**. To address that region reliably, a function anchors it with the **base pointer** register `ebp`. Nearly every function therefore begins with the same two-instruction **prologue**:

```
push    ebp             ; save the caller's base pointer
mov     ebp, esp        ; set our base pointer to the current stack top
```

The `push` writes the caller's `ebp` to the stack (growing the stack downward by four bytes) so it can be restored later; the `mov` then freezes `ebp` at the current top of stack. From this instant, `ebp` is a stable reference point for the whole function, even as `esp` moves around. A function that needs space for locals then subtracts from `esp` (for example `sub esp, 0x10` reserves sixteen bytes).

With `ebp` fixed, the two sides of the frame have opposite signs:

- **Local variables** sit *below* the base pointer, at **negative** offsets: `[ebp-4]`, `[ebp-8]`, `[ebp-0xC]`, and so on. A variable named by convention `var_4` is simply "the variable at

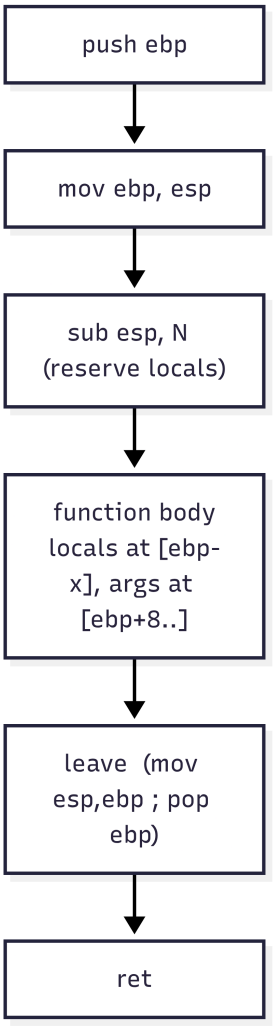
offset 4 below `ebp`." Because these accesses are everywhere, the pattern `[ebp - something]` should immediately read as "local variable" even before its meaning is known. The disassembler lets the analyst assign a name and a size (a 32-bit integer is a `dword`) to each such slot and have every reference updated at once — a core manual-annotation step.

- **Arguments and the return address** sit *above* the base pointer, at **positive** offsets. Immediately after the prologue, `[ebp]` holds the saved old `ebp`, `[ebp+4]` holds the return address pushed by the `call` that entered the function, and `[ebp+8]`, `[ebp+0xC]`, `[ebp+0x10]` hold the first, second, and third arguments respectively.

The mirror image of the prologue is the **epilogue**, which tears the frame down and returns:

```
leave                   ; equivalent to: mov esp, ebp ; pop ebp
ret                     ; pop return address and jump to it
```

The single-byte `leave` instruction restores `esp` and `ebp` in one step; `ret` then transfers control back to the caller. A function that allocated no locals may instead end with a bare `pop ebp` / `ret`. Recognizing prologue and epilogue is what lets an analyst carve an unknown binary into functions in the first place.



2.3 The `__cdecl` calling convention

Under `__cdecl`, arguments are passed on the stack and pushed by the **caller** in right-to-left order, so that the leftmost argument ends up at the lowest address (`[ebp+8]`) inside the callee. Crucially, under `__cdecl` the **caller is responsible for cleaning the arguments off the stack** after the call returns. The function's integer result is returned in the `eax` register. These three facts — arguments on the stack, caller-side cleanup, result in `eax` — are the signature of `__cdecl`, and because it is the default for this platform, they describe almost every hand-written function here.

3. The Driver: `main`

The program's `main` is a thin driver that sets the globals and calls each demonstration function in turn. Its disassembly establishes several patterns at once:

```
push    ebp
mov     ebp, esp
and     esp, 0FFFFFFF0h     ; align stack to a 16-byte boundary
call    ___main             ; MinGW C-runtime initialization
mov     ds:A, 1Eh           ; A = 30
mov     ds:B, 15h           ; B = 21
mov     ds:C, 3             ; C = 3
call    _fun_if
call    _fun_if_else
call    _fun_while
call    _fun_do_while
call    _fun_for
mov     eax, 0              ; return 0
leave
retn
```

Two details deserve comment. The instruction `and esp, 0FFFFFFF0h` clears the low four bits of the stack pointer, forcing it to a multiple of sixteen; this is **stack alignment**, required by the platform's ABI for performance and for certain instructions, and it is a normal part of a `main` prologue rather than anything suspicious. The call to `___main` is **MinGW runtime initialization** — compiler support code, not program logic — and is exactly the kind of boilerplate the analyst learns to recognize and step past. The constants `1Eh`, `15h`, and `3` are the decimal values 30, 21, and 3 written in hexadecimal, the native radix of disassembly; fluent hex reading is non-negotiable.

4. Recognizing Control-Flow Constructs

The heart of the chapter is translating conditional and iterative C back out of assembly. Assembly has no `if` or `while`; it has only **comparisons** that set the `EFLAGS` register and **conditional jumps** that act on those flags. The compiler builds every high-level construct from this pair, and it does so following consistent templates.

The single most important idea is **condition inversion**. High-level code reads *"if the condition holds, execute the body."* The compiler finds it more convenient to generate *"if the condition does **not** hold, jump **past** the body."* As a result, the comparison in the assembly almost always tests the **logical opposite** of the condition written in C: a source-level `if (A < x)` becomes an assembly `jge` (jump if greater-or-equal) that *skips* the body. Reading the jump and mentally negating it to recover the original condition is a reflex the analyst must build.

4.1 The `if` statement — `fun_if`

The source computes a simple guarded subtraction:

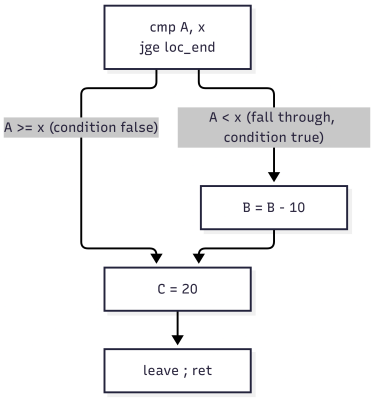
```
void fun_if() {
    int x = 30;
    if (A < x) {
        B = B - 10;
    }
    C = 20;
}
```

Its disassembly:

```
push    ebp
mov     ebp, esp
sub     esp, 10h
mov     [ebp-4], 1Eh        ; x = 30            (local var_4)
mov     eax, ds:A
cmp     eax, [ebp-4]        ; compare A with x
jge     short loc_end       ; if A >= x, skip the body   <-- inverted
condition
mov     eax, ds:B           ; --- body: B = B - 10 ---
sub     eax, 0Ah
mov     ds:B, eax
loc_end:
mov     ds:C, 14h           ; C = 20
nop
leave
ret
```

The local `x` is materialized at `[ebp-4]` and set to `30`. The comparison `cmp eax, [ebp-4]` evaluates `A - x` only to set flags. The source condition is `A < x`; the compiler emits its negation, `jge` (jump if `A >= x`), to **jump over** the body when the body should *not* run. If the jump is not taken, control falls through into the body, which loads `B`, subtracts ten

(`0Ah`), and stores it back. Either way, execution converges on the final `C = 20` (`14h`) and the epilogue. This convergence — two paths meeting at a common successor — is the structural signature of an `if` .



4.2 The `if / else` statement — `fun_if_else`

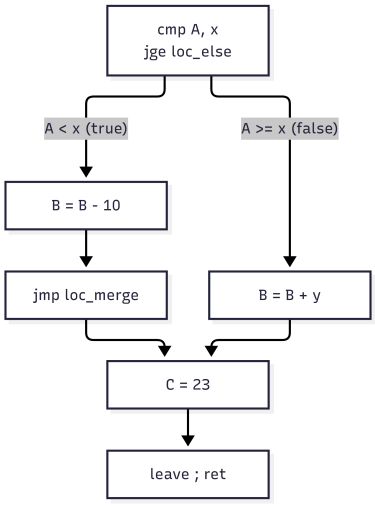
Adding an `else` introduces a second body and a mandatory unconditional jump:

```
void fun_if_else() {
    int x = 30, y = 5;
    if (A < x) {
        B = B - 10;
    } else {
        B = B + y;
    }
    C = 23;
}
```

```
sub     esp, 10h
mov     [ebp-4], 1Eh        ; x = 30
mov     [ebp-8], 5          ; y = 5
mov     eax, ds:A
cmp     eax, [ebp-4]
jge     short loc_else      ; if A >= x, go to the else branch
mov     eax, ds:B           ; --- then: B = B - 10 ---
sub     eax, 0Ah
mov     ds:B, eax
jmp     short loc_merge     ; skip over the else branch
loc_else:
mov     edx, ds:B           ; --- else: B = B + y ---
mov     eax, [ebp-8]
add     eax, edx
mov     ds:B, eax
```

```
loc_merge:
mov     ds:C, 17h           ; C = 23
```

The inverted `jge` now selects between two real alternatives rather than skipping one. The decisive new element is the `jmp short loc_merge` at the end of the *then* branch: once the true path finishes, it must leap over the *else* path, because otherwise execution would fall straight into code meant only for the false case. That **unconditional jump at the tail of the first branch, landing on the same block the second branch falls into**, is the distinguishing mark of an `if/else` as opposed to a bare `if`. Both branches reunite at `loc_merge`, where the shared tail (`C = 23`, i.e. `17h`) runs.



4.3 The `while` loop — `fun_while`

Loops reveal a second compiler habit: the **condition test is placed at the bottom** of the loop, and the function jumps straight to that test on entry.

```
void fun_while() {
    while (B % 4 != 0) {
        C += A * 2;
        B--;
    }
}
```

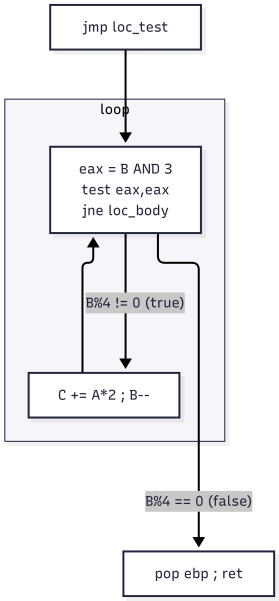
```
push    ebp
mov     ebp, esp
jmp     short loc_test      ; jump straight to the condition
loc_body:
mov     eax, ds:A
lea     edx, [eax+eax]      ; edx = A * 2
mov     eax, ds:C
add     eax, edx
mov     ds:C, eax           ; C = C + A*2
```

```
mov     eax, ds:B
sub     eax, 1
mov     ds:B, eax           ; B = B - 1
loc_test:
mov     eax, ds:B
and     eax, 3              ; eax = B % 4   (keep the low two bits)
test    eax, eax            ; is the remainder zero?
jne     short loc_body      ; if non-zero, loop again
nop
pop     ebp
ret
```

Three idioms appear together here. First, the structure is **test-at-the-bottom with an initial jump to the test**: the very first thing the function does is `jmp loc_test`, so the condition is evaluated before the first iteration, exactly matching `while` semantics (the body may run zero times). The back-edge is the conditional jump `jne loc_body` that climbs back up to the body when the loop should continue. A backward-pointing conditional jump is the universal signature of a loop.

Second, `B % 4` is computed not with a division instruction but with `and eax, 3`. Because four is a power of two, the remainder modulo four is simply the value of the **low two bits**, and `and` with the mask `3` (binary `11`) isolates exactly those bits. The subsequent `test eax, eax` checks whether that result is zero. This "mask with (2^n − 1) to take a modulo by 2^n" trick is extremely common and should be read directly as a modulo operation.

Third, `A * 2` is computed with `lea edx, [eax+eax]` rather than a multiply — the subject of §5.



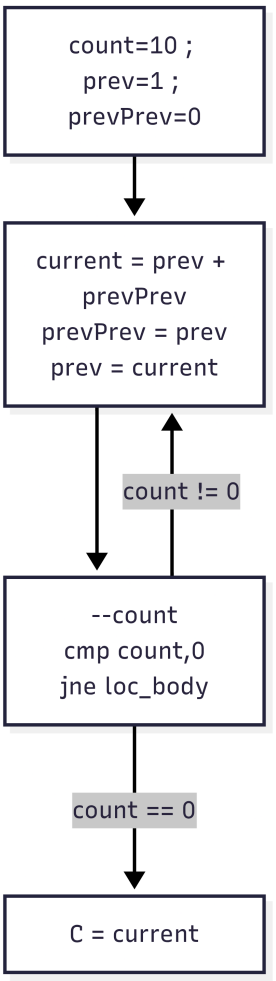
4.4 The `do / while` loop — `fun_do_while`

A `do/while` differs from a `while` in exactly one structural way: there is **no initial jump to the test**, because the body must always run at least once.

```
void fun_do_while() {
    int count = 10;
    int current, prev = 1, prevPrev = 0;
    do {
        current = prev + prevPrev;
        prevPrev = prev;
        prev = current;
    } while (--count);
    C = current;
}
```

```
sub     esp, 10h
mov     [ebp-4], 0Ah        ; count = 10
mov     [ebp-8], 1          ; prev = 1
mov     [ebp-0Ch], 0        ; prevPrev = 0
loc_body:
mov     edx, [ebp-8]        ; prev
mov     eax, [ebp-0Ch]      ; prevPrev
add     eax, edx
mov     [ebp-10h], eax      ; current = prev + prevPrev
mov     eax, [ebp-8]
mov     [ebp-0Ch], eax      ; prevPrev = prev
mov     eax, [ebp-10h]
mov     [ebp-8], eax        ; prev = current
sub     [ebp-4], 1          ; --count
cmp     [ebp-4], 0
jne     short loc_body      ; while (count != 0)
mov     eax, [ebp-10h]
mov     ds:C, eax           ; C = current
```

Execution enters the body directly; only after the body does it decrement `count` and test it with a backward `jne`. The absence of a leading `jmp` to the condition is precisely what separates `do/while` from `while` at the assembly level. This routine also illustrates how multiple locals are packed into the frame — `count`, `prev`, `prevPrev`, and `current` occupy `[ebp-4]`, `[ebp-8]`, `[ebp-0Ch]`, and `[ebp-10h]` — and how a classic Fibonacci-style sliding-window computation looks once reduced to register shuffling.



4.5 The `for` loop — `fun_for`

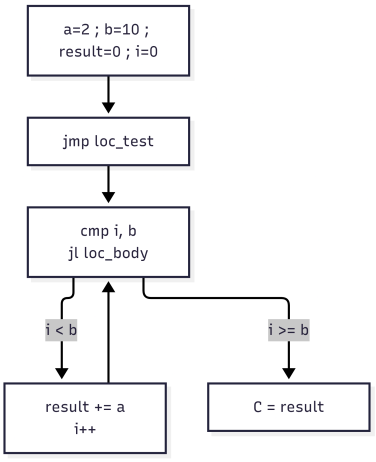
A `for` loop is, at the machine level, a `while` loop with its four parts (initialization, condition, body, increment) laid out in a fixed arrangement:

```
void fun_for() {
    int a = 2, b = 10;
    int i, result = 0;
    for (i = 0; i < b; i++) {
        result += a;
    }
    C = result;
}
```

```
sub     esp, 10h
mov     [ebp-0Ch], 2        ; a = 2
mov     [ebp-10h], 0Ah      ; b = 10
mov     [ebp-8], 0          ; result = 0
mov     [ebp-4], 0          ; i = 0           (initialization)
jmp     short loc_test      ; jump to the condition
loc_body:
mov     eax, [ebp-0Ch]
```

```
add     [ebp-8], eax        ; result += a     (body)
add     [ebp-4], 1          ; i++             (increment)
loc_test:
mov     eax, [ebp-4]
cmp     eax, [ebp-10h]
jl      short loc_body      ; i < b ?         (condition)
mov     eax, [ebp-8]
mov     ds:C, eax           ; C = result
```

The layout matches the `while` template exactly — an initial `jmp` to a bottom-placed test, a backward conditional jump as the back-edge — with one addition: the **increment is emitted at the end of the body**, right before control reaches the test. Recognizing that the last statement of the loop body is a counter update (`add [ebp-4], 1`) separated from the user's actual work (`result += a`) is what lets an analyst reconstruct a `for` rather than an indistinct `while` . Note also that the condition here is `i < b` , and the emitted jump is `jl` (jump if less) — a case where the compiler kept the condition *un-inverted* because the jump target is the body rather than the exit. Whether the condition is inverted depends on whether the jump guards the body or skips it, and the analyst determines that from the jump's target, not from a fixed rule.



5. The `LEA` Instruction

Several of the routines above compute values with `lea` , an instruction whose behavior is a frequent source of confusion and therefore a favorite of both compilers and obfuscators.

`LEA` **(Load Effective Address)** evaluates a memory-address expression and writes the resulting *address* into a destination register. The essential distinction is this: ordinary data-movement and arithmetic instructions that use a memory operand **dereference** it — they read or write the contents of the pointed-to location. `LEA` does **not** touch memory at all. It performs only the address *calculation* implied by the brackets and stores that number. In other words, where `mov eax, [ebx+ecx*4]` fetches the value *at* `ebx + ecx*4` , `lea eax, [ebx+ecx*4]` merely computes `ebx + ecx*4` and keeps it.

This gives `LEA` two distinct uses:

The first is its literal purpose, **address calculation for pointer arithmetic** — computing the address of an array element or structure field without reading it, for example to pass a pointer to another function.

The second, and the reason it appears in arithmetic-heavy code, is as a **fast general-purpose arithmetic instruction**. Because the x86 addressing mode can encode `base + index*scale + displacement` in a single operation — with scale factors of 1, 2, 4, or 8 — `LEA` can evaluate small linear expressions in one instruction, without disturbing the flags register. Thus `lea edx, [eax+eax]` computes `eax * 2` (seen in `fun_while` for `A * 2`), `lea eax, [ecx+ebx]` computes the sum `ecx + ebx`, and `lea eax, [eax+eax*4]` computes `eax * 5`. When an analyst sees `LEA` with an arithmetic-looking bracket expression and the result is subsequently used as a *number* rather than as a pointer, it should be read as plain arithmetic. Confusing the two readings — treating a computed constant as a pointer, or vice versa — is one of the most common early mistakes in reading disassembly.

5bis. Analyst's Reference: Pattern-to-Construct Cues

The following cues summarize the fingerprints established in this chapter. They are stated in terms that survive stripping, since in real analysis the names are gone.

A **prologue** `push ebp` / `mov ebp, esp` (optionally followed by `sub esp, N`) marks a function entry; a matching `leave` / `ret` or `pop ebp` / `ret` marks its exit. Operands of the form `[ebp - k]` are **local variables**; operands `[ebp + 8]`, `[ebp + 0Ch]`, `[ebp + 10h]` are **incoming arguments**; operands that are **absolute addresses** are **globals**.

A `cmp` or `test` followed by a **conditional jump** is a decision point. If the jump **skips a block** and both paths reconverge, it is an **`if`**; the tested condition is the *inverse* of the source condition. If the first block ends in an **unconditional `jmp` over a second block** and both reconverge, it is an **`if/else`**. A **backward** conditional jump is a **loop back-edge**: if the function **jumps to the condition first** and the body is below, it is a **`while`** (or a **`for`**, if the body's last action is a counter increment); if the body runs **before** any test, it is a **`do/while`**.

Arithmetic shortcuts recur constantly: `and reg, (2^n - 1)` is a **modulo by 2^n**; a `shl` / `shr` is a multiply/divide by a power of two; and `lea` with a bracketed expression used as a number is **linear arithmetic**, not a memory access. Hexadecimal constants should be converted on sight (`0Ah` = 10, `14h` = 20, `1Eh` = 30).

Finally, and most importantly: the symbol names, inferred prototypes, and structural comments supplied by the disassembler are **hypotheses to verify, not facts**. In a genuine investigation they will be absent or deliberately misleading, and the only reliable ground truth is the instruction stream itself.

6. Arrays and Indexed Memory Access

An array is a contiguous block of equally sized elements, so accessing element i reduces to a single address computation: `element_address = base + i * element_size`. This maps directly onto the x86 `base + index*scale` addressing mode, where the **scale** is the size of one element. Because the scale is encoded in the instruction, the disassembly reveals the element size for free: a scale of `4` means four-byte elements (`int`, `float`, or a 32-bit pointer), `1` means bytes (`char`), `2` means 16-bit shorts, and `8` means eight-byte elements. Reading the scale is the quickest way to infer the type of the data being iterated.

The routine `fun_array` adds a local array to a global array element by element:

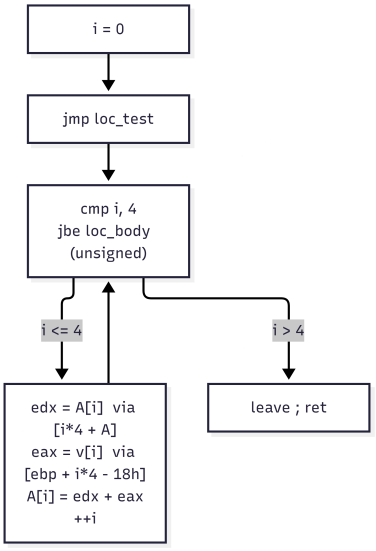
```
int A[5] = { 1, 2, 3, 4, 5 };   // global array
int X, Y;
void fun_array() {
    int v[5] = { 6, 7, 8, 9, 10 };   // local array
    int i;
    for (i = 0; i < sizeof(A)/sizeof(int); ++i)
        A[i] += v[i];
}
```

```
push    ebp
mov     ebp, esp
sub     esp, 20h
mov     [ebp-18h], 6        ; v[0] = 6     (local array begins at ebp-18h)
mov     [ebp-14h], 7        ; v[1] = 7
mov     [ebp-10h], 8        ; v[2] = 8
mov     [ebp-0Ch], 9        ; v[3] = 9
mov     [ebp-8],   0Ah      ; v[4] = 10
mov     [ebp-4],   0        ; i = 0
jmp     short loc_test
loc_body:
mov     eax, [ebp-4]
mov     edx, [eax*4 + A]        ; edx = A[i]   (global base, scale 4)
mov     eax, [ebp-4]
mov     eax, [ebp + eax*4 - 18h] ; eax = v[i]  (frame base ebp-18h, scale 4)
add     edx, eax
mov     eax, [ebp-4]
mov     [eax*4 + A], edx        ; A[i] = A[i] + v[i]
add     [ebp-4], 1             ; ++i
loc_test:
mov     eax, [ebp-4]
cmp     eax, 4
jbe     short loc_body        ; continue while i <= 4   (UNSIGNED compare)
```

```
nop
leave
ret
```

Two addressing patterns stand side by side and should be read as "array element access" on sight. The **global array** `A` is reached as `[eax*4 + A]` — an *absolute base address* plus the index scaled by four. The **local array** `v` is reached as `[ebp + eax*4 - 18h]` — a *frame-relative base* (`ebp - 18h` , the address of `v[0]`) plus the same scaled index. The presence of an index register multiplied by a scale is the giveaway that a single value is being selected out of a sequence; whether the base is an absolute address or an `ebp` -relative offset tells you whether the array is global or local.

The loop bound carries a subtle but important lesson. In source, the condition is `i < sizeof(A)/sizeof(int)` . The expression `sizeof(A)/sizeof(int)` equals `20/4 = 5` and is a **compile-time constant**, so no size calculation appears at run time — the compiler has already folded it. What *does* appear is `cmp eax, 4` followed by **`jbe`** (jump if below-or-equal), and `jbe` is an **unsigned** conditional jump. The compiler emitted an unsigned comparison because `sizeof` yields a value of the unsigned type `size_t` , which forces the whole comparison to be unsigned. Whenever the disassembly shows the unsigned branch family — `ja` , `jae` , `jb` , `jbe` — rather than the signed family — `jg` , `jge` , `jl` , `jle` — it is a strong hint that the operands are unsigned quantities such as sizes, lengths, counts, or pointers. The bound also shifted form: the source `i < 5` became the machine test `i <= 4` , an equivalent the compiler is free to choose.



7. The `switch` Statement

A `switch` is a multi-way branch on a single integer. Compilers implement it with one of two strategies, and the choice depends on how the case labels are distributed. Recognizing which strategy was used — and reversing it back into the original cases — is a frequent exam task and a frequent real-world one, because dispatch logic (command handlers, C2 opcode parsers, state machines) in malware is very often a large `switch` .

7.1 Strategy one: the comparison ladder (sparse cases)

When the case values are few or spread far apart, the compiler emits a straightforward chain of compare-and-jump instructions — effectively an `if / else if / else` ladder. The routine `fun_switch_simple` switches on the global `X` over just two explicit cases:

```
void fun_switch_simple() {
    int result;
    switch (X) {
        case 1:  result = 7; break;
        case 2:  result = 6; break;
        default: result = 5;
    }
    Y = result;
}
```

```
mov     eax, ds:X
cmp     eax, 1
je      short case_1
cmp     eax, 2
je      short case_2
jmp     short case_default
case_1:
mov     [ebp-4], 7
jmp     short loc_merge
case_2:
mov     [ebp-4], 6
jmp     short loc_merge
case_default:
mov     [ebp-4], 5
loc_merge:
mov     eax, [ebp-4]
mov     ds:Y, eax           ; Y = result
```

Structurally this is identical to an `if/else` chain, and that is exactly the point: at the assembly level a sparse `switch` and a hand-written `if/else if` ladder are indistinguishable. Two features identify it as multi-way selection on one variable. First, the *same* value (here `X`, loaded once into `eax`) is compared against a series of constants. Second, each case body ends with a `jmp` to a common **merge** point — the compiled form of the `break` statement. The shared destination is where control reconverges before `Y = result` runs. If a `break` were omitted in source, the corresponding `jmp` would simply be absent and execution would **fall**

through into the next case's body — a pattern worth watching for, since fall-through is both a common bug and a deliberate idiom.

7.2 Strategy two: the jump table (dense cases)

When the case values are many and form a compact, contiguous range, comparing them one by one is wasteful. Instead the compiler builds a **jump table**: an array of code addresses, one per case, stored in read-only data. The switch value is used as an index into this table, and a single indirect jump dispatches directly to the correct case in constant time, regardless of how many cases there are. The routine `fun_switch_complex` switches over the contiguous range `0..4`:

```
void fun_switch_complex() {
    int result;
    switch (X) {
        case 0: result = 7; break;
        case 1: result = 6; break;
        case 2: result = 5; break;
        case 3: result = 4; break;
        case 4: result = 3; break;
        default: result = 8;
    }
    Y = result;
}
```

```
mov     eax, ds:X
cmp     eax, 4
ja      short case_default        ; if (unsigned)X > 4  -> default
jmp     ds:jpt[eax*4]             ; indirect jump through the jump table

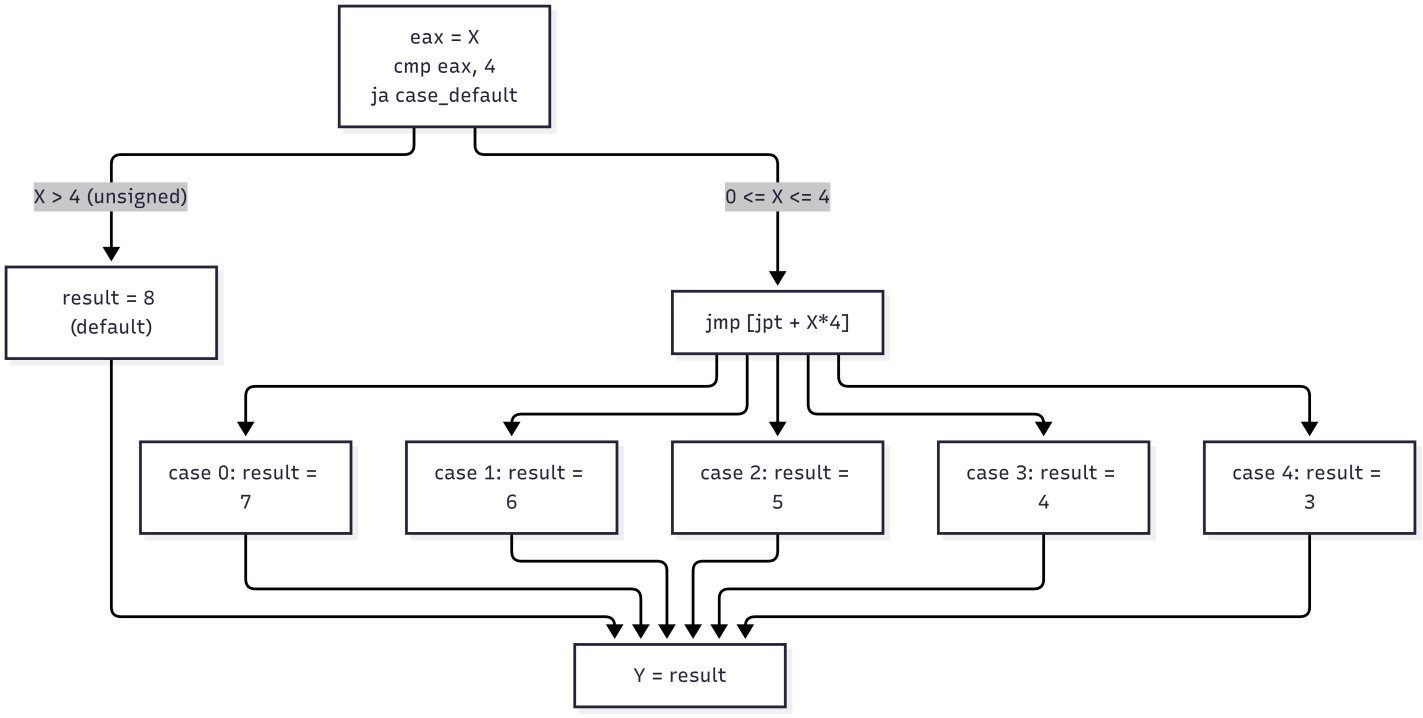
; jpt (in read-only data) holds five code pointers:
;   jpt[0] -> case_0   (result = 7)
;   jpt[1] -> case_1   (result = 6)
;   jpt[2] -> case_2   (result = 5)
;   jpt[3] -> case_3   (result = 4)
;   jpt[4] -> case_4   (result = 3)

case_0: mov [ebp-4], 7 ; jmp short loc_merge
case_1: mov [ebp-4], 6 ; jmp short loc_merge
case_2: mov [ebp-4], 5 ; jmp short loc_merge
case_3: mov [ebp-4], 4 ; jmp short loc_merge
case_4: mov [ebp-4], 3 ; jmp short loc_merge
case_default:
```

```
mov     [ebp-4], 8
loc_merge:
mov     eax, [ebp-4]
mov     ds:Y, eax           ; Y = result
```

Three elements define this pattern and should be memorized as the signature of a jump table. First, a **bounds check**: `cmp eax, 4` followed by `ja case_default`. The use of `ja` (jump if *above*, the unsigned comparison) is deliberate and elegant — a single unsigned test rejects both values greater than the maximum index *and* negative values, because a negative number reinterpreted as unsigned becomes enormous and therefore also "above" the bound. One comparison thus funnels every out-of-range input to the default case. Second, the **indirect jump** `jmp ds:jpt[eax*4]`: the validated index is scaled by four (the size of a 32-bit code pointer) and used to fetch an address from the table, to which control then transfers. An indirect `jmp` whose target is read from `base + index*4` is almost always a compiled `switch`. Third, the **table itself**, a contiguous array of code addresses living in read-only data, each entry pointing at the first instruction of one case body. As before, every case ends with a `jmp` to the shared merge block, the compiled `break`.

A capable disassembler recognizes this idiom, reconstructs the table, and labels each target as a case; without that assistance — in stripped or deliberately obfuscated code — the analyst reads the table manually, following each stored pointer to recover the cases by hand. The deciding factor between the two strategies is case density: a tight contiguous range favors the constant-time table, while sparse or widely separated labels favor the comparison ladder (and very large sparse sets may be compiled into a balanced binary search over the values).



8. Calling Conventions in Depth

A calling convention is the contract between a caller and a callee: where arguments are placed, in what order, who removes them from the stack afterward, which registers must be preserved,

and where the return value is delivered. Because the machine code on each side is generated independently — often by different compilers, or linked in from separate libraries — both sides must agree on the convention or the stack will be corrupted. For an analyst, reading the convention off the disassembly is immediately useful: it reveals how many arguments a function takes, helps distinguish the author's own functions from imported API calls, and, when a convention is hand-rolled or violated, can itself betray hand-written assembly or shellcode.

Three conventions account for nearly all 32-bit Windows code, and all three return a 32-bit integer result in `eax`. They differ mainly in **who cleans the arguments off the stack**.

8.1 `cdecl` — the C default

Under **`cdecl`**, arguments are passed on the stack right-to-left and the **caller** removes them after the call returns. This caller-side cleanup is what makes variadic functions such as `printf` possible, since only the caller knows how many arguments were actually supplied. The callee of `mysum(int a, int b)`:

```
_mysum:
push    ebp
mov     ebp, esp
mov     eax, [ebp+8]        ; first argument  a
mov     edx, [ebp+0Ch]      ; second argument b
add     eax, edx            ; return value left in eax
nop
pop     ebp
ret                         ; plain ret -> the caller will clean the
arguments
```

The arguments are read from `[ebp+8]` and `[ebp+0Ch]`, the sum is left in `eax`, and the function ends with a **plain `ret`** that does not touch the arguments. The single most reliable convention cue lives exactly here: **a bare `ret` means the caller cleans the stack (`cdecl`); a `ret N` carrying an immediate means the callee cleaned `N` bytes itself.**

The caller, `test`, shows the complementary half, with a modern-compiler twist:

```
push    esi
push    ebx
sub     esp, 18h
mov     esi, 20h            ; a = 32   (held in a register: 'register int')
mov     ebx, 3Ch            ; b = 60
mov     [esp+4], ebx        ; place 2nd arg into reserved outgoing-arg space
mov     [esp], esi          ; place 1st arg
call    _mysum
mov     [ebp-0Ch], eax      ; save the result
```

```
lea     eax, [esi+ebx]      ; expected = a + b
mov     ds:expected, eax
...
add     esp, 18h            ; tear down the frame (also reclaims the arg
space)
```

The canonical textbook form of a `cdecl` call is "push the arguments, `call`, then `add esp, N` to discard them." What appears here is the optimized realization that current compilers prefer: rather than a `push` / `add esp` pair around every call, the function reserves a block of **outgoing-argument space** once (`sub esp, 18h`) and writes each argument into it with `mov [esp+k]`. The cleanup is then **folded into the epilogue** (`add esp, 18h`), which reclaims arguments and locals together. Both are `cdecl`; the analyst must recognize that the absence of an `add esp, 8` immediately after a call does not contradict caller-side cleanup — it has merely been deferred and merged into the frame teardown. This routine also illustrates the `register` storage hint: `a` and `b` live in `esi` and `ebx` instead of on the stack, and `a + b` is computed with `lea eax, [esi+ebx]`.

8.2 `stdcall` — the Windows API convention

Under **`stdcall`**, arguments are still pushed right-to-left, but the **callee** removes them before returning, using `ret N`, where `N` is the total size of the arguments in bytes. This is the convention of essentially the entire Win32 API, so recognizing it is central to reading any program that calls Windows functions. The three-argument `mysum`:

```
_mysum:
push    ebp
mov     ebp, esp
mov     edx, [ebp+8]        ; a
mov     eax, [ebp+0Ch]      ; b
add     edx, eax
mov     eax, [ebp+10h]      ; c
add     eax, edx            ; a + b + c in eax
pop     ebp
ret     0Ch                 ; callee pops 12 bytes (3 args) -> stdcall
```

The decisive instruction is `ret 0Ch`: the function removes twelve bytes — its three four-byte arguments — from the stack as it returns. A `ret` with a non-zero immediate is the fingerprint of a callee-cleanup convention, and the immediate itself discloses the argument footprint: `ret 0Ch` means twelve bytes, that is, three dword parameters. The caller correspondingly performs **no** post-call `add esp`:

```
sub     esp, 28h
mov     [esp+8], 60h        ; 96   (3rd arg)
```

```
mov     [esp+4], 40h        ; 64   (2nd arg)
mov     [esp],   20h        ; 32   (1st arg)
call    _mysum
sub     esp, 0Ch            ; re-reserve the 12 bytes the callee popped
mov     [ebp-0Ch], eax      ; result
```

Because this compiler again pre-reserves its outgoing-argument block, the callee's `ret 0Ch` leaves `esp` twelve bytes higher than the caller's frame bookkeeping expects, so the compiler re-subtracts (`sub esp, 0Ch`) to restore the reserved region. The direction of this adjustment is the mirror image of `cdecl`: where a `cdecl` caller *adds* to `esp` to discard arguments, a `stdcall` caller here *subtracts* to compensate for arguments the callee has already discarded. Either way the stable rule holds — the `ret N` in the callee is the authoritative sign of `stdcall`.

8.3 Returning data through pointers

Not every result comes back in `eax`. A function that must return more than one value, or that writes into storage owned by the caller, instead receives the **address** of that storage as an argument and writes through it — the compiled form of a C output parameter (`int* result`). The routine `mysum(int a, int b, int* result)`:

```
_mysum:
push    ebp
mov     ebp, esp
mov     edx, [ebp+8]        ; a
mov     eax, [ebp+0Ch]      ; b
add     edx, eax            ; a + b
mov     eax, [ebp+10h]      ; result   (a pointer)
mov     [eax], edx          ; *result = a + b
nop
pop     ebp
ret                         ; returns void; the output went through the
pointer
```

The third argument is not data but an **address**; `mov [eax], edx` dereferences it and stores the sum into the caller's variable, and the function returns `void`. The caller demonstrates the two fundamentally different ways an address is obtained, which must be read differently in disassembly:

```
lea     eax, [esp+14h]      ; &result  -> address of a LOCAL (a stack slot)
mov     [esp+8], eax        ; pass it as the 3rd argument
```

```
...
mov     [esp+8], 407070h    ; &glob    -> address of a GLOBAL is a constant
```

Taking the address of a **local** variable means computing a stack address at run time, which is precisely what `lea` on an `[esp+k]` or `[ebp-k]` expression does — and this is `lea` used for its literal purpose, pointer arithmetic, not as the arithmetic shortcut seen earlier. Taking the address of a **global** is trivial, because its address is a link-time constant that appears directly as an immediate (`407070h`) moved into the argument slot. An operand that is the *address of* a stack slot (via `lea`) versus an *immediate absolute address* therefore tells the analyst whether a pointer refers to local or global storage — a distinction that matters when tracing where a function's output actually lands.

9. String Instructions and the `REP` Prefix

The x86 instruction set includes a family of **string (block-memory) instructions** that move, store, compare, and scan memory using *implicit* register operands. They are compact, fast, and everywhere in compiled code and malware alike: a compiler inlines `memcpy` and `memset` as these instructions, buffer-zeroing and structure initialization appear as them, and `strlen` , `memcmp` , and search loops are built from them. Shellcode and string decryptors favor them for the same compactness. Reading them fluently is therefore essential, and the difficulty is precisely that their operands are not written out — the analyst must remember the fixed register roles.

9.1 The implicit-operand model

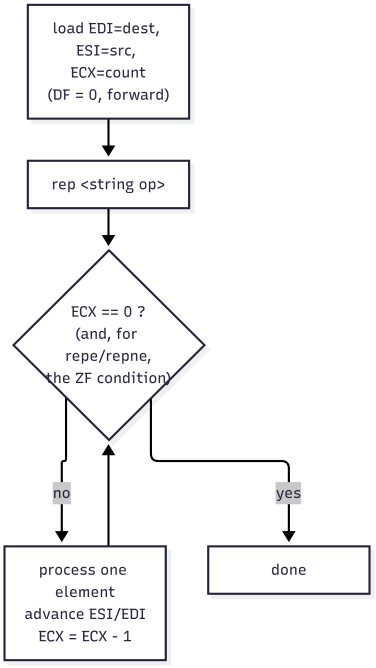
Every string instruction works on one element at a time and takes its operands from fixed registers. The source pointer is always **`ESI`** (`ds:[esi]`), the destination pointer is always **`EDI`** (`es:[edi]`), the count is always **`ECX`** , and the accumulator element is always **`AL`** / **`AX`** / **`EAX`** . A one-letter suffix sets the element size: **`b`** for a byte, **`w`** for a word (two bytes), **`d`** for a dword (four bytes). After each operation the processor **automatically advances** `ESI` and/or `EDI` by the element size. The direction of that advance is governed by the **Direction Flag (`DF`)** in `EFLAGS` : when `DF = 0` the pointers *increment* (forward, low to high addresses), and when `DF = 1` they *decrement* (backward). The instructions `cld` and `std` clear and set `DF` respectively. The platform ABI keeps `DF = 0` by default, so forward processing is the norm; a `std` appearing before a string operation signals deliberate backward processing and is worth noting, as it is occasionally used to handle overlapping regions or as mild obfuscation.

A single string instruction processes just one element. To operate on a whole block it is combined with a **`REP` prefix**, which repeats it under the control of `ECX` :

- **`rep`** repeats the instruction, decrementing `ECX` each time, until `ECX = 0` . It is used with the unconditional movers `movs` and `stos` to process exactly `ECX` elements.

- **`repe` / `repz`** (repeat while equal) repeats while `ECX ≠ 0` **and** the zero flag `ZF = 1`. Paired with the comparators `cmps` and `scas`, it stops early the moment a *difference* appears.
- **`repne` / `repnz`** (repeat while not equal) repeats while `ECX ≠ 0` **and** `ZF = 0`. Paired with `cmps` and `scas`, it stops early the moment a *match* appears.

Because the register roles are fixed, the recognizable prelude to any of these routines is a short sequence that loads `EDI`, `ESI`, `ECX` (and sometimes `AL`) from the arguments, preceded by `push edi` / `push esi` to preserve those registers — `ESI` and `EDI` are callee-saved, so a function that uses them must restore them.



9.2 `MOVSB` — the `memcpy` primitive

`movsb` copies one byte from `ds:[esi]` to `es:[edi]` and advances both pointers. Prefixed with `rep` and a count in `ECX`, it copies a whole buffer — exactly `memcpy(dest, src, n)`:

```
_my_memcpy:              ; my_memcpy(dest, src, n)
push    ebp
mov     ebp, esp
push    edi
push    esi
mov     edi, [ebp+8]        ; dest  -> EDI
mov     esi, [ebp+0Ch]      ; src   -> ESI
mov     ecx, [ebp+10h]      ; n     -> ECX
rep movs byte ptr es:[edi], byte ptr ds:[esi]   ; copy n bytes
pop     esi
pop     edi
leave
ret
```

Seeing `rep movsb` should translate instantly to "copy `ECX` bytes from `ESI` to `EDI`." Its dword cousin `rep movsd` copies four bytes per step and is what a compiler typically emits for a bulk copy, often with a `rep movsd` for the dword-aligned bulk followed by a short `rep movsb` for the trailing bytes.

9.3 `STOSB` — the `memset` primitive

`stosb` stores the byte in `AL` to `es:[edi]` and advances `EDI`. With `rep` it fills `ECX` bytes with a constant — exactly `memset(dest, value, n)`:

```
_my_memset:              ; my_memset(dest, value, n)
push    ebp
mov     ebp, esp
push    edi
mov     edi, [ebp+8]        ; dest  -> EDI
mov     al,  [ebp+0Ch]      ; value -> AL (a single byte)
mov     ecx, [ebp+10h]      ; n     -> ECX
rep stos byte ptr es:[edi], al   ; write AL n times
pop     edi
leave
ret
```

The overwhelmingly common instance of this pattern is **buffer zeroing**: a compiler clears a local array or a structure with `xor eax, eax` followed by `rep stosd` (and a trailing `rep stosb`), writing zeros across the region. Recognizing `rep stos` with a zeroed accumulator as "zero-initialize this buffer" removes a great deal of noise from a function's prologue.

9.4 `CMPSB` with `REPE` — the `memcmp` primitive

`cmpsb` compares the bytes at `ds:[esi]` and `es:[edi]`, setting the flags from their difference, and advances both pointers. Prefixed with `repe`, it keeps comparing **while the bytes are equal** and `ECX` is not exhausted, stopping at the first mismatch — the engine of `memcmp` and of string equality tests:

```
_my_memcmp:              ; my_memcmp(buf1, buf2, n)
push    ebp
mov     ebp, esp
push    edi
push    esi
mov     esi, [ebp+8]        ; buf1 -> ESI
mov     edi, [ebp+0Ch]      ; buf2 -> EDI
mov     ecx, [ebp+10h]      ; n    -> ECX
repe cmps byte ptr ds:[esi], byte ptr es:[edi]   ; compare while equal
jne     DIFF                ; loop ended on a mismatch?
```

```
mov     eax, 0              ; all bytes equal -> return 0
jmp     EXIT
DIFF:
mov     eax, 1              ; a difference was found -> return 1
EXIT:
pop     esi
pop     edi
leave
ret
```

The loop can end for two reasons, and the flags distinguish them. If every byte matched, `ECX` reached zero with `ZF = 1`, so the following `jne` is **not** taken and the function returns `0` (equal). If a mismatch was hit, the last comparison left `ZF = 0`, the `jne` **is** taken, and the function returns `1` (different). The idiom "a `repe cmps` followed by a conditional jump on `ZF`" is the universal shape of a memory or string comparison; a malware analyst meets it constantly in routines that check magic bytes, passwords, or command strings.

9.5 `SCASB` with `REPNE` — the `memchr` / `strlen` primitive

`scasb` compares `AL` against the byte at `es:[edi]` and advances `EDI`. Prefixed with `repne`, it keeps scanning **while the byte does not equal `AL`** and `ECX` is not exhausted, stopping at the first match — the engine of `memchr`, byte search, and (with `AL = 0`) `strlen`:

```
_my_memchr:             ; my_memchr(buf, value, n)
push    ebp
mov     ebp, esp
push    edi
mov     edi, [ebp+8]        ; buf   -> EDI
mov     al,  [ebp+0Ch]      ; value -> AL (the byte to find)
mov     ecx, [ebp+10h]      ; n     -> ECX
repne scas al, byte ptr es:[edi]   ; scan while AL != [edi]
test    ecx, ecx            ; did the scan stop before exhausting the count?
jnz     FOUND
mov     eax, 0              ; not found -> return NULL
jmp     EXIT
FOUND:
dec     edi                 ; EDI advanced one past the match; step back
mov     eax, edi            ; return the address of the matching byte
EXIT:
pop     edi
leave
ret
```

Two subtleties make this routine instructive. First, because `scasb` advances `EDI` *after* each comparison, when a match is found `EDI` already points **one byte past** the byte that matched; the `dec edi` corrects this so the returned pointer addresses the match itself. Second, the function infers success from `ECX`: if the scan stopped with `ECX` still non-zero, it halted early on a match, so control goes to `FOUND`; if `ECX` reached zero, the whole buffer was scanned without a hit and the function returns `NULL`. The canonical application of this same instruction is `strlen`: setting `AL = 0` and scanning with `repne scasb` walks to the terminating NUL byte, and the number of bytes traversed (recoverable from `ECX`) is the string's length. Whenever `repne scasb` appears with a zeroed `AL`, it should be read as a string-length or end-of-string search.

Across all four routines the same mental substitution applies: identify the implicit registers set up before the prefixed instruction, read the prefix to learn the termination rule, and translate the whole block into its single high-level operation — copy, fill, compare, or search.

10. Anti-Analysis Techniques

Everything to this point assumed a cooperative binary: code that means what it says and can be read straight through. Malware frequently does not cooperate. It is written to waste an analyst's time, to confuse automated tools, and to hide its real instructions until the moment they execute. This final section examines a specialized instruction that often appears in such code and then two foundational evasion techniques — anti-disassembly and self-modifying code — using small routines whose true behavior is deliberately concealed.

10.1 The `LOOP` instruction

`LOOP` is a single instruction that fuses a counter decrement with a conditional branch. It **decrements `ECX`** and then **jumps to its target if `ECX` is not zero**, falling through otherwise. It is the compact realization of a counted loop, with `ECX` as the dedicated trip counter:

```
_my_loop:               ; my_loop(a, b) = a * b, by repeated addition
push    ebp
mov     ebp, esp
mov     ecx, [ebp+8]        ; a  -> ECX (loop count)
mov     edx, [ebp+0Ch]      ; b  -> EDX (addend)
xor     eax, eax            ; result = 0
L:
add     eax, edx            ; result += b
loop    L                   ; --ECX; if (ECX != 0) goto L
leave
ret
```

The body adds `b` to an accumulator `a` times, computing `a * b`. The whole loop control is the one instruction `loop L`, equivalent to `dec ecx` / `jnz L` but a byte shorter and tied specifically to `ECX`. Recognizing `loop` as "repeat the preceding block `ECX` times" is straightforward; the reason it earns a place beside the evasion techniques is that compilers rarely emit it (they prefer `dec` / `jnz`, which is faster on modern cores), so its presence often indicates **hand-written assembly** — exactly the kind found in shellcode, decryptors, and the concealed routines below.

10.2 Anti-disassembly: position-independent code and misaligned decoding

Disassemblers reconstruct instructions by deciding where each one begins. The simplest strategy, **linear sweep**, decodes bytes straight through, assuming the byte after one instruction starts the next. Anti-disassembly defeats this assumption by arranging for the real instruction boundaries to differ from the ones a straight-through decode would pick, so that the tool renders plausible-looking but entirely wrong code.

The technique rests on two building blocks. The first is the **`call $+5` / `pop` idiom**, a way for code to learn its own address at run time — *position-independent code*, ubiquitous in shellcode that cannot rely on fixed load addresses:

```
push    ebp
mov     ebp, esp
call    $+5             ; e8 00 00 00 00  — call the very next instruction
pop     eax             ; EAX = address pushed by the call (its own
location)
add     eax, 0Ah        ; EAX = a computed target address
jmp     eax             ; indirect jump to that address
;  ... bytes that a linear sweep now decodes as instructions ...
```

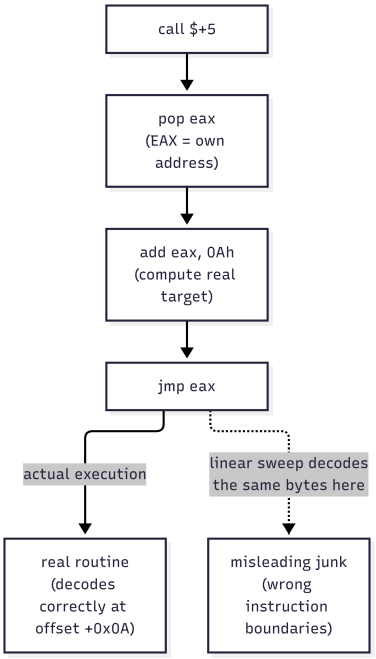
A `call` pushes the address of the following instruction as a return address; because the call target is that same following instruction, nothing is really "called" — the sole effect is to leave the current address on the stack, which `pop eax` retrieves. This is the **GetPC** idiom, and `call $+5` (opcode `E8 00 00 00 00`) is its canonical form.

The second building block is the **indirect jump** `jmp eax`. A disassembler cannot know statically where `jmp eax` leads — the target is a run-time value — so after it, a linear sweep simply keeps decoding whatever bytes follow. Those bytes were crafted so that decoding them from the *wrong* offset yields misleading instructions, while the real control flow (`eax = own_address + 0x0A`) lands a few bytes further in, at an offset that decodes the *same* bytes into the genuine routine. The on-disk, linearly decoded view and the actually-executed view therefore diverge completely:

```
; What a linear disassembler shows after 'jmp eax' (WRONG):
inc     edx
and     eax, [edi+14BA5314h]
add     [eax], al
...                         ; plausible nonsense

; What actually executes, entered via 'jmp eax' a few bytes later (CORRECT):
push    ebx
mov     edx, 14h            ; counter = 20
mov     ebx, 0
mov     ecx, 1
jmp     short L2
L1: mov ecx, eax
L2: lea eax, [ecx+ebx]      ; a running sum: Fibonacci step
    mov ebx, ecx
    sub edx, 1
    jne L1
pop     ebx
...
ret
```

The concealed routine is an ordinary counted loop computing a Fibonacci value — benign in itself, but the point is that its instructions were invisible to a straight reading of the bytes. An analyst defeats this by not trusting linear output across an indirect branch: either reason about the computed target by hand (here, "own address plus ten"), instruct the disassembler to re-decode from the correct offset, or step the `jmp eax` once in a debugger to see exactly where execution goes. More capable disassemblers use **recursive-descent** decoding, which follows branch targets rather than sweeping linearly, and so resist this trick better — but an unresolved indirect jump still stops them, which is precisely why it is used.



10.3 Self-modifying code and runtime unpacking

Self-modifying code (SMC) takes concealment a step further: the real instructions are not merely hidden by misalignment, they are **not present in the file at all** in executable form. The binary stores a scrambled or encrypted blob plus a small **decoder** that rewrites that blob into genuine instructions in memory at run time, and then jumps into the now-decoded region. Static disassembly of the stored bytes yields only the decoder and a block of apparent garbage; the true payload exists only after the decoder has run. This is the governing principle of virtually every **packer** and self-decrypting malware.

Two concrete mechanisms make it work, and both are strong analytic signals. First, the decoder loop walks a range of the program's own code, transforming each unit in place:

```
call    $+5
pop     eax
add     eax, 1Eh            ; EAX = start of the region to decode
mov     edx, eax
add     edx, 24h            ; EDX = end of the region
L1:
mov     ecx, [edx]          ; read a dword of scrambled code
xchg    ch, cl              ; \
ror     ecx, 16             ;  > reverse the four bytes of the dword
xchg    ch, cl              ; /
mov     [edx], ecx          ; write the decoded dword back over itself
sub     edx, 4
cmp     edx, eax
jge     L1                  ; walk backward across the whole region
```

Here the "encryption" is a trivial per-dword byte reversal (the `xchg` / `ror` / `xchg` sequence reverses the order of the four bytes), applied across a range of the function's own body; once the loop finishes, execution falls through into the region it just rewrote, which has become the same Fibonacci routine seen earlier, fronted by a small `jmp` over a junk byte. Real malware substitutes a stronger transform — an XOR key stream, RC4, or a custom cipher — but the shape is identical: a loop reading a code range, transforming it, and writing it back, followed by a transfer of control into the decoded bytes.

Second, and decisively, a program's code section is mapped **read-and-execute but not writable**, so any attempt to overwrite it would fault. Self-modifying code must therefore first make its code pages writable, and on Windows it does so with **`VirtualProtect`**, requesting the `PAGE_EXECUTE_READWRITE` protection — the constant **`0x40`**:

```
mov     [esp+8], 40h            ; flNewProtect = PAGE_EXECUTE_READWRITE
mov     [esp],   401460h        ; lpAddress = the code to be modified
call    _VirtualProtect@16      ; make that code region writable+executable
```

A call to `VirtualProtect` that requests `PAGE_EXECUTE_READWRITE` (`0x40`) over a region of the program's *own code* is one of the loudest tells in static analysis: legitimate application code almost never needs to make itself writable, so its presence strongly suggests unpacking or self-modification and marks the exact spot to focus on. The practical way to recover an SMC or packed payload is dynamic: set a breakpoint just after the decode loop (or at the indirect jump into the decoded region), let the decoder run, and then read or dump the now-plaintext instructions from memory — the standard manual-unpacking workflow.

10.4 Synthesis

The evasions in this section all attack a single assumption — that the bytes on disk, read in order, are the instructions that will run. Anti-disassembly breaks the mapping between byte offsets and instruction boundaries; self-modifying code removes the real instructions from the file entirely until run time. The defenses mirror the attacks: prefer recursive-descent decoding over linear sweep, never trust a static listing across an unresolved indirect branch, treat `VirtualProtect(..., PAGE_EXECUTE_READWRITE)` over code as a red flag, and fall back to dynamic analysis — breakpoints, single-stepping, and memory dumps — whenever the static picture is deliberately false. The recurring lesson of the whole chapter reaches its sharpest form here: the instruction stream the processor actually executes is the only ground truth, and against adversarial code even the disassembly must be treated as a hypothesis to be verified at run time.

11. Analyst's Reference: Pattern-to-Construct Cues

The following cues summarize the fingerprints established in this chapter. They are stated in terms that survive stripping, since in real analysis the names are gone.

A **prologue** `push ebp` / `mov ebp, esp` (optionally followed by `sub esp, N`) marks a function entry; a matching `leave` / `ret` or `pop ebp` / `ret` marks its exit. Operands of the form `[ebp - k]` are **local variables**; operands `[ebp + 8]`, `[ebp + 0Ch]`, `[ebp + 10h]` are **incoming arguments**; operands that are **absolute addresses** are **globals**.

A `cmp` or `test` followed by a **conditional jump** is a decision point. If the jump **skips a block** and both paths reconverge, it is an **`if`**; the tested condition is the *inverse* of the source condition. If the first block ends in an **unconditional `jmp` over a second block** and both reconverge, it is an **`if/else`**. A **backward** conditional jump is a **loop back-edge**: if the function **jumps to the condition first** and the body is below, it is a **`while`** (or a `for`, if the body's last action is a counter increment); if the body runs **before** any test, it is a **`do/while`**.

Array access appears as an operand containing an index register scaled by the element size: `[base + reg*scale]`. An **absolute base** means a global array, an **`ebp`-relative base** means a local array, and the **scale** (1, 2, 4, 8) reveals the element size. A **`switch`** is either a **comparison ladder** — the same value compared against a run of constants, each case ending in a `jmp` to a common merge — or a **jump table** — a `cmp` / `ja` bounds check followed by an indirect `jmp [base + index*4]` into an array of code pointers in read-only data. In both, the per-case `jmp` to the merge block is the compiled `break`; its absence is fall-through.

Watch the **signedness of conditional jumps**: the signed family (`jg`, `jge`, `jl`, `jle`) indicates signed operands, while the unsigned family (`ja`, `jae`, `jb`, `jbe`) indicates unsigned operands such as sizes, lengths, counts, or pointers — often the footprint of `sizeof` or of an index-range check.

Arithmetic shortcuts recur constantly: `and reg, (2^n - 1)` is a **modulo by 2^n**; a `shl` / `shr` is a multiply/divide by a power of two; and `lea` with a bracketed expression used as a number is **linear arithmetic**, not a memory access. Hexadecimal constants should be converted on sight (`0Ah` = 10, `14h` = 20, `1Eh` = 30).

Calling convention is read mainly off the function's exit. A **plain `ret`** means the caller cleans the arguments — **`cdecl`**, the C default and the only convention that supports variadic functions; a **`ret N`** means the callee cleaned `N` bytes itself — **`stdcall`**, used throughout the Win32 API, with `N` revealing the argument footprint (`ret 0Ch` = three dwords). Arguments are read at `[ebp+8]`, `[ebp+0Ch]`, `[ebp+10h]`, and the integer result returns in `eax`. Modern compilers reserve outgoing-argument space once (`sub esp, N`) and write arguments with `mov [esp+k]` rather than `push`, folding cleanup into the epilogue, so the classic `add esp, 8` after a call may be absent or appear as a compensating `sub esp`. A result delivered by `mov [reg], ...` through a pointer argument is an **output parameter**; the pointer's origin is visible in how its address was formed — `lea` on a stack slot for a local, an immediate absolute address for a global.

String instructions carry implicit operands — source in `ESI`, destination in `EDI`, count in `ECX`, element in `AL/AX/EAX`, direction set by the `DF` flag — so they are recognized by the

register setup that precedes them. A **`rep movs`** is a block copy (`memcpy`); a **`rep stos`** is a block fill (`memset`), and with a zeroed accumulator it is buffer zeroing; a **`repe cmps`** followed by a `ZF` jump is a memory/string comparison (`memcmp`); a **`repne scas`** is a byte search (`memchr`), and with `AL = 0` it is a string-length or end-of-string walk (`strlen`). The prefix states the termination rule: `rep` runs the count out, `repe/repz` stops at the first difference, `repne/repnz` stops at the first match.

Anti-analysis leaves its own fingerprints. The `loop` instruction (decrement `ECX`, jump if non-zero) is rarely compiler-generated, so it hints at hand-written assembly. The **`call $+5`** / **`pop`** idiom (`E8 00 00 00 00` followed by a `pop`) is position-independent GetPC code, the staple of shellcode. An **indirect `jmp reg`** or `call reg` with a run-time-computed target defeats linear-sweep disassembly and should never be trusted blindly — resolve it dynamically. A loop that reads a range of the program's own code, transforms each unit, and writes it back, followed by a transfer into that region, is a **decoder/unpacker**, and a **`VirtualProtect` call requesting `PAGE_EXECUTE_READWRITE` (`0x40`)** over code is the signature act of self-modifying code. All of these say the same thing: switch from static reading to dynamic analysis at that point.

Finally, and most importantly: the symbol names, inferred prototypes, and structural comments supplied by the disassembler are **hypotheses to verify, not facts**. In a genuine investigation they will be absent or deliberately misleading, and the only reliable ground truth is the instruction stream itself.

CHAPTER 2: Basic Static Analysis

Lab 03 — Basic Static Analysis

Basic static analysis is the first disciplined pass over a suspicious binary. It is performed **without ever executing the sample**: the file is treated purely as data and interrogated through its own structure — its headers, the libraries it declares, the functions it imports, and the human-readable text embedded in its sections. The objective at this stage is not to produce a definitive account of what the program does, but to generate a small set of well-grounded **hypotheses** about its capabilities. These hypotheses then direct the slower and more expensive phases of the investigation (dynamic analysis and full disassembly), telling the analyst where to spend attention and where to ignore the overwhelming amount of irrelevant material that every Windows executable carries.

Two ideas govern everything that follows. First, **signal is scarce and noise is abundant**. A Windows program drags in a large, complex runtime, so the majority of what is visible — imports such as `CreateThread`, boilerplate from the C runtime, padding bytes — has nothing to do with the malicious behavior. The skill being built is the rapid separation of the few meaningful indicators from the mass of routine ones. Second, **nothing here is proof**. A static observation (an imported function, a string, a timestamp) is a clue whose interpretation must

later be confirmed or discarded. The analyst advances by accumulating small confirmations until a coherent picture emerges.

---

1. How Executables Reuse External Code: Linking

A program interacts with the operating system through **APIs that Windows exports from its libraries (DLLs)**. By identifying precisely which functions a sample pulls from which libraries, an analyst obtains a rough but valuable map of the program's potential functionality. Understanding *how* that external code is attached to the program is therefore foundational, because the linking method determines how much of that map is visible to static analysis in the first place. There are three mechanisms.

Static linking

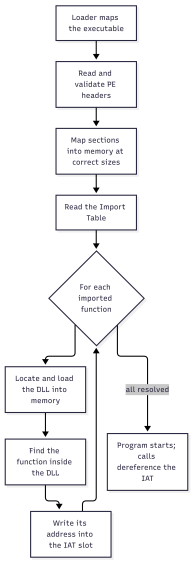
With static linking, the actual code of the library is copied into the executable **at compile time**, during the linker phase, just before the final file is written. The library's pre-compiled routines are merged with the programmer's own code and packed into a single file. Afterwards the two are indistinguishable: a disassembly shows the author's functions and the library's functions side by side, all embedded in the binary.

The advantage is self-containment — a statically linked executable has no external dependencies and will run on any compatible system even if the corresponding libraries are absent. The cost is large duplication: a widely used library is copied, in whole or in part, into every program that uses it. For this reason static linking is common in the Unix world but rare in the Windows ecosystem.

Dynamic linking

Dynamic linking is the de-facto standard on Windows. At compile time the program **declares** which functions from which libraries it needs; the linker writes the correct references into the file but does **not** embed the library code. What remains inside the executable is a set of "dangling" references — entries pointing to code that does not yet exist in the file.

Supplying that missing code is the responsibility of the **Windows loader** at execution time. When the loader maps an executable, it reads the headers, validates them, places each section in memory at the correct size, and then consults a dedicated structure in the PE header called the **import table**. That table lists every function the program uses and the library each function comes from, with an empty slot reserved for an address. The loader processes each entry in turn: it locates the library, loads it into memory if necessary, finds the target function inside it, takes the address of that function's first instruction, and writes that address into the reserved slot of the **Import Address Table (IAT)**. Once every slot is filled, the program can run: whenever the code calls a dynamically linked function, it dereferences the corresponding IAT entry and reaches the correct memory location.



The consequence for analysis is decisive. Under static linking the library **code** is visible but its identity is not; under dynamic linking the **code is absent**, but the **names of the libraries and of the imported functions are plainly listed** in the import table. Dynamic linking is thus the case that basic static analysis exploits most directly — the import list is read straight out of the file.

Runtime linking

Runtime linking is the third mechanism and the one most useful to malware authors. Here the program does **not** declare its dependencies at compile time. Instead, the code itself carries the instructions needed to locate a library at runtime, load it into memory, find the required function inside it, and call it — effectively performing the loader's job by hand, later and on demand. The canonical building blocks are `LoadLibrary` (bring a DLL into the process) and `GetProcAddress` (resolve a function's address by name within that DLL).

For legitimate software this has real benefits: a library is loaded only if and when it is actually needed, which is exactly how plugin systems work. For malware the attraction is concealment. Because the library and function names are only referenced at runtime, they need not appear in the import table at all, and the strings naming them can be obfuscated until the moment of use. A sample that relies on runtime linking can therefore interact heavily with the operating system while presenting an import table that looks almost empty — a mismatch that is itself a strong indicator.

> **Imports versus exports.** The PE format is used for both executables and libraries. An executable is expected to show many *imported* functions and essentially no *exported* ones; a library shows the opposite. A mixed file — an executable that also exports functions — is unusual and worth investigating, because it suggests the program is also meant to be loaded and called by something else.

> **Imports by name versus by ordinal.** A function can be listed in the import table either by its **name** or by its **ordinal**, the numeric position of the function inside the DLL. Importing by ordinal removes the readable name from the table, a light form of obfuscation: instead of `connect` the table shows a number. Public ordinal-to-name

> mappings exist for the standard system libraries, so the obfuscation is easily reversed, but it does hide the function from a careless first glance.

---

2. A Field Guide to the Common Windows Libraries

A handful of system libraries account for most of what a basic sample imports. Recognizing them on sight, and knowing what their presence implies, is the quickest route to a first hypothesis.

`kernel32.dll` contains the core operating-system functionality exposed at user level — process and thread management, file operations, memory, synchronization. It is imported by very nearly every Windows program, so its presence is unremarkable; the interest lies in *which* of its functions are used.

`advapi32.dll` provides more advanced capabilities, most importantly access to the **Windows registry** and to the **Service Control Manager**. Imports from this library point toward configuration tampering, persistence, or service installation.

`user32.dll` and `gdi32.dll` implement the graphical user interface. Malware rarely has a visible UI, so imports from these libraries deserve a second look: a sample that imports them while showing no window may be using them to inspect or manipulate *other* programs' windows — for example to read data out of them, or (via functions such as `SetWindowsHookEx`) to install a keyboard hook and build a keylogger.

`ntdll.dll` is the low-level interface to the kernel, the Windows analogue of raw system calls. Most programs reach the kernel indirectly through `kernel32.dll`, which sits above `ntdll.dll`. Microsoft never published official documentation for `ntdll.dll`; it is nonetheless stable, widely reverse-engineered, and well described in unofficial references. Direct use of it can indicate an attempt to bypass the normal, more easily monitored user-level API.

`ws2_32.dll` (and the older `wsock32.dll`) is the Winsock library, Windows' implementation of sockets for network communication. It is very close to, though not perfectly compatible with, the Unix socket interface, exposing the familiar `socket`, `connect`, `send`, `recv`, and so on. Its presence signals raw network activity. Note that opening even a single connection to a server requires six or seven distinct socket calls, so a moderately long list of Winsock imports is normal for any networked sample.

`wininet.dll` is a higher-level network library built on top of the sockets layer. It offers pre-packaged operations — download a file from a URL, issue an HTTP request — that hide the underlying connection management. Its presence suggests the sample communicates over standard web protocols (HTTP/HTTPS) rather than hand-rolled socket traffic.

`msvcrt.dll` is the Microsoft Visual C++ runtime. It is imported by essentially everything built with Microsoft's compiler toolchain and is **not** in itself an indicator of suspicious

behavior; it is background, not signal.

The guiding heuristic is that **exotic imports matter more than common ones**. A `CreateProcessA` is used by countless benign programs, so it earns attention only once something more unusual has already raised suspicion. A rarely seen import — `ShellExecuteExA`, a registry or service API where none is expected — is itself grounds to dig deeper.

---

3. Reading the PE Header: What to Extract

Three pieces of header information drive the first questions in any basic analysis.

Compilation timestamp

The PE header contains a field recording the moment the executable was produced by the compiler. It is useful because it is **not** consulted during execution — the loader ignores it entirely. It exists only for the benefit of anyone inspecting the file, which also means an attacker can trivially overwrite it, and some compilers leave it meaningless. A plausible value (a real date, rather than a null or default) is therefore **a clue, not a fact**: it may genuinely record when the sample was built, but it must never be taken for granted. The file-system timestamps, by contrast, record when the file arrived on the current machine and say nothing reliable about the build.

Sections

An executable is divided into **sections**, each with a name, a size, a location, and a set of permissions. The standard ones are:

- `.text` — the executable code. This is the section that holds the program's instructions and therefore contains the **entry point**; it is marked executable (and implicitly readable) but not writable.
- `.data` — writable data used by the program at runtime. Its *virtual* size (the space it will occupy in memory) is typically much larger than its *raw* size on disk, because much of this data does not yet exist while the file sits static; the gap between the two sizes is expected and normal.
- `.rdata` — read-only data, such as constant strings and, often, import/export information. Its virtual and raw sizes are usually close.
- `.rsrc` — resources (icons, dialogs, embedded files). In a minimal program this section is small or nearly empty; a disproportionately large resource section can mean an **embedded payload** is stored there.

Two quantities repay attention. The relationship between **virtual size and raw size** characterizes a section, and a large anomaly — especially code sections with inconsistent sizes — is a red flag. **Entropy**, a measure of randomness from 0 to 8, is the other: a section packed

with compressed or encrypted data shows high entropy (approaching 7.9+), whereas ordinary code and text sit well below. Unusual section **names** are equally telling; the standard `.text` / `.data` / `.rdata` / `.rsrc` set is benign, while invented names are a hallmark of packing (discussed at the end of this chapter).

Other optional-header fields

The optional header also records the **image base** (typically `0x00400000` for an executable), the **entry-point address** (expressed relative to the image base and normally located inside `.text`), and the section and file alignment values that explain the padding between sections. These confirm the file's basic sanity and locate where execution begins.

---

4. The Analysis Workflow and Tooling

The sample is **never executed** during this phase. The practical loop uses a small, free toolset:

- A **PE inspection tool** (for example PeStudio) parses the headers and presents them through a readable interface, with separate views roughly mirroring the PE structure: file header, optional header, sections, libraries, imports, and an extracted **strings** list. The compilation timestamp and the file's hashes appear on the first view.
- The hashes — typically both an MD5 and a SHA-based digest — are the file's fingerprints. Submitting a hash to an online reputation service such as **VirusTotal** reveals whether the sample has been seen before and how it has been classified, without ever uploading or running the file itself.
- The **strings** utility extracts printable character sequences from the file. It reads the file as a raw byte stream and does **not** understand the PE structure, which has two consequences: (a) imported function names, which physically live inside the import table, will also surface as strings — this is expected and duplicates the import view rather than adding to it; and (b) strings stored as **wide (UTF-16) characters** require the tool to be run in a wide-string mode, or they will be missed by a default ASCII scan.
- When an imported function is unfamiliar, its behavior is looked up in the **official Microsoft documentation**, which gives the signature, parameters, and semantics. The reading must stay shallow and purposeful: the aim is a rough sense of what a function enables, not mastery of every parameter. Detail is revisited only if later analysis proves the first pass insufficient.
- Any concrete artifact — an IP address, a domain, an unusual string — can be searched on the web to check whether it is a known **indicator of compromise (IOC)**, i.e. a data element associated with recorded malicious activity.

---

5. Worked Analysis — `Lab_03-1.exe`

The sample is a small 32-bit Windows GUI executable of 8,192 bytes. Its fingerprints are:

Property	Value
MD5	`4280aac55c1d3c327a6c00f0f0085677`
SHA-256	`84b7967aad00e982842045e7b9744af0a457d46bba70456e5f99e7eb9cd783c7`
Size	8,192 bytes
Type	PE32 executable (GUI), Intel 80386, 4 sections

5.1 When was the file compiled?

The compiler timestamp in the PE header resolves to **14 May 2009**. This is a well-formed, non-default value, consistent with a real build date rather than a compiler placeholder — and the file's Rich header confirms a **Microsoft Visual C++ 6** toolchain, which also explains the presence of `msvcrt.dll`. As always, the date is taken as indicative only, since the field is freely modifiable and plays no role in execution.

5.2 Section layout

The four sections are the standard set, which by itself argues **against** packing:

Section	Virtual size	Raw size	Entropy	Role
`.text`	3,750	4,096	5.93	Code; contains the entry point
`.rdata`	1,662	2,048	4.42	Read-only data and import information
`.data`	1,576	512	1.89	Writable runtime data (raw ≪ virtual, as expected)
`.rsrc`	124	512	3.38	Resources; nearly empty in this minimal program

The entropy values are moderate throughout, the names are conventional, and the entry point lies within `.text`. Nothing in the section table suggests obfuscation; the code is available for straightforward inspection.

5.3 Imports and the hypotheses they generate

The sample imports from **five** libraries. Four of them — `kernel32.dll`, `user32.dll`, `shell32.dll`, and `msvcrt.dll` — are immediately recognizable, and the fifth, `ws2_32.dll`, is the Winsock library. The distribution of imported functions is itself informative: a single function from `user32.dll`, only two from `shell32.dll`, a larger set from Winsock, and the bulk from `kernel32.dll`.

The functions that matter group into four capabilities.

Network communication (`ws2_32.dll`). The sample imports `WSAStartup`, `socket`, `connect`, `send`, `recv`, `closesocket`, `htons`, and `inet_addr`. This is precisely the minimal set needed to initialise Winsock, create a socket, resolve and connect to a hard-coded address, and exchange data — the signature of a program that **opens an outbound network connection**. Notably, every one of these functions is **imported by ordinal rather than by name** (ordinals 115, 23, 4, 19, 16, 3, 9, 11): in a raw PE view they appear as bare numbers, a mild obfuscation that a PE tool or a public ordinal table resolves back to the familiar socket calls.

Shell interaction (`shell32.dll`). The two imports are `ShellExecuteExA` and `SHChangeNotify`. The first is the significant one: `ShellExecuteExA` launches other programs through the shell and is rare enough in ordinary software that its mere presence justifies closer study (see 5.4). This is a far stronger indicator than the process/thread functions below, which are common even in benign code.

Process, thread, and pipe management (`kernel32.dll`). The sample imports `CreateProcessA`, `CreateThread`, `TerminateThread`, `TerminateProcess`, together with `CreatePipe`, `PeekNamedPipe`, `DisconnectNamedPipe`, `DuplicateHandle`, and the file primitives `ReadFile` and `WriteFile`. Process and thread creation are unremarkable in isolation, but **in combination with pipes and with an outbound socket** they form a recognizable pattern: a child process whose standard input and output are redirected through pipes and relayed over the network — in other words, a **remote shell**. `GetComputerNameA` additionally lets the program identify the host it is running on.

Host/environment helpers. `GetEnvironmentVariableA`, `GetShortPathNameA`, `GetModuleFileNameA`, and the string helpers `lstrcatA` / `lstrcpyA` let the program read its own location and build command strings — relevant to the self-deletion behavior discussed below.

The single `user32.dll` import, `LoadStringA`, is minor (it reads a string resource) and does not change the picture.

5.4 Deep dive: `ShellExecuteExA` and `SHELLEXECUTEINFO`

`ShellExecuteExA` is the standout import, and its documentation illustrates a recurring Windows pattern. The function itself takes a **single parameter**: a pointer to a `SHELLEXECUTEINFO` structure. Early Windows APIs accumulated so many parameters over successive versions that Microsoft eventually collapsed them into one structure with many fields — so the one-line description of the function is uninformative, and the real meaning lives in the structure's fields.

Two fields carry the payload of intent. **`lpVerb`** is a string naming the action to perform — among the allowed verbs are `open`, `edit`, `explore`, `print`, `properties`, and `runas` (execute with elevated privileges). **`lpFile`** names the file or object the verb acts upon, and **`lpParameters`** supplies optional command-line arguments to pass to it. The structure also

contains an **`HWND` handle**; a *handle* in Windows is an opaque reference to an operating-system object — a file, a network connection, a window, a UI control — analogous to a Unix file descriptor but used far more pervasively.

The takeaway is that `ShellExecuteExA` can launch an arbitrary executable with arbitrary arguments and an arbitrary verb. The exact program and arguments are not fixed by the function; they are assembled by the surrounding code and would be recovered, during later disassembly, by reading the instructions that populate the `SHELLEXECUTEINFO` fields immediately before the call.

5.5 Strings that stand out

Filtering out the noise — short fragments such as `D$`, `QJ`, and the long `PADDING` run — several strings are meaningful, and together they confirm and extend the import-based hypotheses.

- **`60.248.52.95:443`** — an **IP address with a port**, stored as a **wide (UTF-16) string**, which is why it surfaces only under a wide-character scan. This both reinforces the network hypothesis (the Winsock imports now have a concrete destination) and supplies a hard indicator. The port **443** conventionally carries HTTPS, suggesting the sample may attempt a TLS-encrypted channel to this address. Public reputation lookups flag this IP as a known indicator of compromise associated with historical malware analysis material.
- **`http://www.ueopen.com/test.html`** — an embedded URL, physically preceded by a stray `0x01` byte. A reachable remote resource of this kind is a second network artifact; the leading non-printable byte hints that the string is consumed by code rather than displayed.
- **`cmd.exe`**, **`COMSPEC`**, **`/c del`**, and **`> nul`** — the components of a **command-shell invocation**. `cmd.exe` is the Windows command interpreter; `COMSPEC` is the environment variable that points to it (so the path to the interpreter need not be hard-coded); `/c` runs a single command and exits; `del` deletes a file; and `> nul` discards the command's output (the Windows equivalent of redirecting to `/dev/null`). The strings `cmd.exe` and `/c del` appear *separately* because they are copied into two different fields of the `SHELLEXECUTEINFO` structure — the executable in `lpFile`, the arguments in `lpParameters`. The file actually being deleted is not present among the strings, which implies it is **determined at runtime** — most plausibly the program's own path, obtained via `GetModuleFileNameA` and appended to the `del` command.
- **`urlmon.dll`** — named in the strings but **absent from the import table**. This mismatch is the fingerprint of a library **loaded at runtime** (via `LoadLibrary` / `GetProcAddress`) rather than at load time. `urlmon.dll` provides URL and Internet-protocol handling (notably file downloads), a third confirmation of network capability whose exact use can only be pinned down later, in the code.
- **`ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789-/`** — a 64-character alphabet. A string of this exact shape is the lookup table for a **Base64-style encoding** (here a variant using `-` in place of the usual `+`). Its presence indicates the

sample encodes or lightly obfuscates data — most likely its network traffic — rather than sending it in the clear.

- `*(SY)#`, `*(SY)# cmd`, and `send = %d` — operational strings. `*(SY)#` has the form of a **command prompt** that a backdoor would send to its operator, and the `cmd` suffix ties it to launching the shell. The `%d` placeholder marks a format string (used with `sprintf`-style functions) assembled at runtime, consistent with reporting the number of bytes sent.

5.6 What happens when the sample runs

Executed in a protected environment (and only after a backup copy is made), the program appears to do nothing visible — the common experience with malware — but one observable effect does occur: **the executable deletes itself**. This is exactly the behavior the `cmd.exe` + `/c del` + `COMSPEC` strings predicted, and it explains why the target filename was absent from the static strings: the program inspects its **own** path at runtime and appends it to the delete command.

Self-deletion is a classic concealment technique. The sample is designed to be loaded into memory, remain resident performing its function, and simultaneously remove its on-disk copy, leaving a **memory-only** process that leaves no file behind for the user or a casual examiner to find. (A fuller account of the runtime behavior — whether the network connection is actually established, and what traverses it — belongs to dynamic analysis, which observes the running process directly.)

5.7 Host-based and network-based signatures

Even at the static level the sample yields concrete indicators.

Host-based signatures. The reference to `cmd.exe` (shell execution on the host) and the runtime-loaded `urlmon.dll` are host-level artifacts. The self-deletion is itself a host-based behavioral signature. Registry- and process-level artifacts would be enumerated during dynamic analysis.

Network-based signatures. The hard-coded endpoint `60.248.52.95:443`, the embedded URL `http://www.ueopen.com/test.html`, and the use of port **443** are the network indicators. Together they define what a defender would watch for on the wire.

Indicator	Type	Significance
`60.248.52.95:443`	Network (IP + port)	Hard-coded C2 endpoint; known IOC; port 443 implies TLS
`http://www.ueopen.com/test.html`	Network (URL)	Embedded remote resource
`cmd.exe`, `COMSPEC`, `/c del`	Host (shell command)	Shell execution and self-deletion

Indicator	Type	Significance
`urlmon.dll`	Host (runtime-loaded DLL)	Network capability resolved at runtime
MD5 `4280aac5…` / SHA-256 `84b79…`	File hash	Sample identity for reputation lookup

5.8 What impeded the analysis, and how to overcome it

The principal limitation of basic static analysis is reached here directly: several questions cannot be answered from the file alone. The exact **filename deleted at runtime** is not in the strings because it is built dynamically; whether the **network connection is actually opened** and what data crosses it cannot be confirmed without running the sample; and the role of the runtime-loaded **`urlmon.dll`** is only inferable. These gaps are overcome by the next two phases — **dynamic analysis** (observing the process, its file and registry activity, and its network traffic as it runs, for example with a process monitor filtered to the sample and its children) and **full disassembly** (reading the code that assembles the `SHELLEXECUTEINFO` structure and resolves the runtime imports). Notably, the self-deletion did **not** obstruct the investigation: because the standard discipline is never to run the sample during static analysis, the only on-disk copy was preserved, and the deletion instead became an informative observation rather than a loss of the specimen.

5.9 Purpose of the sample

Assembling every confirmed clue — outbound Winsock connection to a hard-coded `IP:443`, `cmd.exe` launched through `ShellExecuteExA`, standard I/O marshalled through pipes, host identification via `GetComputerNameA`, a Base64-style encoding table, a command-prompt string `*(SY)#`, runtime-loaded `urlmon.dll`, and self-deletion for stealth — the sample is a **network backdoor / remote shell**. It connects to a remote controller, hands the operator an interactive command shell on the infected host with its traffic lightly encoded, and erases its own file to survive only in memory. Every element of this conclusion is a hypothesis grounded in a static observation and would be confirmed, one piece at a time, by dynamic analysis and disassembly.

```
Lexical error on line 10. Unrecognized text.
    M -. runtime-loaded urlmon.dll .-> URL[http://www.ueopen.com/test.html]
------------------------------^
```

---

6. Bridge: From Clean Samples to Packed Samples

`Lab_03-1.exe` was cooperative: standard sections, a readable import table, and informative strings. Not every sample is. **Packing** is a common technique for defeating exactly the basic static analysis performed above. A packed program has had its real code **compressed or encrypted** and wrapped inside a small stub containing only the few routines needed to **load**

the packed body into memory, restore it, and transfer execution to it. From the outside, a packed sample reveals almost nothing: its import table shrinks to a handful of functions (often just `LoadLibrary`, `GetProcAddress`, and memory-allocation calls), its strings dissolve into high-entropy garbage, and its sections carry unusual names and near-maximal entropy. Detecting these tell-tales — anomalous section names, high entropy, a suspiciously small import table — and deciding whether the sample can be unpacked is the subject of the next samples in this lab.

Lab 03 — Basic Static Analysis (Part 2: Samples 2–4)

The first sample was deliberately cooperative — standard sections, a readable import table, informative strings. The remaining three raise the difficulty in two directions: the second introduces a different command-and-control model (HTTP rather than raw sockets, with active camouflage), while the third and fourth introduce **packing**, the technique whose whole purpose is to defeat the basic static analysis performed so far. Each is examined with the same disciplined loop: hashes and reputation, section layout, imports, strings, and the hypotheses they license — never executing the sample.

---

1. `Lab_03-2.exe` — The HTTP Backdoor

An 8,704-byte 32-bit GUI executable. Its fingerprints are:

Property	Value
MD5	`02658bc9801f98dfdf167accf57f6a36`
SHA-256	`8a35842d3f5963f715def0bbd0a53d7ffaae2d2ca79f56a5ac8bede64749d279`
Size	8,704 bytes
Type	PE32 executable (GUI), Intel 80386, 4 sections

1.1 Reputation and compilation

Submitting the MD5 to a reputation service identifies the file as previously seen, classified across engines as a backdoor/downloader. The compiler timestamp resolves to **16 September 2008**; as always, it is treated as indicative rather than authoritative. The section layout is the standard, unremarkable `.text` / `.rdata` / `.data` / `.rsrc` set with moderate entropy — the sample is **not packed**, and its code and strings are available for direct inspection.

1.2 Imports and hypotheses

The import table is dominated by a single, decisive library.

`wininet.dll` — high-level web communication. The sample imports the full WinINet request chain: `InternetOpenA`, `InternetConnectA`, `HttpOpenRequestA`, `HttpSendRequestA` / `HttpSendRequestExA` / `HttpEndRequestA`, `InternetReadFile`, `InternetWriteFile`, `InternetQueryOptionA` / `InternetSetOptionA`, and `InternetCloseHandle`. This is the complete toolkit for driving an HTTP(S) conversation — opening a session, connecting to a server, issuing requests, and streaming data in both directions. Where the first sample built its channel from raw Winsock primitives, this one communicates over **standard web protocols**, which blend into ordinary traffic far more easily.

`kernel32.dll` — shell plumbing and files. `CreateProcessA` together with `CreatePipe` and `PeekNamedPipe`, and `ReadFile` / `WriteFile`, reproduce the familiar pattern of a child process whose input and output are marshalled through pipes — here almost certainly `cmd.exe`, so that commands received over HTTP can be executed and their output captured. `GetWindowsDirectoryA` locates system paths for dropping or staging files.

`msvcrt.dll` — runtime and file I/O. Beyond the usual runtime boilerplate, the C runtime's file functions (`fopen`, `fread`, `fwrite`, `fseek`, `ftell`, `fclose`) indicate the program **reads and writes files on disk** — consistent with a download/upload capability.

The combined hypothesis from imports alone is a **remote-access tool that speaks HTTP, executes shell commands, and transfers files**.

1.3 Strings that stand out

The strings confirm and sharpen every part of that hypothesis, and reveal the sample's camouflage.

- **`69.25.50.10`** — a hard-coded **command-and-control (C2) IP address**, the destination the WinINet calls will target. A concrete network IOC.
- **`wuauclt.exe`** — the filename of the legitimate **Windows Update Automatic Update Client**. Its presence is a strong sign of **masquerading**: the backdoor impersonates (or launches itself under the identity of) a trusted system process so that its network activity looks like routine Windows Update traffic.
- **`Hello.I am here!`** — a **beacon / check-in message**, the announcement a newly active implant sends to its controller to signal that the host is compromised and awaiting instructions.
- **`Begin Download`**, **`Begin Upload`**, **`getf`**, **`putf`**, **`type`** — the backdoor's **command vocabulary**. `getf` / `putf` download and upload files; `type` prints a file; the "Begin Download/Upload" markers frame the transfers. The usage string **`use error! putf [transpeed] [filepath]`** is an operator help message that confirms `putf` takes a transfer-speed and a file path.
- **`cmd /c`** — the prefix for **running an arbitrary command** through the interpreter, tying the `CreateProcessA` /pipe imports to remote command execution.

- `\tasks\` — a directory used to stage **tasks** fetched from the controller.
- `CONNECT` and `HTTP/1.0` — the sample supports the HTTP `CONNECT` method, i.e. it can tunnel through an **HTTP proxy**, allowing it to reach its C2 from inside networks that only permit proxied web access.
- `<head>` / `</head>` — the implant **parses HTML** returned by the server, a common way to smuggle commands inside pages that look benign to a casual observer.
- `Type command disable.Go on!` and `begin` — further protocol/status strings exchanged with the controller.

1.4 What happens when it runs, and its signatures

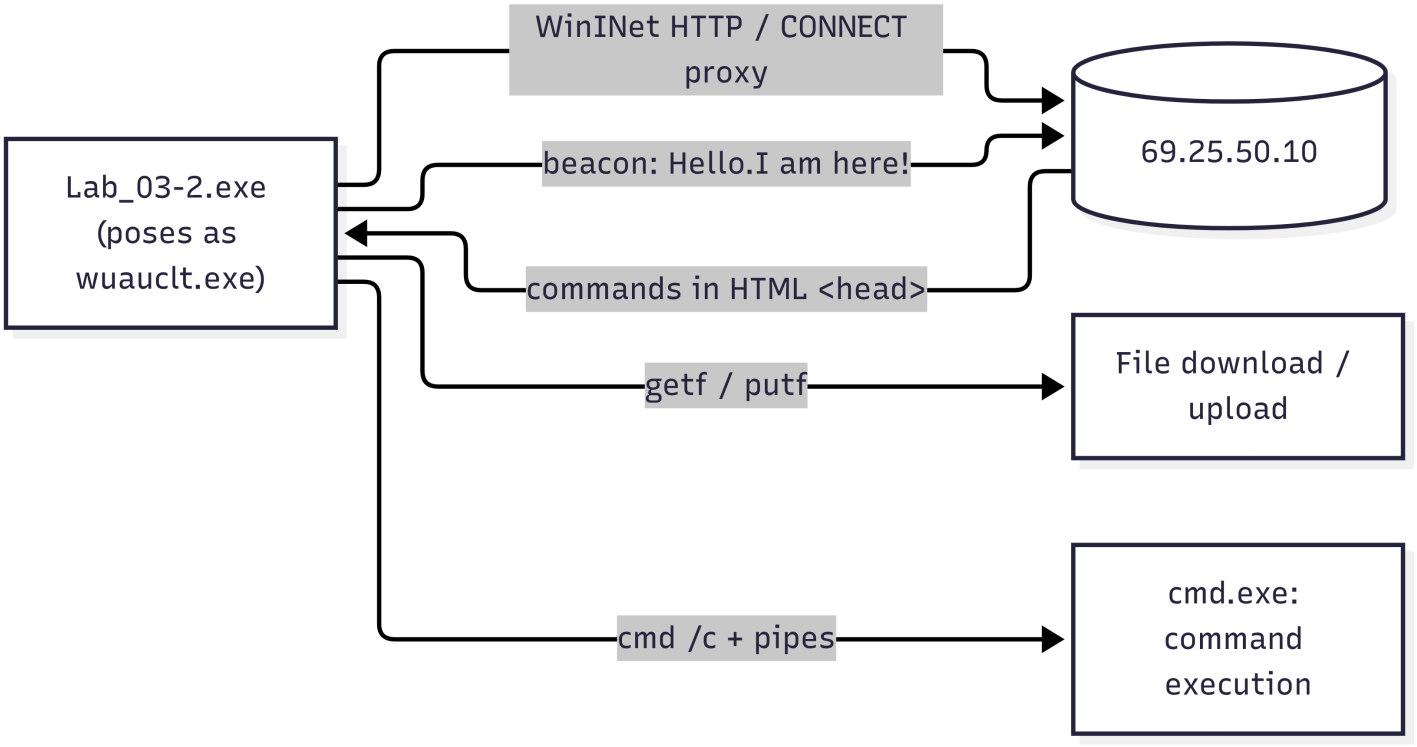
Executed in isolation the program produces no visible window; its activity is entirely network- and file-oriented. Behaviorally it initialises a WinINet session, beacons to `69.25.50.10` with `Hello.I am here!`, and then waits for commands — downloading files, uploading files, or executing shell commands via `cmd /c` and returning their output — while presenting itself as `wuauclt.exe`.

Host-based signatures. The impersonation of `wuauclt.exe`, the `\tasks\` staging directory, the spawning of `cmd.exe`, and files written or read on disk are the host-level indicators. **Network-based signatures** are the C2 address `69.25.50.10`, the beacon string `Hello.I am here!`, the HTTP request structure (including the `CONNECT`/proxy behavior), and the `getf` / `putf` transfer markers on the wire.

On the dynamic questions. Confirming *which* URL path is requested, what the beacon interval is, and whether a proxy is actually used requires running the sample under observation — a process monitor filtered to the sample's process name (and its `cmd.exe` children), plus a network capture. A sensible process-monitor filter isolates events whose *Process Name* is the sample (or `wuauclt.exe`) to cut away the enormous volume of unrelated system activity; a network-focused filter narrows to connections toward `69.25.50.10`. These belong to the dynamic-analysis phase.

1.5 Purpose

`Lab_03-2.exe` is an **HTTP backdoor / remote-access tool**. It beacons to a hard-coded controller over HTTP (tunnelling through a proxy when necessary), disguises its traffic as Windows Update activity by posing as `wuauclt.exe`, and gives its operator file download/upload and arbitrary command execution on the infected host. It is functionally close to the first sample — a remote shell with file transfer — but trades raw sockets for the stealthier, proxy-friendly web-protocol channel.



---

2. `Lab_03-3.exe` — A Packed Sample (Custom Packer)

A 14,336-byte 32-bit **console** executable. Its fingerprints are:

Property	Value
MD5	`6c2a33512b8b0ec906982783e82b9678`
SHA-256	`212b29a8e36ccc8f65205122e33d84940a156a9a91329f747a713f988a157948`
Size	14,336 bytes
Type	PE32 executable (console), Intel 80386, 3 sections

2.1 Is it packed? Packing recognised from the header

Every structural indicator points to **packing**, and together they are conclusive.

Anomalous section names. The sections are named `LOL0`, `LOL1`, and `.rsrc` — the first two are invented names, not the conventional `.text` / `.data` / `.rdata`. Non-standard section names are a hallmark of a packer, which carves the file into its own layout.

A section with virtual size but no raw data. `LOL0` has a virtual size of 28,672 bytes but a **raw (on-disk) size of 0**. An empty-on-disk, large-in-memory section is the **destination** into which the stub will write the unpacked code at runtime: the space is reserved in memory, but there is nothing to store on disk because the real code does not yet exist in cleartext.

A high-entropy data section. `LOL1` carries 12,800 bytes of raw data at **entropy 7.82** (out of a maximum of 8). Near-maximal entropy means the content is **compressed or encrypted**

— this is the packed body of the real program.

Executable *and* writable sections. Both `LOL0` and `LOL1` are marked as simultaneously writable and executable. Legitimate code sections are executable but not writable; a section that is both is a region the program intends to **rewrite and then run**, exactly what an unpacking stub does.

A minimal, revealing import table. Only a handful of functions are imported: `LoadLibraryA`, `GetProcAddress`, `VirtualAlloc`, `VirtualProtect`, `VirtualFree`, and `ExitProcess`. This is precisely the **unpacking stub's** toolkit — allocate memory (`VirtualAlloc`), write the decompressed code into it and make it executable (`VirtualProtect`), then resolve the real APIs at runtime (`LoadLibraryA` / `GetProcAddress`). The genuine imports of the underlying program are hidden inside the packed body and will only materialise after unpacking. The two further imports that *do* leak — `InternetOpenA` (WinINet) and `WSAStartup` (Winsock) — are a tantalising hint that the hidden payload has **network capability**.

The sample is therefore packed, but the packer is **not a standard, recognised one**: the section names `LOL0` / `LOL1` are a bespoke marker rather than the signature of a known tool such as UPX.

2.2 Can it be unpacked?

Not with an automated, format-aware unpacker. A UPX unpacker, for instance, refuses the file outright — it is not UPX, and its sections do not match any tool UPX understands. Because the packer is custom, there is no off-the-shelf utility that recognises its format and reverses it. Unpacking it requires the **manual** approach: run the sample under a debugger in a protected environment, let the stub decompress the real code into memory, stop execution at the **original entry point** (the moment control transfers from the stub to the unpacked program), and **dump** the reconstructed process image from memory — rebuilding its import table afterward. This crosses into dynamic and advanced techniques and lies beyond basic static analysis.

2.3 Strings, and what they tell us

The direct consequence of packing is visible in the strings: there are **almost none that are meaningful**. The readable text dissolves into high-entropy fragments, which is itself the strongest confirmation that the payload is compressed or encrypted. A few artifacts leak from the stub and the resource section — the library names `KERNEL32.DLL`, `WININET.dll`, `WS2_32.dll`; the stub API names; a `http://%s` format fragment and a `Mozilla`-style user-agent remnant suggesting the hidden payload builds HTTP requests; and a short token `KIZXORXZWV` containing the substring **`XOR`**, hinting at an XOR-based decoding step. The scarcity of strings is the finding: a program whose strings have vanished is advertising that it is packed.

2.4 Behaviour, signatures, and purpose

With the real code encrypted, basic static analysis can state only what the structure implies: the leaked `InternetOpenA` and `WSAStartup`, together with the `http://%s` and user-agent fragments, indicate that the unpacked program performs **network communication** — most plausibly another downloader or backdoor contacting a remote host. No concrete host- or network-based signatures (specific IPs, domains, or filenames) can be extracted at this level, because they are inside the packed body. The thing that **impeded the analysis is the packing itself**, and the way to overcome it is to unpack the sample (manually, as above) and then repeat the ordinary static loop on the recovered code. The purpose can only be hypothesised as a **packed network backdoor/downloader**; a definitive answer awaits unpacking.

---

3. `Lab_03-4.exe` — A UPX-Packed Service-Installing Backdoor

A 34,816-byte 32-bit GUI executable. Its fingerprints are:

Property	Value
MD5	`9f880ac607cbd7cdfffa609c5883c708`
SHA-256	`a0dbee0e7238d028e4efd54fa24b94dc10c6a5f488b09ba49b37d90f22845b00`
Size	34,816 bytes (packed)
Type	PE32 executable (GUI), Intel 80386, **UPX compressed**, 3 sections

3.1 Is it packed, and with what?

Yes, and here the packer is a **known** one. The sections are named `UPX0`, `UPX1`, and `.rsrc` — the unmistakable signature of **UPX (the Ultimate Packer for eXecutables)**. As with the previous sample, `UPX0` has a large virtual size (77,824 bytes) but **zero raw data** (the unpack destination), while `UPX1` holds the compressed body at **entropy 7.94** (essentially maximal). The import table of the packed file is correspondingly minimal — `LoadLibraryA`, `GetProcAddress`, `ExitProcess`, and a bare handful more — the UPX stub's toolkit.

3.2 Can it be unpacked? Yes

Because the packer is standard, the sample unpacks cleanly and automatically. UPX stores enough metadata to reverse its own compression, so running the unpacker in decompress mode restores the original file — here expanding it from 34,816 bytes back to **110,592 bytes**, with the normal `.text` / `.rdata` / `.data` / `.rsrc` sections, moderate entropy, and a **full, readable import table and string set** recovered. This is the decisive difference from `Lab_03-3.exe`:

a known packer is trivially reversed, whereas a custom one is not. All of the analysis below is performed on the **unpacked** image.

3.3 Imports and strings after unpacking

The recovered file is far richer than the packed stub suggested and reveals a two-stage design: a **dropper** that installs a persistent **service**.

Dropper / resource extraction (`kernel32.dll`). `FindResourceA`, `SizeofResource`, `LoadResource`, `LockResource`, and `FreeResource` read an embedded **resource**, which `CreateFileA` / `WriteFile` write to disk. The resource stored in the file is **not a plain executable** — its first bytes are not the `MZ` signature but encoded data — so the dropper **decodes the resource at runtime** before writing it out as the payload DLL. `GetFileTime` / `SetFileTime` then **timestomp** the dropped file, copying an existing file's timestamps onto it so the new file does not stand out as recently created. `GetSystemDirectoryA` / `GetWindowsDirectoryA` and `lstrcatA` build the destination path in the system directory.

Service installation and persistence (`advapi32.dll`). `OpenSCManagerA`, `CreateServiceA`, `StartServiceA`, and the registry functions `RegCreateKeyA` / `RegSetValueExA` / `RegOpenKeyExA` install the payload as a **Windows service**. The strings make the mechanism explicit:

- **`Rasmon.dll`** and **`\Rasmon.dll`** — the name of the dropped payload DLL.
- **`ServiceMain`** and **`ServiceDll`** — the DLL exports a `ServiceMain` entry point and is registered as the `ServiceDll` value under the service's **`Parameters`** key.
- **`%SystemRoot%\System32\svchost.exe -k SysIns`** and **`... -k netsvcs`** — the service is configured to run **inside `svchost.exe`** as part of a service group (`SysIns`, added alongside the standard `netsvcs`). Hosting a malicious DLL inside the trusted `svchost.exe` is a classic persistence-and-camouflage technique.
- **`SYSTEM\CurrentControlSet\Services\%s\Parameters`**, **`SYSTEM\CurrentControlSet\Services\%s\Security`**, and **`SOFTWARE\Microsoft\Windows NT\CurrentVersion\SvcHost`** — the exact registry paths written to register the service, its parameters, and its `svchost` group membership.

Capabilities revealed in the recovered strings. The names surfaced after unpacking describe the payload's intended reach:

- **Privilege escalation** — `AdjustTokenPrivileges`, `LookupPrivilegeValueA`, and the privilege names **`SeDebugPrivilege`** and `SeShutdownPrivilege`, used to gain debug rights over other processes.
- **Anti-forensics** — **`ClearEventLogA`**, `OpenEventLogA`, `CloseEventLog`: the ability to **wipe the Windows event logs** and erase traces of the intrusion.
- **System and process reconnaissance** — `CreateToolhelp32Snapshot` (enumerate running processes), `EnumServicesStatusA` (enumerate services),

`GetLogicalDrives` / `GetDriveTypeA` / `GetDiskFreeSpaceA` (enumerate drives), `FindFirstFileA` / `FindNextFileA` (traverse the file system), and reads of `HARDWARE\DESCRIPTION\System\CentralProcessor\0` (fingerprint the CPU).

- **Associated components** — `VedioDriver.dll` (note the characteristic misspelling of "Video"), `acelpvc.dll`, `mdm.exe`, and `\drivers\etc\networks.ics`: additional modules the payload references, consistent with a remote-control / screen-capture capability installed alongside the service.
- **Self-deletion** — a dropped batch file (`%s\DFS.bat`) built around `if exist "%s" goto pp` / `del "%s"` loops until it can delete the original dropper, removing the installer after the service is in place.

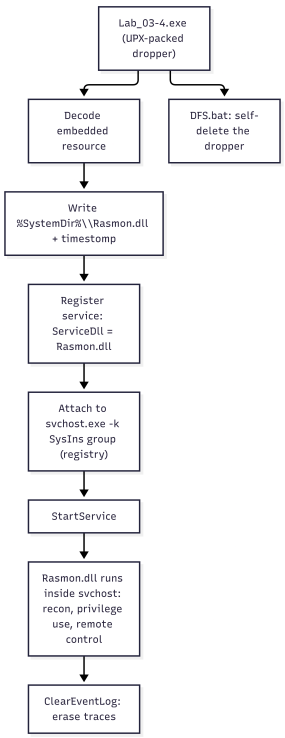
3.4 Behaviour and signatures

On execution the dropper decodes its embedded resource, writes it to the system directory as **`Rasmon.dll`**, timestomps it, registers it as a `svchost`-hosted service (adding the `SysIns` group), starts the service, and deletes itself via `DFS.bat`. From that point the malicious code runs as a legitimate-looking system service under `svchost.exe`.

Host-based signatures: the dropped file `%SystemDir%\Rasmon.dll` (and the associated `VedioDriver.dll`, `acelpvc.dll`, `mdm.exe`); the new service and its registry footprint under `SYSTEM\CurrentControlSet\Services\<name>` with `ServiceDll = ...\Rasmon.dll` and membership in the `SysIns` `svchost` group under `...\SvcHost`; the `DFS.bat` self-deletion artifact; and cleared event logs. **Network-based signatures** are not visible in this dropper stage — the remote-communication logic lives inside the dropped `Rasmon.dll` and would be analysed by extracting and examining that DLL separately.

3.5 Persistence strategy

The sample achieves persistence by **installing itself as a Windows service whose logic is a DLL (`Rasmon.dll`) hosted inside `svchost.exe`**. Registering the payload as a `ServiceDll` under a service's `Parameters` key and attaching it to a `svchost` service group means the operating system **launches the malware automatically at every boot**, as part of a trusted system process, with no file of its own left running in the open. Combined with timestomping, event-log clearing, and self-deletion of the dropper, this makes the implant both durable across reboots and difficult to notice.



---

4. Synthesis Across the Four Samples

The lab's four specimens form a deliberate progression. The first two are **unpacked backdoors** that differ only in their C2 channel — raw Winsock to `60.248.52.95:443` with a lightly encoded shell (sample 1), versus HTTP/WinINet to `69.25.50.10` disguised as `wuauclt.exe` with file transfer and proxy support (sample 2). The last two introduce **packing** as an anti-analysis layer and teach the central lesson of basic static analysis of obfuscated code: recognise packing from its structural fingerprints (anomalous section names, a zero-raw/large-virtual section, near-maximal entropy, executable-and-writable sections, a minimal stub import table), then judge whether the packer is **standard** (UPX — reversed automatically, exposing the full service-installing dropper of sample 4) or **custom** (the `LOL0` / `LOL1` packer of sample 3 — requiring manual unpacking in a debugger). In every case the method is the same: extract what the file's own structure is willing to reveal, form grounded hypotheses, and carry the unanswered questions forward to dynamic analysis and disassembly.

Basic Static Analysis — Exam Quick-Reference Sheet

A scanning aid for the exam: what to look at, in what order, and how to read it. The first half is sample-independent reference (imports → capabilities, packing tells, PE fields, string categories, workflow). The second half is the consolidated IOC tables for the four Lab 03 samples.

---

1. The Workflow, in Order

Step	Question it answers	Where to look
Hashes (MD5 / SHA-256)	Has this been seen before?	PE tool first view → VirusTotal
Compilation timestamp	When was it built?	PE / File header (**treat as a clue, not a fact**)
Sections	Is it packed? What is where?	Sections table (names, raw vs virtual size, entropy)
Libraries + imports	What *can* it do?	Imports/Libraries view → form hypotheses
Strings (ASCII **and** wide)	IPs, URLs, commands, filenames	Strings view → web-search the artifacts
(Unpack if needed)	Reveal hidden code	Automatic for UPX; manual/debugger for custom

> **Golden rules:** never execute the sample during static analysis · signal is rare, noise is huge · every finding is a hypothesis to confirm later · exotic imports matter more than common ones · always run strings in **wide/UTF-16** mode too.

---

2. Imports → Likely Capability

Read the import list as a capability map. The high-value functions to flag:

Import(s)	Library	Suggests
`socket`, `connect`, `send`, `recv`, `WSAStartup`, `inet_addr`, `htons`	ws2_32	Raw socket network comms (backdoor / reverse shell)
`InternetOpenA`, `InternetConnectA`, `HttpOpenRequestA`, `HttpSendRequestA`, `InternetReadFile`	wininet	HTTP(S) C2 / downloader
`URLDownloadToFile`	urlmon	Download a file from a URL
`CreateProcessA` + `CreatePipe` + `PeekNamedPipe` + `ReadFile` / `WriteFile`	kernel32	Shell with redirected I/O (remote shell)
`ShellExecuteExA` / `ShellExecuteA` (+ `SHELLEXECUTEINFO`)	shell32	Launch another program/command (**rare → suspicious**)

Import(s)	Library	Suggests
`cmd.exe` / `COMSPEC` strings + `/c`	—	Run a shell command; `/c del` = self-deletion
`LoadLibraryA` + `GetProcAddress`	kernel32	**Runtime linking** (hides real imports)
`VirtualAlloc` + `VirtualProtect` + `VirtualFree`	kernel32	Allocate+rwx memory → unpacking / injection
`OpenSCManagerA`, `CreateServiceA`, `StartServiceA`	advapi32	Install a **service** (persistence)
`RegCreateKeyA`, `RegSetValueExA`, `RegOpenKeyExA`	advapi32	Registry read/write (config, persistence)
`FindResourceA`, `LoadResource`, `LockResource`, `SizeofResource`	kernel32	Read an **embedded payload** from `.rsrc` (dropper)
`GetFileTime` + `SetFileTime`	kernel32	**Timestomping** (anti-forensics)
`AdjustTokenPrivileges`, `LookupPrivilegeValueA` (+ `SeDebugPrivilege`)	advapi32	Privilege escalation / debug rights
`ClearEventLogA`, `OpenEventLogA`	advapi32	**Wipe event logs** (anti-forensics)
`CreateToolhelp32Snapshot`, `EnumServicesStatusA`, `GetLogicalDrives`	kernel32/advapi32	Host/process/drive reconnaissance
`SetWindowsHookEx`	user32	Keyboard/input hook → **keylogger**
`FindFirstFileA` / `FindNextFileA`	kernel32	File-system traversal
`GetComputerNameA`	kernel32	Identify the victim host

Library at a glance: kernel32 = core OS (ubiquitous) · advapi32 = registry + services · user32/gdi32 = GUI (rare in malware → look twice) · ntdll = low-level/undocumented kernel · ws2_32/wsock32 = sockets · wininet = HTTP · msvcrt = C runtime (**noise, not a signal**).

Context rule: `CreateProcessA`, `CreateThread`, `ReadFile` etc. are common and benign *alone* — treat them as meaningful only once a stronger indicator (an exotic import, an IP, a packer) is already present.

---

3. Linking — Why Some Imports Are Invisible

Type	Resolved	Code visible?	Names visible?	Relevance
Static	Compile time	Yes (embedded)	No	Rare on Windows
Dynamic	Load time (loader → IAT)	No	**Yes** (import table)	The normal case basic static analysis reads
Runtime	At runtime (`LoadLibrary` + `GetProcAddress`)	No	**Not necessarily**	Hides real behaviour → malware favourite

Also: imports **by ordinal** (a number instead of a name) are a light obfuscation — map the number with a public ordinal table (common for ws2_32).

---

4. Packing — Structural Tells (read from the header, no execution)

Tell	Clean file	Packed file
Section names	`.text` / `.rdata` / `.data` / `.rsrc`	Odd names: `UPX0/1`, `LOL0/1`, random
Section raw vs virtual	Comparable	A section with **raw size 0** but large virtual size (unpack target)
Entropy	~4–6	A section near **7.9+** (compressed/encrypted body)
Section flags	code = exec, not writable	Section **executable *and* writable**
Import table	Full, descriptive	**Tiny stub**: `LoadLibrary`, `GetProcAddress`, `VirtualAlloc`, `VirtualProtect`, `ExitProcess`
Strings	Informative	**Almost none** / high-entropy garbage

Can you unpack it? Known packer (UPX → `UPX0/UPX1`) → reverse automatically (`upx -d`). **Disguised** known packer (UPX with sections renamed, e.g. `LOL0/LOL1`, and/or the `UPX!` marker stripped) → identify it by the **stub signature / entropy** in Detect It Easy, **restore the markers** (rename sections back to `UPX0/UPX1`, put the `UPX!` magic back), then

`upx -d` works. Genuinely **custom/unknown** packer → no auto-unpacker; unpack **manually** in a debugger: run to the Original Entry Point, dump memory, rebuild the IAT.

> **UPX disguise tell:** `upx -d` fails with *"file is modified/hacked/protected"* even though the layout (empty first section + high-entropy second section) and the stub epilogue (`… 58 61 8D4424 80 6A00 39C4 75FA 83EC80 E9 … jmp OEP`) are pure UPX → it is UPX wearing a mask, not a custom packer.

---

5. PE Header & Sections — Field Cheatsheet

Field	Normal	Flag when
Image base	`0x00400000` (exe)	—
Entry point	Inside `.text`	Points into an odd/high-entropy section → packing
Timestamp	Plausible date	Null/0 or absurd → stripped or fake (but any value is spoofable)
`.text`	exec, R	writable too → packing/self-modifying
`.data`	raw ≪ virtual (normal)	—
`.rdata`	raw ≈ virtual	—
`.rsrc`	small	**Very large** → embedded payload (dropper)
Exports in an .exe	none	Many exports → also meant to be loaded as a library

---

6. Strings — Categories to Hunt For

Category	Examples to expect	Meaning
Network endpoints	`1.2.3.4` , `:443` , `http://…` , domains	C2 address / URL (**check wide strings!**)
Shell / commands	`cmd.exe` , `COMSPEC` , `/c` , `del` , `> nul`	Command execution / self-delete
Masquerade names	`wuauclt.exe` , `svchost.exe` , `*.dll`	Impersonating trusted processes
Persistence	`...\Services\...` , `Run` keys, `ServiceDll` , `ServiceMain`	Service / registry persistence
Beacons / protocol	`Hello.I am here!` , prompts, `%d` / `%s` format strings	C2 chatter / strings built at runtime

Category	Examples to expect	Meaning
Encoding tables	64-char `A–Za–z0–9+/` (or `-/`) alphabet	Base64 / custom encoding of traffic
Dropped filenames	`Rasmon.dll`, `*.bat`, `VedioDriver.dll`	Files it writes to disk
Format strings	`http://%s`, `%s\%s`	URLs/paths assembled at runtime

> Imported function names also appear in strings — that is expected (the import table is bytes too), **not** a separate finding.

---

7. Host vs Network Signatures (how the exam questions split)

Host-based	Network-based
Dropped files (`Rasmon.dll`, `DFS.bat`)	Hard-coded IP / port
New service + registry keys	Domain / URL
Mutexes, processes spawned (`cmd.exe`)	Beacon strings, request structure
Self-deletion, timestomping, cleared logs	Proxy/`CONNECT`, user-agent

What usually "impedes analysis": packing (unpack it), runtime linking (resolve in a debugger), filenames/targets built at runtime (recover during dynamic analysis), encoding/encryption of traffic (decode in disassembly). **Procmon filter tip:** filter `Process Name is <sample>` (and its children) to cut the noise; add `Operation is …` or a network filter to focus.

---

8. Consolidated IOC Tables — Lab 03 Samples

8.1 Identity & classification

Sample	MD5	Compiled	Packed?	Nature
Lab_03-1.exe	`4280aac55c1d3c327a6c00f0f0085677`	2009-05-14	No	Reverse-she backdoor (s deleting)
Lab_03-2.exe	`02658bc9801f98dfdf167accf57f6a36`	2008-09-16	No	HTTP back (download/ cmd)

Sample	MD5	Compiled	Packed?	Nature
Lab_03-3.exe	`6c2a33512b8b0ec906982783e82b9678`	2011-11-08	**Yes — UPX disguised** (`LOL0/LOL1` , `UPX!` stripped)	Network backdoor/d (unpacks w
Lab_03-4.exe	`9f880ac607cbd7cdfffa609c5883c708`	2009-07-20	**Yes — UPX**	Service-inst dropper (R

SHA-256: 1 = `84b7967aad00e982842045e7b9744af0a457d46bba70456e5f99e7eb9cd783c7` · 2 = `8a35842d3f5963f715def0bbd0a53d7ffaae2d2ca79f56a5ac8bede64749d279` · 3 = `212b29a8e36ccc8f65205122e33d84940a156a9a91329f747a713f988a157948` · 4 = `a0dbee0e7238d028e4efd54fa24b94dc10c6a5f488b09ba49b37d90f22845b00` .

8.2 Network signatures

Sample	Indicator	Notes
Lab_03-1	`60.248.52.95:443`	C2 IP:port — **stored as a wide/UTF-16 string** (ASCII scan misses it); known IOC; 443 ⇒ likely TLS
Lab_03-1	`http://www.ueopen.com/test.html`	Embedded URL (leading `0x01` byte)
Lab_03-2	`69.25.50.10`	Hard-coded C2 IP
Lab_03-2	`Hello.I am here!`	Beacon / check-in string
Lab_03-2	`CONNECT` , `HTTP/1.0`	Proxy tunnelling support
Lab_03-3	`http://%s` , `Mozilla` UA fragment	Hints leak while packed; full C2 artifacts recovered after `upx -d`
Lab_03-4	*(none at dropper stage)*	Network logic lives in dropped `Rasmon.dll`

8.3 Host signatures

Sample	Indicators
Lab_03-1	`cmd.exe` + `COMSPEC` + `/c del` (self-delete) · runtime-loaded `urlmon.dll` · base64-style alphabet (`…0123456789-/`) · prompt `*(SY)#`
Lab_03-2	Masquerades as `wuauclt.exe` · `\tasks\` staging dir · spawns `cmd.exe` · file read/write (`getf` / `putf` / `type`)

Sample	Indicators
Lab_03-3	Few/no strings (**packing is the signature**) · renamed UPX sections `LOL0/LOL1` + UPX stub epilogue · stub imports `VirtualAlloc` / `VirtualProtect` / `LoadLibrary` / `GetProcAddress` · (after `upx -d`: full imports/strings of a network backdoor)
Lab_03-4	Drops `Rasmon.dll` (ServiceDll, exports `ServiceMain`) · service under `SYSTEM\CurrentControlSet\Services\…` · `svchost.exe -k SysIns` group · `DFS.bat` self-delete · timestomping · `ClearEventLog` · refs `VedioDriver.dll`, `acelpvc.dll`, `mdm.exe`

8.4 Key imports per sample

Sample	Signature imports
Lab_03-1	`socket` / `connect` / `send` / `recv` (by **ordinal**) · `ShellExecuteExA` · `CreateProcessA` + `CreatePipe` + `PeekNamedPipe` · `GetComputerNameA`
Lab_03-2	Full WinINet chain (`InternetOpenA` → `HttpSendRequestA` → `InternetReadFil` `CreateProcessA` +pipes · C-runtime file I/O (`fopen` / `fread` / `fwrite`)
Lab_03-3	`LoadLibraryA` + `GetProcAddress` + `VirtualAlloc` + `VirtualProtect` + `Virtu` (stub) · leaked `InternetOpenA`, `WSAStartup`
Lab_03-4 (unpacked)	`FindResourceA` / `LoadResource` / `WriteFile` (drop) · `OpenSCManagerA` / `CreateServiceA` / `StartServiceA` · `RegCreateKeyA` / `RegSe` `GetFileTime` / `SetFileTime` · `AdjustTokenPrivileges`

8.5 Persistence & purpose

Sample	Persistence	Purpose (one line)
Lab_03-1	None on disk (runs in memory, self-deletes)	Remote shell to attacker; encoded traffic
Lab_03-2	None explicit (tasking-based)	HTTP RAT: file transfer + command exec, posing as Windows Update
Lab_03-3	Recovered after UPX unpack	Network backdoor/downloader hidden under a disguised-UPX wrapper
Lab_03-4	**Windows service DLL under `svchost`** (auto-start at boot)	Dropper installing a persistent RAT with recon + anti-forensics

CHAPTER 3: Basic Dynamic Analysis

Basic Dynamic Analysis

Scope of this chapter This chapter covers basic dynamic analysis: the practice of learning what a program does by *running it under observation* in

a controlled environment, rather than by reading its code. It assumes that basic static analysis (hashing, inspecting imports, extracting strings, examining the PE structure) has already been performed and has produced a set of *hypotheses* about the sample's behaviour. Dynamic analysis is the step that confirms, refutes, or extends those hypotheses by watching the sample act.

The methodology is illustrated throughout with four progressively harder samples (`Lab_03-1.exe` … `Lab_03-4.exe`). Each sample is analysed against the nine standard questions that frame every basic-analysis report.

1. Why dynamic analysis is needed

Static analysis answers *what a program could do*; dynamic analysis answers *what it actually does*. The two are complementary, and the richest, fastest understanding comes from using static findings to steer dynamic observation.

The central difficulty is that malware is engineered to be silent. A well-written sample runs with no window, no message, and no obvious effect, precisely so that a victim notices nothing. Visible symptoms are the exception: a dialog that flashes and vanishes before it can be read, or — the one reliably loud category — a ransomware ransom note. In the ordinary case, executing the sample and simply *watching the screen* reveals nothing at all. Observation therefore has to be instrumented: the analyst interposes monitoring tools between the malware and the operating system so that every file, registry, process, and network operation the sample performs is recorded for later inspection.

A second difficulty is containment. Running live malware on a normal machine with a normal internet connection is reckless: the sample may spread, may contact a live command-and-control (C2) server, or may receive instructions. Dynamic analysis is therefore always carried out inside an isolated, disposable environment, and network access is *simulated* rather than *granted*.

---

2. The laboratory environment

Dynamic analysis is performed inside a **virtual machine** dedicated to the task, for three reasons: the VM can be isolated from the host and from the real network; it can be returned to a known-clean state in seconds by restoring a **snapshot**; and the inevitable damage caused by executing the sample is confined to a throwaway system.

The working discipline that follows from this is strict. A clean snapshot is taken before any sample is run. The sample is executed, observed, and its artefacts collected. The VM is then reverted to the clean snapshot before the next sample — or the next run of the same sample

— so that every experiment starts from an identical baseline and the artefacts of one run never contaminate another.

> ⚠ **Always run a copy, never the original Several samples delete, move, or overwrite themselves on execution (see Lab_03-1.exe — self-deleting backdoor). Keep the pristine sample untouched and execute a copy (for example `Lab_03-1 - Copy.exe`). If the copy destroys itself, the original is still available for the next attempt.**

2.1 Network posture

Two network postures are used in sequence, and the distinction matters for interpreting results.

With the **network adapter disabled**, the machine has no connectivity at all. This is the safest posture and is ideal for isolating a sample's *host* behaviour. It has a side effect that is itself diagnostic: a sample that checks for connectivity before acting will query the Windows Sockets layer, be told there is no adapter, and silently skip all of its network logic. The absence of any network events under these conditions is therefore not proof that the sample is harmless on the network — it may simply mean the sample refused to run its network code because no adapter was present.

With a **simulated network**, connectivity appears to exist but all traffic is intercepted locally and never reaches the real internet. This is the posture in which network signatures are actually captured, and it is provided by FakeNet-NG (see 5. FakeNet-NG — simulating the internet).

---

3. The dynamic analysis toolkit

Four tools carry the basic dynamic workflow, each answering a different question. A convenient way to remember their division of labour:

Tool	Answers	Nature of the view
Process Explorer	*What is running right now, and how are processes related?*	Live snapshot of the current process tree
Process Monitor	*What file, registry, process and network operations occurred?*	Continuous, time-ordered event log
FakeNet-NG	*What does the sample try to do on the network?*	Intercepted DNS and connection log
Detect It Easy	*Is the sample packed, and how?*	Packer / compiler fingerprint

A fifth tool, **PEStudio**, belongs to static analysis but is the natural bridge into the dynamic phase: it produces the initial assessment — imports, strings, PE indicators, and a hash that can be looked up online — from which the hypotheses to be tested are drawn.

3.1 PEStudio — the initial assessment and the web lookup

Before a sample is ever executed, its imports and strings have already suggested what it might do. One further static step should always precede execution: take the sample's hash (MD5 or SHA-256) and search for it on the web and on multi-engine services. A known sample will usually have public reports describing its behaviour, which both saves effort and provides an independent check on the analyst's own conclusions. For the first lab sample this lookup returns a high detection rate across engines and numerous existing write-ups, confirming before any execution that the file is indeed malicious.

> **Hash first, run second** Looking up the hash is free, fast, and risk-free, and it may hand you the entire answer. Make it a reflex at the start of every dynamic analysis.

---

4. Process Explorer

Process Explorer is best understood as the Windows Task Manager "on steroids": it lists running processes, but with far more information and, crucially, with the **parent–child hierarchy** made explicit.

4.1 Reading the process tree

The hierarchy itself teaches what *normal* looks like, which is a prerequisite for spotting the abnormal. On a healthy system `services.exe` hosts a large number of `svchost.exe` children — this is entirely expected, because `svchost.exe` is the standard host process under which many Windows services run, so multiple instances are the rule rather than a red flag. Likewise, `explorer.exe` is the parent of the interactive desktop's child processes, and whenever a command prompt (`cmd.exe`) is running it is normally accompanied by `conhost.exe` , the console host support process. `conhost.exe` is one of the most common false positives for beginners: it is spawned legitimately by the use of the console and is not, by itself, evidence of malware.

4.2 Per-process detail

Double-clicking a process opens a properties window whose tabs answer specific questions:

- **Image** — the on-disk path the process was launched from, parent, command line, and start time.
- **Threads** — the threads currently executing inside the process.

- **TCP/IP** — any network endpoints the process currently holds open.
- **Strings** — strings found in the image on disk and in memory (the two can differ for a packed sample, which is diagnostically useful later).

The row colours encode process categories. The practically important ones for analysis are:

Colour	Meaning
Light blue	Processes running as the same user as Process Explorer (your own processes)
Pink	Processes that are hosting Windows **services**
Purple	Images flagged by a heuristic as **packed / compressed** — worth a closer look
Green / Red	Processes **starting / exiting** (shown briefly as the event occurs)

4.3 The fundamental limitation

Process Explorer, like Task Manager, shows only the *present instant*. It does not record history. This makes it almost useless against a sample that starts and exits immediately: the malware's process flashes into the tree and is gone before it can be inspected. It remains valuable in two situations — a sample that keeps running after launch (so there is something persistent to inspect), and later topics such as **process injection**, where it is used to spot a new thread that the malware has injected into an otherwise legitimate, long-lived process. For capturing the behaviour of a short-lived sample, however, a tool that *records continuously* is required.

---

5. Process Monitor

Process Monitor (Procmon) is the core instrument of basic dynamic analysis. Where Process Explorer takes a snapshot, Procmon **records every low-level operation** performed at the user level — file, registry, process/thread, and network activity — as a continuous, time-ordered log.

Procmon requires administrator privileges, because it installs a driver to observe system-wide activity, and it begins capturing the moment it starts. The event rate is enormous: within a few seconds of running it on an idle machine the log already holds well over a hundred thousand events. The capture is therefore toggled on and off deliberately (the capture button, a magnifying glass, in the toolbar), and the list is cleared to a blank slate before an experiment.

5.1 Anatomy of an event

Each line in the log is a single operation, with these principal columns:

- **Time of Day** — the timestamp; rarely important for analysis, but it preserves ordering.
- **Process Name** and **PID** — which process generated the event.
- **Operation** — the operation performed, shown both as text (e.g. `CreateFile`) and as a category **icon**. The icon groups operations into families — file-system, registry, process/thread, network — so that `CreateFile` carries a file icon while `Thread Exit` carries a process/thread icon.
- **Path** — the target of the operation; its meaning depends on the operation (a file path, a registry key, a network endpoint…).
- **Result** — the outcome (`SUCCESS`, `NO MORE ENTRIES`, `BUFFER OVERFLOW`, and so on).

Double-clicking an event opens further detail, including the **call stack** at the moment the operation occurred and information about the originating process.

> ✎ **`CreateFile` does not mean "a file was created" `CreateFile` is the Windows API used to open a handle to a file (or device), not necessarily to create a new one. The *Detail* column resolves the ambiguity by showing the desired access — for example `Read Data / List Directory / Read Attributes / Synchronize` for a plain open, versus `Delete` when the handle is opened in order to remove the file. Always read the desired access before concluding what a `CreateFile` actually did.**

5.2 Filtering: turning noise into signal

The log's usefulness depends entirely on filtering, because the events generated by the sample are buried among hundreds of thousands of unrelated ones. Procmon ships with a set of **predefined filters** that exclude its own activity and that of the other analysis tools; these should be left in place.

Filters are defined over a column, a relation, a value, and an action (**Include** or **Exclude**). Two properties of the filter engine must be kept in mind:

1. **Filters combine as a conjunction (logical AND).** An event is displayed only if it satisfies *every* active filter. A single unsatisfied condition hides the event. (Hence a careless `Include → PID is 12345` on a run where no such PID existed empties the list entirely.)
2. **Excluding hides, it does not delete.** A filtered-out event is merely not displayed; disabling the filter in the Filter dialog brings it back. Nothing is lost.

Filters can be built in two convenient ways. From the Filter dialog, a rule is composed explicitly and added to the list. From the log itself, right-clicking a value offers **Exclude** (or Include) directly — excluding by **PID** removes one specific process instance, while excluding by **Process Name** removes *all* instances of that image, present and future.

The decisive choice for malware work is to **include by process name rather than by PID**. The PID changes on every execution; the process name does not. A filter of `Process Name is`

`Lab_03-1.exe → Include` therefore keeps working across repeated runs without being rebuilt each time.

A complementary device is the toolbar's category quick-filter: a row of icons that toggle whole operation families (registry, file system, network, process/thread, profiling) on and off. Hiding the categories that are irrelevant to the current question — and un-hiding them afterwards so nothing is permanently lost — is the fastest way to read a dense log.

> ***A worked filter (from the Lab03-1 capture) The Filter dialog during the Lab_03-1 analysis combined an Include on the malware's process instances (by PID while live, better replaced by _Process Name is Lab_03-1.exe) with the standard Exclude set: `Procmon.exe`, `Procexp.exe`, `Procmon64.exe`, `Procexp64.exe`, `Autoruns.exe`, `System`, `ctfmon.exe`; operations whose name begins with `IRP_MJ_` or `FASTIO_`; results beginning with `FAST IO`; and paths ending in NTFS metadata files (`pagefile.sys`, `$Mft`, `$MftMirr`, `$LogFile`, `$Volume`, `$AttrDef`, `$Root`, `$Bitmap`, `$Boot`, `$BadClus`). The first rule isolates the sample; the rest suppress tool and filesystem noise.***
>
> [Photo: Process Monitor Filter dialog for Lab_03-1] — a two-pane dialog: at the top, a rule being composed (*Column = Process Name, Relation = is, Value = Lab_03-1.exe, Action = Include*); below, the scrolling list of active rules with the Include rule(s) for the sample above the inherited Exclude rules for the analysis tools and NTFS metadata.

5.3 The capture workflow

The procedure that makes Procmon reliable is a fixed sequence:



Even a two-to-three-second window of capture around the launch yields a complete record of the sample's activity while keeping the log small enough to read.

---

6. FakeNet-NG — simulating the internet

With host behaviour captured, the remaining hypotheses concern the network. Granting the sample real internet access is unacceptable, and disabling the adapter suppresses the very behaviour to be observed. **FakeNet-NG** resolves the dilemma: it simulates a working internet entirely on the local machine.

FakeNet-NG runs from an administrator command prompt (from its own folder, e.g. `fakenet-ng-1.4.11`). Once started, it **intercepts** outbound traffic and **diverts** every connection the machine attempts toward its own listeners, which answer with standard, plausible responses. The effect is twofold: the sample perceives an active, responsive network and therefore runs its network logic, while no packet ever leaves the machine. Every DNS query and connection

attempt is logged, so the sample's intended destinations are revealed even when the real servers are long dead and answer nothing. FakeNet-NG can additionally record intercepted traffic to a **PCAP** file, which can later be opened in **Wireshark** to inspect the full content of each connection.

> Capturing Lab*03-1's network signature Visiting any URL in a browser while FakeNet-NG runs returns FakeNet's own placeholder page instead of the real site — a quick confirmation that interception is working. Running the sample then produces, in the FakeNet log, a line of the form:* `Diverter Lab_03-1-copy.exe (PID) requested TCP 60.248.52.95:443` *This matches the URL hard-coded in the sample's strings (* `http://www.ueopen.com/test.html` *). The connection does not appear in Process Monitor: the TCP three-way handshake never completes (the real host is unreachable and FakeNet returns no application response), so the operation is an _attempt* only and is not logged as a completed network event. FakeNet nonetheless captures the destination, which is the signature sought.
>
> [Photo: FakeNet-NG Diverter log] — a console of green-on-black log lines; among the browser "noise" (DNS A requests for `www.google.com`, `accounts.google.com`, etc.) a single diagnostic line shows the sample's own process name requesting an outbound TCP connection to the hard-coded address on port 443.

> ⚠ **FakeNet changes your DNS setting — revert it To intercept name resolution, FakeNet-NG writes a static DNS server into the adapter's *TCP/IPv4* properties. If FakeNet is stopped without undoing this, ordinary browsing breaks, because the machine keeps trying to resolve names through a DNS server that is no longer listening. After finishing, open *Network Connections → adapter → Properties → Internet Protocol Version 4 → Properties* and restore "Obtain DNS server address automatically." (This, not any "deleted network driver," is the usual cause of "I lost internet after running FakeNet.")**

---

7. Dealing with packed samples

Some samples resist both static and dynamic inspection because they are **packed**: the real code is stored compressed (or encrypted) on disk and is only unfolded into memory at runtime by a small bundled *unpacking stub*. Two signatures betray packing, and both are visible with the basic tools:

1. **A tiny import table.** A packed file imports only the handful of functions the stub needs — characteristically `LoadLibraryA`, `GetProcAddress`, `VirtualAlloc`, `VirtualProtect`, `VirtualFree` — because the real imports are reconstructed at

runtime, after unpacking. The absence of the APIs implied by the sample's purpose is itself the clue.

2. **An executable section with no on-disk content.** A section whose *virtual size* is substantial (tens of kilobytes) but whose *raw size* on disk is zero is an empty, writable, executable region: a place for the stub to write the unpacked code and then jump into it. Non-standard section names reinforce the suspicion.

Detect It Easy (DIE) confirms and names the packer. It fingerprints the file and reports both the compiler/linker and the packer in use. For the lab samples it reports **UPX** — a legitimate, open-source executable packer — but with an important qualifier for the third sample: `UPX(-)[modified]`.

The distinction between *standard* and *modified* UPX is the practical crux:

- **Standard UPX** (as in `Lab_03-4.exe`) produces the canonical section names `UPX0` / `UPX1`. It can be reversed with the UPX utility itself: `upx -d packed.exe` writes a new, fully unpacked executable, which is then analysed with ordinary static techniques.
- **Modified UPX** (as in `Lab_03-3.exe`, whose sections are renamed `LOL0` / `LOL1`) has been deliberately altered — a few bytes changed, the section names rewritten — specifically so that the stock `upx -d` refuses to process it. The file is still UPX-compressed, but the standard unpacker reports that it is non-compliant and fails. Unpacking it requires manual work (a debugger, dumping the process after the stub has run) rather than a one-line command.

> **Unpack, then re-apply static analysis** Packing only delays analysis; it does not prevent it. Once an unpacked executable exists — produced by `upx -d` for standard UPX, or by manual unpacking for a modified packer — the full static toolkit (imports, strings, PE structure) applies to it unchanged, and the real behaviour becomes visible.

---

8. The nine-question report framework

Every basic-analysis report answers the same nine questions. They are the scaffold for the worked examples that follow, and the checklist to apply to any new sample.

1. **When was this file compiled?** (PE compilation timestamp — noting it can be forged.)
2. **List a few imports (or sets of imports) and describe how the malware might use them.**
3. **What are a few strings that stand out, and why?**
4. **What happens when you run the malware? Is it what you expected, and why?**
5. **Name a Procmon filter and why you used it.**

6. **Are there host-based signatures** (files, registry keys, processes, services)? **If so, what are they?**
7. **Are there network-based signatures** (URLs, IPs, packet contents)? **If so, what are they?**
8. **Is there anything that impeded your analysis? How, and how might you overcome it?**
9. **What do you think is the purpose of this malware?**

> ✎ **Static imports and strings carry most of the answer The majority of usable behavioural information comes from the imported functions and the embedded strings. An analyst who can make well-reasoned guesses from those is already halfway to understanding the sample; disassembly is then reserved for confirming *how* each behaviour is implemented, not for discovering *that* it exists. Basic tools look naive but are indispensable — exhaustively reading the disassembly of a real sample within the time available for triage is simply not feasible, so the fast, basic view comes first.**

---

9. Worked example — Lab_03-1.exe (self-deleting backdoor)

9.1 Static assessment (recap)

`Lab_03-1.exe` is a 32-bit GUI PE with four sections, compiled **2009-05-14 17:12:41 UTC**. Its imports already tell most of the story. From `WS2_32.dll` it pulls the full Winsock client set — `WSAStartup`, `socket`, `connect`, `send`, `recv`, `inet_addr`, `htons`, `closesocket` — indicating a network client. From `KERNEL32.dll` it pulls `CreateProcessA` together with `CreatePipe`, `PeekNamedPipe`, `DuplicateHandle`, `ReadFile` and `WriteFile`, the classic combination for launching a child process and shuttling its standard input and output through pipes — in other words, a **remote shell**. `GetComputerNameA` suggests the victim is identified to the attacker. `CreateEventA` is a candidate single-instance / mutex mechanism. From `SHELL32.dll` it imports `ShellExecuteExA` and `SHChangeNotify`.

The strings sharpen the hypotheses: `http://www.ueopen.com/test.html` (a hard-coded URL), `cmd` and `cmd.exe`, the fragment `/c del` together with `> nul` and `COMSPEC` (a self-contained command-line deletion), `send = %d` (a format string for reporting bytes sent), the full Base64 alphabet, and a reference to `urlmon.dll` (the home of `URLDownloadToFile`, loaded dynamically). The leading characters on the URL string are a disassembly/offset artefact and should be read as the URL `http://www.ueopen.com/test.html`.

9.2 Dynamic observation

Executed as a copy, the sample shows **no visible activity whatsoever**. Checking the directory afterwards reveals why the screen stayed blank: **the executed copy has deleted**

itself — it is no longer in the folder. This is consistent with the `/c del` and `ShellExecuteExA` findings from the strings.

Under Process Monitor, filtered to the sample and read in order, the chain is explicit. The launching `cmd.exe` first issues a `CreateFile` against the sample's path (desired access *Read Data / List Directory / …*, i.e. opening it), then a **Process Create** whose result is the child PID — the newly spawned malware process. The child's first operations are a run of **Load Image** events: the OS loader mapping the sample's own sections, then its imported libraries (`kernel32` , `user32` , `shell32` , `msvcrt` , `ws2_32`), followed by a much larger set of further DLLs. That larger set is not evasion; it is the natural consequence of (a) the **dependency cascade** — each loaded library pulls in the libraries *its* functions need — and (b) the **WOW64 translation layer**, since a 32-bit sample on 64-bit Windows loads `ntdll.dll` , the `wow64*.dll` set, and 32-bit library variants to bridge 32-bit calls onto the 64-bit kernel. A 64-bit sample would show far fewer.

Filtering to network operations shows **nothing** — because the adapter was disabled, Winsock reports no network, and the sample skips its network code. Filtering instead to process and command-line activity exposes the self-deletion directly: the sample issues a **Process Create** for `cmd.exe` whose **command line** is `cmd.exe /c del <path-to-sample> > nul` , and among the subsequent `CreateFile` operations on its own file is the final one opened with desired access **Delete**. The hypothesis built from the strings is thus confirmed by observation.

Re-running with FakeNet-NG supplies the network signature: the sample attempts an outbound **TCP connection to 60.248.52.95:443** (the address behind `www.ueopen.com`), captured by FakeNet even though the handshake never completes and the attempt never reaches Procmon's completed-event log.

9.3 Report answers

#	Question	Answer for `Lab_03-1.exe`
1	Compiled	2009-05-14 17:12:41 UTC (timestamp; may be forged)
2	Imports	Winsock client (`socket` / `connect` / `send` / `recv`) → C2 channel; `CreateProcessA` + `CreatePipe` + `PeekNamedPipe` + `DuplicateHandle` → piped remote shell; `GetComputerNameA` → victim ID; `ShellExecuteExA` → launch helper command; `CreateEventA` → single-instance marker; `urlmon` (dynamic) → file download
3	Strings	`http://www.ueopen.com/test.html` ; `cmd.exe` / `/c del` / `> nul` , `COMSPEC` ; `send = %d` ; Base64 alphabet; `urlmon.dll`
4	On execution	No visible effect; the executed copy deletes itself — expected, given the `/c del` + `ShellExecuteExA` evidence
5	Procmon filter	`Process Name is Lab_03-1.exe → Include` (survives re-runs), plus category toggles to isolate process/file/network events
6	Host-based signatures	Spawns `cmd.exe /c del <self> > nul` ; opens its own file with **Delete** access; `CreateEventA` single-instance event

#	Question	Answer for `Lab_03-1.exe`
7	Network-based signatures	`www.ueopen.com`, path `/test.html`; outbound TCP to **60.248.52.95:443**
8	Impediments	**Self-deletion** (work on a copy; Process Explorer is too slow to catch the short-lived process) and **disabled network** (no network events until FakeNet is used; attempted connections may not reach Procmon)
9	Purpose	A **backdoor / remote-access tool**: beacons to a hard-coded C2, can download a file (`urlmon`) and open a piped `cmd` shell for remote control, and deletes its dropper to minimise forensic traces

---

10. Lab_03-2.exe — HTTP backdoor masquerading as Windows Update

`Lab_03-2.exe` (32-bit GUI PE, compiled **2008-09-16**) moves the C2 channel from raw sockets to **HTTP**. Its imports come from `WININET.dll`: `InternetOpenA`, `InternetConnectA`, `HttpOpenRequestA`, `HttpSendRequestA` / `HttpSendRequestExA`, `InternetReadFile`, `InternetWriteFile`, `HttpEndRequestA`. Alongside these it retains `CreateProcessA` + `CreatePipe` + `PeekNamedPipe` (a command shell) and `GetWindowsDirectoryA` (a destination for a self-copy).

The strings expose the rest: a hard-coded IP **`69.25.50.10`**, `HTTP/1.0`, the command tokens `D-o-w-n-l-o-a-d-f-i-l-e…` and `U-p-l-o-a-d-f-i-l-e…` (interleaved to evade naive string scanning), `cmd /c`, the usage hint `use error! putf [transpeed] [filepath]`, and — most telling — **`wuauclt.exe`**, the name of the genuine Windows Update Automatic Update client. The sample masquerades as that process so that a casual look at the process list raises no alarm.

Its expected signatures are therefore a process named `wuauclt.exe` running from an unusual location (e.g. the Windows directory rather than System32) and spawning `cmd`, on the host side; and HTTP traffic to `69.25.50.10` carrying file-download, file-upload, and command instructions, on the network side. Its purpose is an **HTTP-based backdoor / bot** that downloads and uploads files and executes commands on request, hiding behind a trusted-sounding name. *(This sample is recommended for independent practice; apply the full workflow and the nine questions to it.)*

---

11. Lab_03-3.exe — a packed sample (modified UPX)

`Lab_03-3.exe` (console PE, compiled **2011-11-08**) is the first sample where packing dominates the analysis. It has only **three** sections, named **`LOL0`** / **`LOL1`** / **`.rsrc`** rather

than the usual four, and `LOL1` has a virtual size of roughly 28 KB with a **raw size of zero** — an empty, readable, executable region into which code will be unfolded at runtime. Its import table is strikingly small: from `KERNEL32` essentially `LoadLibraryA`, `GetProcAddress`, `VirtualProtect`, `VirtualAlloc`, `VirtualFree`, `ExitProcess`; a single `InternetOpenA` from `WININET`; and `WSAStartup` from `WS2_32`. Both signatures of packing — the stub-sized import table and the empty executable section — are present.

Detect It Easy confirms the diagnosis and adds the decisive detail: the packer is **`UPX(-) [modified]`**. The author took the open-source UPX packer and altered it (renaming `UPX0` / `UPX1` to `LOL0` / `LOL1`, among other changes) precisely so that the standard `upx -d` refuses the file and reports it as non-compliant. The one genuine *network* hint that survives packing is the `http://%s` format string plus `InternetOpenA` and `WSAStartup`, indicating that the unpacked payload will build and contact a URL.

The impediment here is packing by a **modified** packer: the stock unpacker will not work, so the file must be unpacked manually (running the stub under a debugger and dumping the process once the real code is in memory) before static analysis can resume and before dynamic behaviour — the payload's own actions rather than the stub's — can be meaningfully attributed. Until then the sample's full purpose remains undetermined by basic tools; what can be stated is that it is a packed, network-capable sample deliberately hardened against routine unpacking.

---

12. Lab_03-4.exe — a service installer (standard UPX)

`Lab_03-4.exe` (32-bit GUI PE, compiled **2009-07-20**) is packed with **standard UPX** — sections `UPX0` / `UPX1` / `.rsrc` — and therefore yields to a one-line unpack: `upx -d Lab_03-4.exe` produces a clean, roughly 110 KB executable to which ordinary static analysis applies. (A copy of UPX, version 3.93, is provided in the lab's `tools` folder.)

The unpacked file is a **Windows service installer**. From `ADVAPI32.dll` it imports the service-control set — `OpenSCManagerA`, `CreateServiceA`, `StartServiceA`, `ControlService`, `DeleteService`, `ChangeServiceConfigA` — together with the registry functions `RegCreateKeyA`, `RegSetValueExA`, `RegOpenKeyExA`. From `KERNEL32` it imports the resource functions `FindResourceA`, `LoadResource`, `LockResource`, `SizeofResource` with `CreateFileA` / `WriteFile`: the executable carries an **embedded payload in its resources and writes it to disk**. The strings name the pieces: `%SystemRoot%\System32\svchost.exe -k SysIns` and `… -k netsvcs` (the service is hosted inside `svchost.exe` under a service group), the dropped DLL **`Rasmon.dll`**, a dropped batch file **`DFS.bat`**, the registry sub-keys `\Parameters` and `\Security`, the account `LocalSystem`, the hardware key `HARDWARE\DESCRIPTION\System\CentralProcessor\0`, and a hard-coded IP **`192.168.5.164`**.

The host-based signatures are therefore the installation of a new service that runs under `svchost.exe -k SysIns`, the files `Rasmon.dll` and `DFS.bat` dropped into the system directory, and the service's registry entries (notably a `ServiceDll` under `\Parameters`). The network-based signature is the hard-coded `192.168.5.164`. The executable itself is only the **installer and persistence mechanism**; the actual malicious logic lives in the dropped `Rasmon.dll`, which `svchost.exe` loads as a service — so complete analysis continues by examining that DLL. The only impediment is the UPX layer, trivially removed. *(This sample is also recommended for independent practice.)*

---

13. Signatures at a glance

Sample	Packing	Host-based signatures	Network-based signatures	Purpose
Lab_03-1	None	`cmd /c del <self> > nul`; self-file opened with Delete access; `CreateEventA` marker	`www.ueopen.com/test.html`; TCP `60.248.52.95:443`	Self-deleting backdoor (download + piped cmd shell)
Lab_03-2	None	Process `wuauclt.exe` from unusual path; spawns `cmd`; self-copy to Windows dir	HTTP to `69.25.50.10`; Download/Upload-file commands	HTTP bot masquerading as Windows Update
Lab_03-3	**Modified UPX** (`LOL0` / `LOL1`)	*Undetermined until unpacked*	`http://%s` + `InternetOpenA` (payload builds a URL)	Packed, network-capable payload hardened against unpacking
Lab_03-4	**Standard UPX** (`upx -d`)	New service via `svchost.exe -k SysIns`; drops `Rasmon.dll`, `DFS.bat`; service registry keys	Hard-coded IP `192.168.5.164`	Service installer / persistence; payload in `Rasmon.dll`

Host-based vs network-based signatures Host-based signatures are artefacts the sample leaves *on the machine* — created or deleted files, registry keys, mutexes/events, installed services, child processes and their command lines. Network-based signatures are artefacts *on the wire* — domains, URLs, IP addresses, ports, User-Agent strings, and packet contents. The former are gathered chiefly with Process Monitor; the latter chiefly with FakeNet-NG (and, for packet contents, a FakeNet PCAP opened in Wireshark). A good report supplies both, because defenders use host signatures for endpoint detection and network signatures for perimeter detection.

---

14. Exam quick-reference

The basic dynamic analysis loop

1. **Static first** — hash-lookup, imports, strings → form hypotheses.
2. **Snapshot** the clean VM. Work on a **copy** of the sample.
3. **Process Monitor**: stop → clear → enable capture → run copy → stop capture → filter `Process Name is <sample> → Include` → read by category, follow the process chain → record host signatures.
4. **FakeNet-NG** (admin): run it, execute the sample again, read the Diverter log for domains/IPs/ports → record network signatures. **Revert the adapter's DNS to automatic afterwards.**
5. If imports are tiny / a section is empty+executable / section names are odd → **packed**. Confirm with **Detect It Easy**; `upx -d` for standard UPX, manual unpack for modified UPX; then re-run static analysis on the unpacked file.
6. **Revert the snapshot** before the next run. Answer the **nine questions**.

Fast heuristics

- No visible effect on run is normal — instrument, don't stare at the screen.
- `CreateFile` = *open a handle*; read the **desired access** in Detail.
- Filter by **Process Name**, not PID (PID changes every run).
- Procmon filters are **AND**; Exclude **hides**, it does not delete.
- **Empty network log** can mean *adapter disabled*, not *no network behaviour*.
- `conhost.exe` beside `cmd.exe`, and many `svchost.exe` under `services.exe`, are **normal**.
- Tiny import table + empty executable section = **packed**; most real import/string intelligence returns only after unpacking.

CHAPTER 4: Advanced Static Analysis

This chapter develops a complete, repeatable workflow for understanding a compiled Windows executable using **advanced static analysis** alone — that is, without ever executing the program under a debugger. The vehicle for the lesson is a small console application, `Bomb.exe`, a "binary bomb": a program structured as a sequence of challenges ("phases") that each read a line of input and either accept it and move on, or *detonate* — printing `BOOM!!!` and terminating. The task is to recover, purely by reading disassembled machine code, exactly what input each phase expects.

The exercise is valuable precisely because the program looks more complicated than it is. The central skill being trained is the ability to take an apparently opaque blob of assembly and, through disciplined reasoning, reconstruct its logic, its data structures, and its expected inputs. Everything below is derived from the bytes of the binary itself; where a diagram or schema would normally appear, its logic has been folded directly into the prose.

> ⓘ **Sample identity `Bomb.exe` — PE32 executable (console), Intel 80386, 5 sections, 13 312 bytes, compiled with Microsoft Visual C++ 9.0 (Visual Studio 2008). It is not malicious; it is a Windows port of the classic Carnegie Mellon University "Bomb Lab." Write-ups for the Linux original are plentiful, but the Windows build differs in its concrete solutions, so the answers below are specific to *this* binary.**

---

1. The analysis mindset

Before any tool is opened, two principles frame the work.

Static first, execution later. The entire point of the exercise is to demonstrate that a program's behaviour and its expected inputs can be deduced from its code without running it. Dynamic analysis — stepping through with a debugger, inspecting live registers and memory — is powerful and will eventually make several of these phases trivial, but reasoning statically builds the mental model that makes dynamic analysis meaningful. The discipline is therefore to push static reasoning as far as it will go.

Handling an unknown binary safely. Although this particular sample is benign, the habits practised here are the habits one must have with real malware. Two are worth stating as rules:

When triaging an unknown file, it is safe to compute its cryptographic hash and *look that hash up* on a service such as VirusTotal to see whether the sample is already known. It is **not** safe to *upload the file itself*. Uploading makes the sample available for download to anyone with a subscription, and it signals to an attacker — who can watch for their sample's hash appearing — that their delivery vector reached a target. In a security-sensitive environment, uploading a

freshly received file is a disclosure event in its own right. The rule is therefore: *search the hash, never upload the file.*

Finally, a benign command-line sample like this can be examined on an ordinary machine, but the correct reflex with anything genuinely suspicious is an isolated virtual machine with snapshots, so that a mistake costs a rollback rather than an infection.

---

2. Step 1 — Behavioural triage

The first question about any unknown executable is simply: *what does it do when you run it, and what does it want?*

`Bomb.exe` is a console application. Launched with no arguments, it prints a greeting and then waits:

```
Welcome to my fiendish little bomb. You have 6 phases with
which to blow yourself up. Have a nice day!
```

It then reads a line from standard input. Typing an arbitrary line causes the program to detonate and exit. From this alone three facts are established: the program is interactive, it consumes input one line at a time, and there are to be **six phases**. The greeting also hints that input might come from somewhere other than the keyboard — a possibility confirmed in the next step when the usage string is recovered. Nothing more can be learned by poking at it blindly; brute force is hopeless because the input space of each phase is effectively unbounded. The investigation must move to the code.

---

3. Step 2 — Basic static analysis (PE triage)

The second step gathers cheap, high-value information from the file's structure using a PE-inspection tool (for example PEStudio). None of it requires understanding a single instruction.

3.1 Headers, compiler, and sections

The file hash, searched on public reputation services, returns *no detections* across dozens of engines and — tellingly — no community reports or behavioural analyses, consistent with a benign, obscure sample.

The PE header carries a **compile timestamp** of 2010 and marks the file as a 32-bit executable (not a DLL). A timestamp is attacker-controllable and can be forged to any value, so it is treated as a weak hint rather than evidence. The section layout is entirely ordinary: `.text` (code), `.rdata` (read-only data), `.data` (read/write data), `.rsrc` (resources), `.reloc` (relocations). The only resource present is the application **manifest**, an XML blob

declaring the target architecture (x86) and the toolchain (`Microsoft.VC90.CRT`), which independently confirms the Visual C++ 2008 origin.

The 13 KB size and the choice of an old, lean compiler are consistent with a deliberately minimal build; older runtimes link smaller, lighter support code, which is a common reason to keep an aging compiler around for compact executables.

3.2 Imports

The import table is the first real window into capability. `Bomb.exe` imports from only two libraries:

- **`MSVCR90.dll`** — the Visual C++ 2008 C runtime. The imported functions are the tell: `printf`, `fopen`, `__iob_func`, `sscanf`, `atoi`, `isspace`, `fgets`, `strlen`, `getenv`. This set describes a program that *reads text from an input stream and parses it* — exactly what a challenge-response game would need, and nothing more.
- **`KERNEL32.dll`** — present in essentially every Windows executable. Its imports here (`Sleep`, `GetTickCount`, `QueryPerformanceCounter`, `IsDebuggerPresent`, `TerminateProcess`, `UnhandledExceptionFilter`, and so on) are all standard runtime and startup scaffolding. In particular `IsDebuggerPresent` is **not** evidence of anti-debugging intent here; it is a routine import that the CRT and exception machinery pull in on their own.

Crucially, there is *nothing* for networking, cryptography, process creation, file-system tampering, or persistence. The capability surface is "parse some strings read from input." This both confirms the sample is benign and tells the analyst what kind of logic to expect inside.

3.3 Strings — the goldmine

Listing the printable strings is where basic static analysis stops being inconclusive and becomes decisive. Several categories emerge.

A **leftover debug path** is present:

```
C:\Documents and Settings\user\Desktop\Bomb\Release\Bomb.pdb
```

This is the path to the program database (PDB) holding debug symbols, embedded by the linker. Its presence shows the build was not stripped for production. In real malware such a leak is a serious operational-security failure for the author: PDB paths routinely expose usernames, project names, and directory structure that help attribute a sample. Here it merely reveals a generic `user` account.

A set of **format and usage strings** reveals the command-line contract:

```
Usage: %s [<input_file>]
%s: Error: Couldn't open %s
```

The square brackets around `<input_file>` mark it optional, confirming the earlier suspicion: the program accepts a file whose contents are fed in place of keyboard input — a great convenience, since solutions can be written once into a file rather than retyped on every run.

The **phase banners** lay out the whole progression in plain text:

```
Phase 1 defused. How about the next one?
That's number 2.  Keep going!
Halfway there!
So you got that one.  Try this one.
Good work!  On to the next...
```

Alongside these sit the raw material of individual phases — a candidate answer string, a parsing format, an encoded-looking blob, a list of hostnames, and hints of something hidden:

```
Public speaking is very easy.
%d %c %d
isrveawhobpnutfggiants
...a list of <host>.cmcl.cs.cmu.edu domains...
Wow! You've defused the secret stage!
%d %s
austinpowers
Curses, you've found the secret phase!
But finding it and solving it are quite different...
Congratulations! You've defused the bomb!
```

The strings establish the shape of the entire program before a single instruction is read: six visible phases, a hidden "secret" phase, and several per-phase data items whose meaning must now be pinned down in the disassembly. Any of these could in principle be a decoy, so each is a hypothesis to be *validated* against the code, not a conclusion.

---

4. Step 3 — Structure in the disassembler

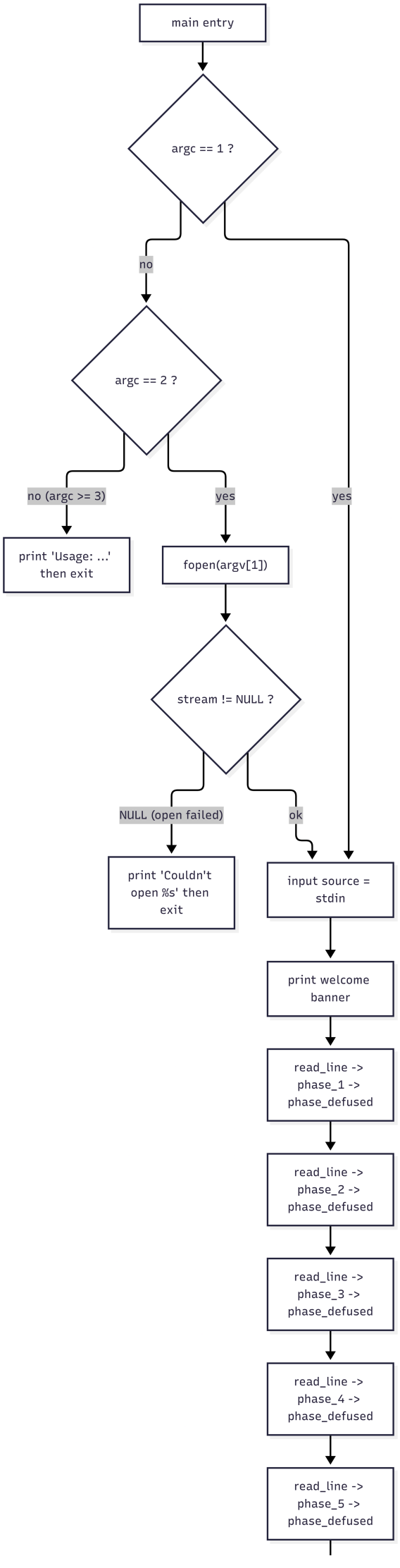
Loading the binary into IDA (declining the prompt to fetch the absent PDB from the Microsoft symbol server) gives the third step: understand the program's skeleton before diving into any one phase.

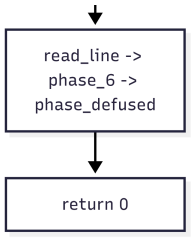
4.1 Finding `main`

Execution does not begin at `main`. The PE entry point lands in the C runtime startup sequence: first a routine that installs the **stack security cookie** (a guard value used to detect stack-smashing buffer overflows), then the compiler's `__tmainCRTStartup`, which sets up the structured exception handler, initialises the environment and terminal, and only then calls the user's `main`. A reliable heuristic locates `main` even without symbols: it is the single user call that sits just *before* the startup code's final exit calls, with both post-call branches flowing into process termination. Modern disassemblers label it automatically; the heuristic is what to fall back on when they do not.

4.2 The shape of `main`

`main` begins at `0x401000`. Its control flow is simple: a short preamble that inspects the argument count, and then one very large basic block that runs the six phases in sequence.


main entry
argc == 1 ?
no
argc == 2 ?
no (argc >= 3)
print 'Usage: ...' then exit
yes
fopen(argv[1])
stream != NULL ?
NULL (open failed)
print 'Couldn't open %s' then exit
ok
yes
input source = stdin
print welcome banner
read_line -> phase_1 -> phase_defused
read_line -> phase_2 -> phase_defused
read_line -> phase_3 -> phase_defused
read_line -> phase_4 -> phase_defused
read_line -> phase_5 -> phase_defused



The preamble reconstructs directly from the first instructions:

```
cmp DWORD PTR [ebp+0x8], 1      ; argc == 1 ?  (no user arguments)
jne ...                         ; if not, test argc == 2
...
cmp DWORD PTR [ebp+0x8], 2      ; argc == 2 ?  (one argument = a filename)
jne ...                         ; if argc >= 3 -> print Usage and exit
...
call fopen                      ; open argv[1] as the input file
cmp DWORD PTR ds:0x405610, 0    ; stream == NULL ?
jne ...                         ; NULL -> "Couldn't open %s" and exit
```

So input reaches the phases by one of exactly two routes: no arguments (read from the terminal) or exactly one argument naming a file that exists (read from the file). Two or more arguments trigger the usage message. This matches the usage string recovered earlier and is confirmable by passing deliberately wrong arguments and observing the printed errors.

4.3 Reading strings back into code: cross-references

A disassembler's most important navigational tool at this stage is the **cross-reference** (xref): given a data address, it lists every instruction that touches it. Selecting a string such as *"Wow! You've defused the secret stage!"* and pressing `X` jumps straight to the instruction that pushes it as an argument, revealing that it lives inside subroutine `0x401620`, which is reached from `0x401990`, which is called by `main`. Chasing xrefs this way maps relationships that are invisible when reading top-to-bottom, and it is how the hidden "secret" machinery is discovered without yet understanding it.

4.4 Naming landmarks: `explode_bomb`

A single helper subroutine, `0x401960`, recurs throughout. It prints

```
BOOM!!!
The bomb has blown up.
```

and then calls `exit`. Because it terminates the process, the disassembler draws no outgoing edge from the calls to it — a useful visual signature. This routine is the detonation. Renaming it something memorable such as `explode_bomb` immediately clarifies every phase: wherever control can reach `explode_bomb`, the input has failed a check, and the analyst's goal in each phase is simply to find the path that *avoids* it.

4.5 The per-phase sandwich

A value highlighted in `main` lights up its other occurrences. Doing this reveals that two subroutines are each called **six times**, once per phase, interleaved with a third subroutine that differs every time. The pattern is a sandwich:

```
read_line()            ; same function every phase  (0x401870)
phase_N(line)          ; different function each phase
phase_defused()        ; same function every phase  (0x401990)
```

- `0x401870` is called first in every phase, and its return value is passed straight into the phase function; it is the **input reader**. Renaming it `read_line` (or `get_input`) is justified by both its position and, confirmed later, its string-scanning body.
- The phase functions `0x401190`, `0x4011b0`, `0x401210`, `0x401390`, `0x4013e0`, `0x401460` are the six **checkers**.
- `0x401990` runs after every phase. It is `phase_defused`, and it hides a secret (Section 7).

This skeleton is the single most important deliverable of Step 3: with it, each phase can be attacked in isolation.

---

5. The reusable helper functions

Three small routines are shared across phases. Understanding them once pays off repeatedly.

5.1 `read_line` / `get_input` (`0x401870`)

This routine reads one line of input (from the terminal or the input file), guards against a line that is too long or a premature end-of-file (printing the corresponding error and detonating in those cases), stores the line in a global line buffer, and returns a pointer to it. One side effect inside it is central to the secret phase and is examined in Section 7: it increments a global counter every time it runs.

5.2 `string_length` (`0x401700`)

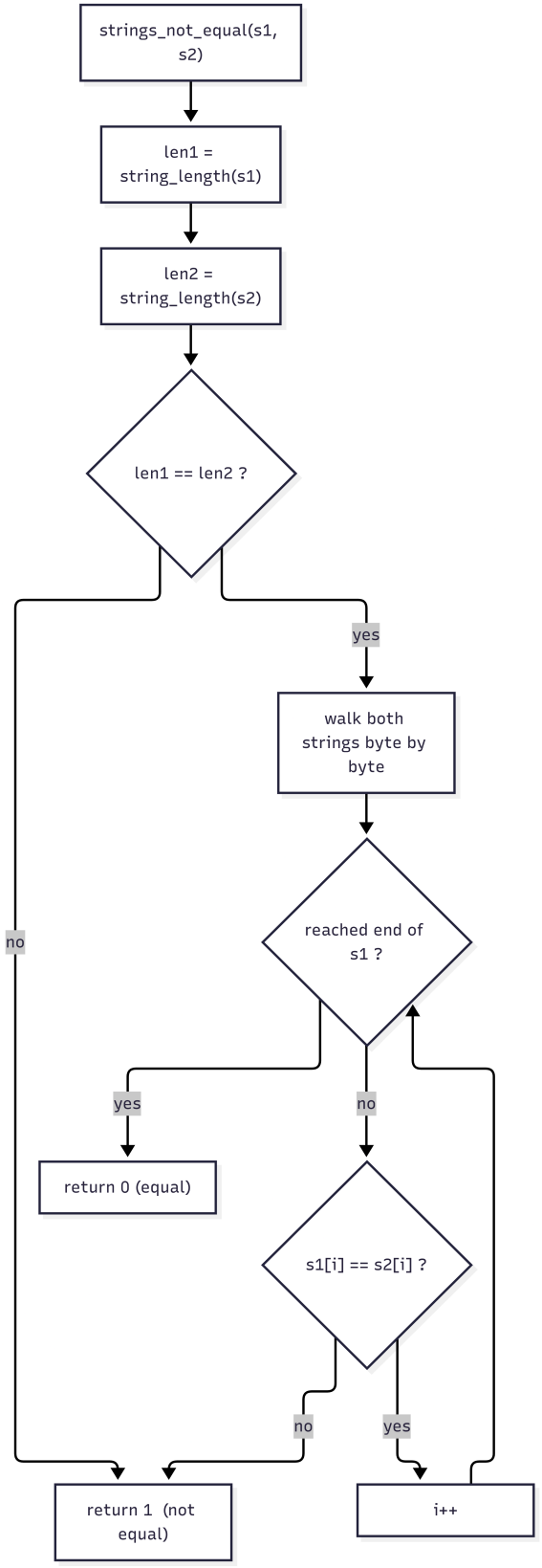
A textbook `strlen`. It walks a pointer along the buffer, incrementing a counter until it meets the NUL terminator, and returns the count:

```
movsx edx, BYTE PTR [ecx]   ; read one byte, sign/zero-extend to 32 bits
test  edx, edx              ; is it the NUL terminator (0) ?
je    done                  ; yes -> return count
...                         ; no  -> pointer++, count++, loop
```

The `movsx` (move with sign-extension) widening a single byte into a 32-bit register is unoptimised scaffolding — the compiler prefers 32-bit arithmetic to 8-bit even though a plain byte compare would do. Recognising such noise, and mentally reducing the body to `count++` over a `pointer++` walk to the terminator, is a core reading skill.

5.3 `strings_not_equal` (`0x401740`)

This routine compares two strings and returns **0 when they are equal**, non-zero otherwise — the inverse polarity is important, because the callers treat a non-zero result as failure (detonation). Its logic:



It first compares lengths (a quick reject), then compares character by character, returning success only if it reaches the terminator with every byte matching. This single routine underpins Phase 1, Phase 5, and the secret phase's password check.

---

6. Challenge #1 — a string comparison

Phase 1's checker is `0x401190`. Its body is tiny:

```
push 0x404550               ; address of the fixed string
mov  eax, [ebp+0x8]         ; the input line
push eax
call strings_not_equal      ; 0 if input == fixed string
test eax, eax
je   defused                ; equal -> phase passes
call explode_bomb           ; otherwise detonate
```

The phase passes if, and only if, the input line equals the fixed string at `0x404550`, which the disassembler decodes (via its printable-ASCII heuristic) as:

```
Public speaking is very easy.
```

A productive way to *confirm* a readable constant is not a decoy — without yet trusting the string heuristic — is the analyst's shortcut of simply supplying the candidate and watching the program's state change; here, entering the sentence advances the phase. But the static reading already settles it: Phase 1 is a plain string-equality test.

✓ Solution — Phase 1

```
Public speaking is very easy.
```

The match is exact, including the trailing period. Because the check is byte-for-byte, **a single extra space or missing character fails it** — a caution that matters especially when answers are supplied from a file.

---

7. The secret-stage gate

The subroutine that runs after every phase, `phase_defused` (`0x401990`), hides the entry to a seventh, undocumented challenge. Its mechanism is a small state machine built around one global variable, and it is a clean illustration of reading data flow through cross-references.

7.1 The counter

A global at `0x404c04` acts as a **phase counter**. Tracing its xrefs shows that it is *read* in several places but *written* in exactly one: inside `read_line`, where it is incremented by one on every call. Since `read_line` runs once per phase, the counter equals the number of lines consumed so far. (It is zero at startup — the binary places its uninitialised globals in a zero-filled region rather than a separate `.bss` section, so the first read sees 0. This is the kind of assumption a debugger would later confirm directly.)

7.2 The gate

`phase_defused` begins:

```
cmp DWORD PTR ds:0x404c04, 6   ; have all six phases been read?
jne skip                       ; if not, do nothing special
```

The secret logic therefore activates **only after the sixth line has been read** — i.e. while defusing phase 6. When the gate is open, the routine re-parses the *most recent input line* with `sscanf` using the format `"%d %s"`, extracting an integer and a string:

```
lea eax, [ebp-0x58]            ; -> string buffer
push eax
lea ecx, [ebp-0x4]             ; -> integer
push ecx
push 0x404b34                  ; "%d %s"
push 0x405070                  ; the stored input line
call sscanf
cmp DWORD PTR [ebp-0x5c], 2    ; did it parse BOTH a number and a word?
jne no_secret
push 0x404b3c                  ; "austinpowers"
lea edx, [ebp-0x58]
push edx
call strings_not_equal         ; is the trailing word exactly
"austinpowers"?
test eax, eax
jne no_secret
... print "Curses, you've found the secret phase!" ...
call secret_phase              ; 0x401620
```

7.3 The trick

The gate is subtle in two ways. First, the line it re-reads is the answer to **phase 4** — the phase whose own parser reads only a leading integer and ignores anything after it. Appending

a word to the phase-4 answer therefore does not disturb phase 4, but leaves a second token for `phase_defused` to find later. Second, the program prints *"Congratulations! You've defused the bomb!"* **either way** — whether or not the secret word is present. The secret stage is thus invisible unless one already expects it: solving the six visible phases looks complete, and nothing on screen reveals that a hidden path was skipped.

> **Unlocking the secret phase** Make the phase-4 answer be its required integer followed by the word `austinpowers` (for example `9 austinpowers`). The integer still satisfies phase 4; the trailing word opens the gate when the sixth line is read.

The `secret_phase` body itself (`0x401620`) reads a further line, converts it with `atoi`, requires the value to lie in `[1, 1001]`, and feeds it to a recursive routine over a binary tree, demanding a specific return value — a challenge left for later analysis.

---

8. Challenge #2 — a numeric recurrence

Phase 2's checker is `0x4011b0`. It is the first phase with real arithmetic, and it introduces the pattern of reading several numbers into a stack array.

8.1 Reading six integers

The checker reserves a block of stack space and passes its address, together with the input line, to a helper (`0x4016b0`). Inside that helper the structure is unmistakable once the addresses are lined up:

```
; push the addresses of six consecutive 4-byte slots, highest first
lea/compute &a[5], &a[4], &a[3], &a[2], &a[1], &a[0]
push 0x404a94                  ; "%d %d %d %d %d %d"
push <input line>
call sscanf
cmp DWORD PTR [ebp-0x4], 6     ; did sscanf assign all six?
jge ok
call explode_bomb              ; fewer than six numbers -> detonate
```

So the first requirement is simply that the line contain **six integers**; `sscanf` returns the count of successfully parsed values, and anything less than six detonates. The six values are stored in a six-element integer array — call it `a[0..5]`.

8.2 The checks

Back in the checker, the first number is pinned:

```
cmp DWORD PTR [ebp-0x1c], 1    ; a[0] == 1 ?
je  ...
call explode_bomb
```

Then a loop runs the index from 1 to 5 and enforces a relationship between each element and its predecessor:

```
mov eax, [ebp-0x4]             ; i
add eax, 1                     ; i + 1
mov ecx, [ebp-0x4]             ; i
imul eax, [ebp+ecx*4-0x20]     ; eax = (i+1) * a[i-1]
mov edx, [ebp-0x4]             ; i
cmp [ebp+edx*4-0x1c], eax      ; a[i] == (i+1) * a[i-1] ?
je  continue
call explode_bomb
```

The recurrence is therefore

$$a[0] = 1, \qquad a[i] = (i+1) \cdot a[i-1] \quad \text{for } i = 1 \ldots 5.$$

This is the factorial sequence. Unrolling it:

i	relation	value
0	given	1
1	2×1	2
2	3×2	6
3	4×6	24
4	5×24	120
5	6×120	720

The `imul reg, r/m` form seen here multiplies the register by the memory operand in place (`eax *= …`); it is the two-operand signed multiply, worth noting because it is the phase's only unfamiliar instruction. The recurrence can be unrolled with nothing more than paper and pencil, simulating one loop iteration at a time — the decompiler shortens the work but is not required.

✓ Solution — Phase 2

```
1 2 6 24 120 720
```

Separate the numbers with single spaces and add no trailing characters.

---

9. Practical workflow notes

Two conveniences make the remaining phases less tedious.

Answers in a file. Because `main` accepts an optional filename, the accumulated solutions can be written one per line into a text file and the bomb run as `Bomb.exe solutions.txt`, so correct phases need not be retyped on every attempt.

Whitespace discipline. Phases built on `strings_not_equal` compare byte-for-byte. A stray trailing space or an accidental extra blank line will fail an otherwise-correct answer. Numeric phases parsed with `sscanf` are more forgiving of spacing, but the safe habit is to keep answer lines exactly as the checks expect.

The decompiler as a last resort. A decompiler can collapse the ugliest of these routines into a line or two of C and is a legitimate aid when the hand-reading becomes a headache — but the skill being built is the hand-reading itself. Likewise, dynamic analysis with a debugger will later make the arithmetic phases almost immediate by letting one watch the expected values being computed; reaching for it too early short-circuits the understanding the static pass is meant to produce.

---

10. Where this leaves us

At this point the full analytical machinery is in place: the program's skeleton is mapped, the shared helpers (`read_line`, `string_length`, `strings_not_equal`, `explode_bomb`, `phase_defused`) are named and understood, the hidden secret-stage gate is explained, and the first two phases are solved. The accumulated answer file so far reads:

```
Public speaking is very easy.
1 2 6 24 120 720
```

with the phase-4 line later to be written as `… austinpowers` to arm the secret stage.

The remaining visible phases escalate in difficulty and in the language features they exercise:

- **Phase 3** parses with the `%d %c %d` format and branches on a jump-table `switch`, mapping a selector integer to an expected character and a second integer.
- **Phase 4** reads a single integer and feeds it to a self-recursive routine, then checks the result — a recursion to unwind.

- **Phase 5** maps each character of a six-character input through a lookup table keyed by the low nibble of its byte, requiring the result to spell a target word.
- **Phase 6** reads six integers and reorders a linked list, demanding a particular ordering of node values.
- **Secret phase** drives a value through a recursive binary-tree search for a fixed return value.

Each of these yields to the same method practised here: isolate the checker, find every path to `explode_bomb`, and reason backwards to the input that avoids them all.

CHAPTER 5: Advanced Dynamic Analysis

Overview

A *binary bomb* is a compiled program that reads input and, for each of several phases, calls a function `explode_bomb` — which prints `BOOM!!!` and terminates the process — unless the input exactly matches a hidden check. Defusing a phase means reading its compiled code, recovering the condition it tests, and supplying input that satisfies it. The bomb analyzed here is a 32-bit Windows console program with six phases and one hidden secret phase.

The work is done with a disassembler and debugger (IDA with the Hex-Rays plugin, driving the local Windows debugger). Disassembly is shown in Intel syntax (`destination, source`). The same tools and habits apply to real malware; a bomb is a safe target to practise them on because it carries no payload. Static reading answers what the code *could* do; the debugger confirms what it *does* with a given input, and the two are alternated throughout.

---

1. From the entry point to `main`

The address in the PE header's entry point is not `main`. It points to the C runtime (CRT) startup code, labelled `start`, which initializes the runtime — thread-local storage callbacks, exception handlers, environment variables, command-line arguments — then calls `main`, and after `main` returns calls `exit`. In a stripped binary (compiled without symbols) IDA cannot label `main`, so it is found by hand: it is the last substantial call before the final `exit`, sitting between the argument/environment setup and process termination.

The body confirms the identification. A representative `main`, located this way and then renamed, begins:

```
; int __cdecl main(int argc, const char **argv, const char **envp)
main proc near
    push    ebp
    mov     ebp, esp
    and     esp, 0FFFFFFF0h          ; enforce 16-byte ESP alignment
    call    sub_401AF0              ; CRT helper — step over
```

```
    mov     ds:dword_407074, 30     ; A = 30
    mov     ds:dword_407070, 21     ; B = 21
    mov     ds:dword_407078, 3      ; C = 3
    call    sub_401460
    ...
```

The pair `push ebp` / `mov ebp, esp` is the standard prologue: it saves the caller's frame pointer and makes `EBP` address the current frame, so arguments sit at positive offsets (`[ebp+8]` is the first argument) and locals at negative offsets (`[ebp-4]`, `[ebp-8]`). The instruction `and esp, 0FFFFFFF0h` clears the low four bits of `ESP`, forcing it to a multiple of 16. After the mask `ESP` either stays put or decreases, and a decreasing stack pointer means the stack grows to reach alignment. Alignment is set up once, at the start of a real function, not in arbitrary code.

> Exam tip Always break at the real `main`, not at the entry point. Setting a breakpoint on the loader's `start` forces you to step through the entire CRT startup chain, which is wasted effort. The `and esp, 0FFFFFFF0h` idiom is a reliable marker of a genuine entry/setup point when hunting for `main` in a stripped binary.

1.1 Reading an operand's region

The three writes target `dword_407074`, `dword_407070`, `dword_407078`, which are **global variables**, and the operand alone proves it. A value near `0x407074` sits just past the code and import sections of an image based at `0x400000`. It is far too low to be a stack address (stack addresses are high, near `0x0060FFxx`) and following the reference lands in `.bss`, the section for uninitialized globals. The constants `30`, `21`, `3` are the globals later named `A`, `B`, `C`.

Memory region	Address pattern (32-bit)	Meaning
`.text` / code	`0x401xxx`	instructions
`.data` / `.bss`	`0x404xxx` – `0x407xxx`	global/static variables
Stack	`0x0060FFxx` (high, grows down)	locals, saved registers, return addresses
Loaded DLLs	begins with `0x7`	library code and data

> Exam tip While debugging, a four-byte value that starts with `7` is almost certainly an address inside a loaded DLL (`kernel32`, `msvcrt`), because Windows maps libraries into a high, ASLR-controlled range. A value that reads as plausible data one way and as a `0x7…` address another way is usually a pointer into a library.

1.2 Little-endian memory

Multi-byte values are stored **little-endian**: the number `0x0000001E` appears in memory as the bytes `1E 00 00 00`, least-significant byte at the lowest address. Watching a `mov` write a global in the debugger makes this concrete — the value `30` (`0x1E`) lands as `1E 00 00 00`, not `00 00 00 1E`. Registers, by contrast, have **no endianness**: a register shown as `0x2D0` is simply the number `0x2D0`. Endianness is only about how a multi-byte value is laid out in memory.

---

2. Recognizing compiled C constructs

Reading assembly back to the C that produced it is the core static skill. A handful of patterns recur in every compiled function.

An `if` with no `else` compiles to a single conditional jump that skips the body. An `if` / `else` needs **two** jumps: a conditional jump that selects the else-branch and an unconditional jump at the end of the then-branch that hops over the else-branch. Counting the jumps distinguishes them — one conditional jump is a plain `if`, a conditional jump followed by an unconditional jump is an `if` / `else`.

Loops are built from a jump whose target is *earlier* than the jump itself, a **back edge**, which in the graph view is the visual signature of a loop. A `while` tests its condition at the top and skips the body entirely when the condition is false on entry; a `do` / `while` runs the body once before testing at the bottom; a `for` adds an initialization before the loop and an increment at the bottom. The loop counter is the local that is incremented or decremented and compared against a bound each iteration.

> **Exam tip** A local variable that is decremented and then tested against zero is a loop counter. Rename it `count` on sight — naming it early makes the surrounding loop far easier to read.

Compilers rewrite arithmetic into cheaper instructions, and these rewrites must be recognized on sight. Modulo by a power of two becomes a bitwise AND: `B % 4` compiles to `and eax, 3`, because `x % 2ⁿ == x & (2ⁿ − 1)`. Multiplication by a small constant becomes an `lea` or a shift: `A * 2` compiles to `lea edx, [eax+eax]`. `lea` is used for arithmetic because it evaluates an address expression without accessing memory and without touching the flags, so `lea eax, [eax+eax+1]` computes `2·eax + 1` in a single instruction.

---

3. Driving the debugger

3.1 Breakpoints and stepping

An **instruction breakpoint** halts execution when the program reaches a chosen address. It can be disabled (kept but inactive) or deleted. Four controls drive execution once the program is stopped:

- **Step into** — execute one instruction; on a `call`, enter the called function.
- **Step over** — execute one instruction; on a `call`, run the whole function and stop after it.
- **Run until return** — continue until the current function returns.
- **Run to cursor** — continue until a chosen instruction is reached.

Step into and step over are identical until a `call`. The practical rule is to step **over** library functions and compiler helpers — a CRT helper, `printf`, `__iob_func` — and step **into** only the code that matters. Stepping into `printf` by mistake drops you into a long runtime routine that IDA may not even recognize as a function; the recovery is `run until return`.

> **Exam tip** Before stepping into an unknown `call`, set a breakpoint on the instruction right after it. Then if the function turns out to be uninteresting you simply *continue* and regain control at that breakpoint, instead of single-stepping to its end. `run until return` is only a heuristic: if you are already deep inside a long function it may fail to find the return and cost you the whole session.

3.2 Flags and predicting branches

Changed registers are highlighted after each step, so effects are visible: a `push` decreases `ESP` by 4 and writes the value to the new top of stack; the instruction pointer changes on every step. A `cmp` or `test` writes no register and no memory — it changes only the flags. To predict a conditional jump, step the compare and read the flag it set.

Flag	Set when
ZF (zero)	result is zero / operands equal
SF (sign)	result is negative
CF (carry)	unsigned borrow or overflow
OF (overflow)	signed overflow

`jz` and `je` are the same opcode (jump if `ZF = 1`), as are `jnz` / `jne`. After `cmp a, b`, `je` means `a == b`; `jge` (signed) is taken when `SF == OF`; `ja` / `jae` are the unsigned counterparts of `jg` / `jge`.

> **Exam tip** A `cmp` or `test` only sets flags — nothing else changes. To know whether the jump below it will be taken, step the compare and watch which flag turns on, rather than trying to reason out the comparison in your head.

3.3 Forcing execution and patching

The debugger can change a running program, and there are two distinct mechanisms that must not be confused.

Forced execution sets the instruction pointer to another instruction, overriding the natural control flow — for example, redirecting a conditional jump down the branch it would not have taken. The debugger marks the forced target in a distinct colour. This bypasses a *check* but supplies none of the *data* the skipped branch would have produced.

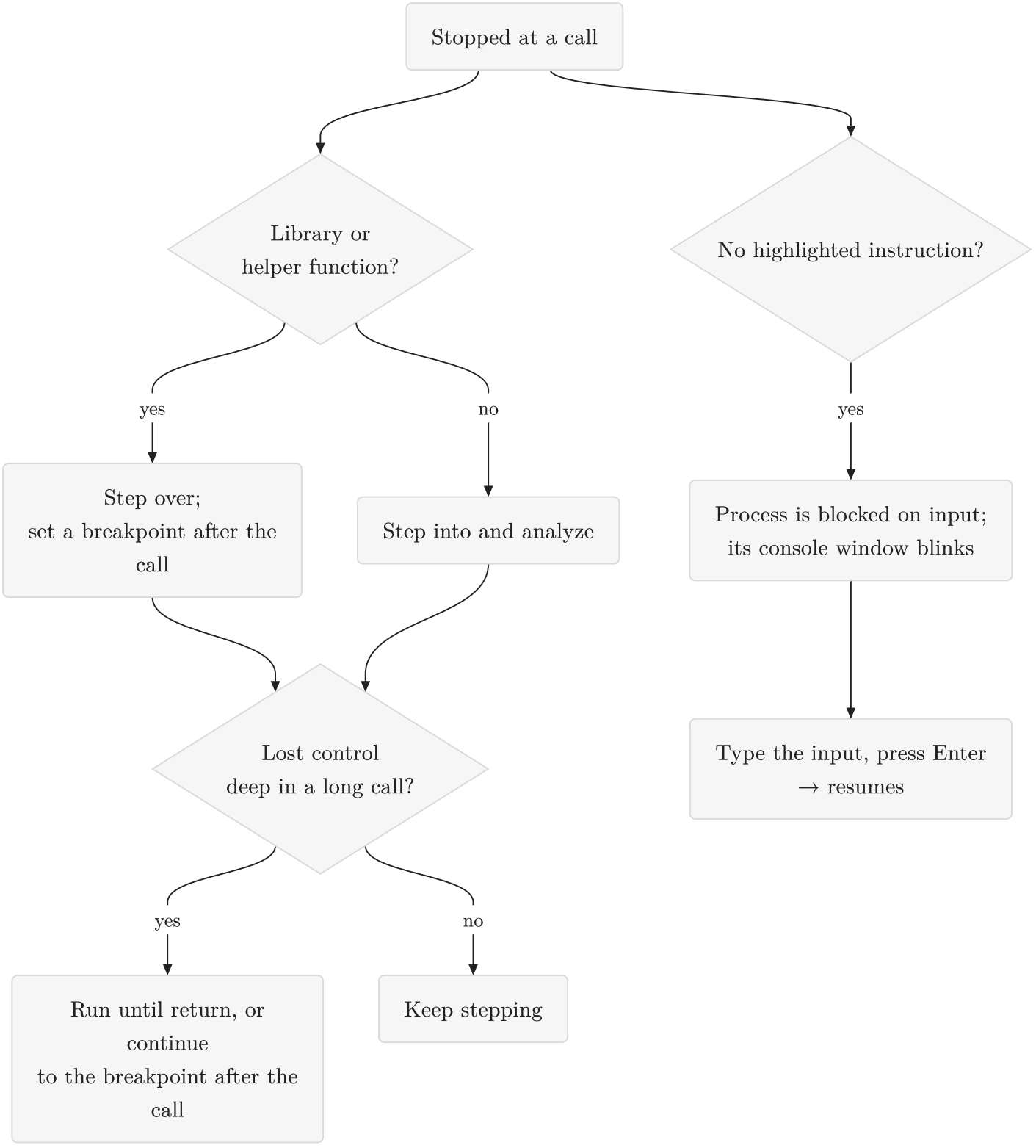
Memory patching edits the contents of a stack cell, global, or heap location directly: select the bytes, edit, and apply the change. This changes *data*, and it must respect little-endian order and the operand width. Editing a four-byte comparison means writing all four bytes, so `0x2D0` is patched as `D0 02 00 00`. After applying the edit, the cell's colour returns to normal, which confirms the patch took.

> **Exam tip** Forcing a branch is only sound when nothing downstream reads the data that branch was guarding. If a later step reuses a value the skipped branch should have written, the computation runs on stale data and fails further on — the classic "defused one step, exploded on the next." When later code depends on the value, patch the *data* to the expected value rather than only forcing the *branch*.

3.4 Blocked processes and the database

When a program calls a blocking function — reading standard input, waiting on the OS — the debugger shows no highlighted instruction because the thread is suspended inside the call. This is not a crash. The program's own console window, which may be hidden behind the debugger, blinks while waiting for input; typing the expected input and pressing Enter resumes execution. Breakpoints can still be set while the process is blocked, so control can be regained even if none was placed beforehand.

IDA stores all analysis — renamed functions, renamed variables, comments — in a database (`.idb` / `.i64`) separate from the executable. These annotations persist across sessions and carry between the disassembler and debugger views, so renaming `sub_401960` to `explode_bomb` or a local to `count` is work that is not lost when the program is reopened.



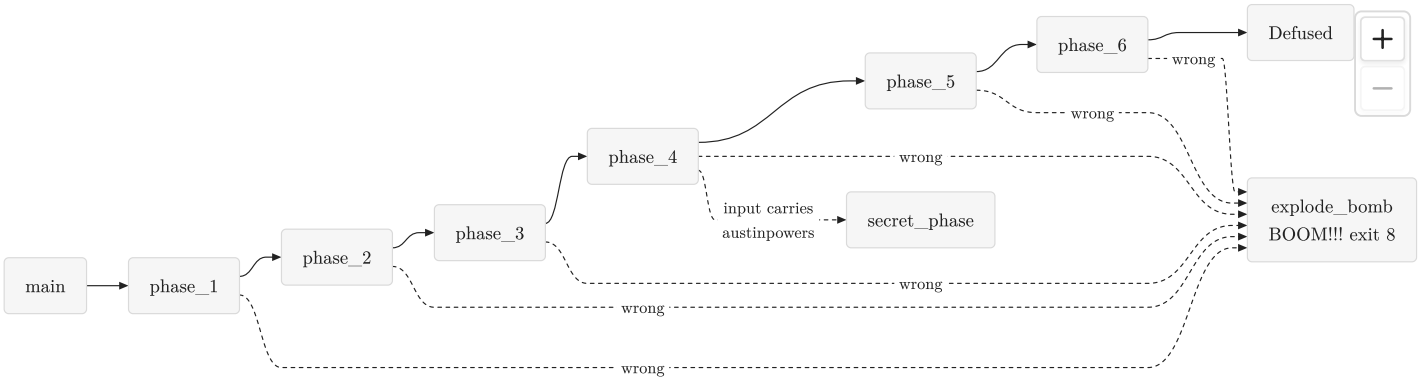
---

4. The binary bomb

4.1 Structure

The bomb's `main` prints a welcome banner, then for each phase reads a line of input, passes it to the phase function, and on success calls `phase_defused`. Any wrong input reaches `explode_bomb`, which prints `BOOM!!!` and calls `exit(8)`. Input arrives either from standard input or from a file named on the command line. The program inspects `argc`: one argument (the program name alone) reads from standard input; two arguments open the named file; more than two prints a usage error.

Several helper functions recur and are worth naming once. `explode_bomb` prints `BOOM!!!` and exits; every failed check jumps to it. `read_six_numbers` wraps `sscanf(line, "%d %d %d %d %d %d", …)` and explodes unless at least six integers were parsed. `strings_not_equal` compares two strings and returns non-zero if their lengths differ or any character differs. `phase_defused` runs after each phase and also holds the trigger for the secret phase.



The phase answers for this binary are summarized below and derived in the sections that follow. The answers are specific to this binary; the reasoning is general.

Phase	Mechanism	Answer
1	string equality	`Public speaking is very easy.`
2	recurrence over six integers	`1 2 6 24 120 720`
3	jump-table switch on the first integer	e.g. `0 q 777`
4	recursive function must return 55	`9`
5	nibble-indexed substitution cipher	`opekma`
6	reorder a linked list to descending order	`4 2 6 3 1 5`
secret	binary-search-tree path	`1001`

> **Exam tip** Solve statically whatever you can — a direct string comparison is readable straight from the data section with no debugger at all. Reserve the debugger for the phases whose expected values are *computed at runtime*, where watching the comparisons is faster than reconstructing the arithmetic by hand.

4.2 Phase 1 — string equality

Phase 1 passes the input and a fixed string to `strings_not_equal` and explodes if they differ.

```
    push    offset aPublicSpeaking   ; "Public speaking is very easy."
    mov     eax, [ebp+8]             ; user input
    push    eax
```

```
    call    strings_not_equal
    test    eax, eax
    jz      ok                      ; equal → defused
    call    explode_bomb
ok: retn
```

The answer is the referenced string read verbatim: **`Public speaking is very easy.`** This phase is solvable purely statically.

4.3 Phase 2 — a recurrence over six integers

Phase 2 reads six integers, requires the first to be `1`, and then checks a recurrence in a loop:

```
    cmp     [ebp+arr], 1            ; first number must be 1
    jz      cont
    call    explode_bomb
cont:
    mov     [ebp+i], 1              ; i = 1
loop:
    cmp     [ebp+i], 6
    jge     done
    mov     eax, [ebp+i]
    add     eax, 1                  ; (i+1)
    imul    eax, [ebp+(i-1)*4+arr]  ; (i+1) * arr[i-1]
    cmp     [ebp+i*4+arr], eax      ; arr[i] == (i+1)*arr[i-1] ?
    jz      next
    call    explode_bomb
next:
    inc     [ebp+i]
    jmp     loop
```

The condition is `arr[0] == 1` and `arr[i] == (i+1) · arr[i-1]`. Expanding gives the factorials offset by one:

```
arr[0] = 1
arr[1] = 2 · 1   = 2
arr[2] = 3 · 2   = 6
arr[3] = 4 · 6   = 24
arr[4] = 5 · 24  = 120
arr[5] = 6 · 120 = 720
```

The answer is **`1 2 6 24 120 720`**.

Solving this in the debugger is a complete worked example of the patching workflow. Enter six distinct placeholder numbers such as `1 2 3 4 5 6`, so each slot is distinguishable in the stack, and step the loop. At each comparison IDA displays the expected value it computed. The first two placeholders are already correct (`1`, `2`). At the third element the expected value is `6` but the stack holds `3`, and the next step would explode. Two recoveries are possible. The stack cell can be **patched** from `3` to `6` by editing the four bytes `06 00 00 00` and applying the change. Alternatively the comparison can be bypassed by **forcing** the instruction pointer onto the non-exploding branch.

Forcing without patching exposes the soundness trap directly. The recurrence reads `arr[i-1]` to compute the next expected value, so if the third element is left as `3` instead of `6`, the fourth iteration multiplies by the wrong previous value and every later check is doomed. The fix is to patch the data, or to set the instruction pointer back to the start of the computation block and re-execute it after restoring the registers (feasible here only because that block writes no memory and the registers can be reconstructed from `count`). Continuing to read the expected values this way recovers `1`, `2`, `6`, `0x18` (=24), `0x78` (=120), `0x2D0` (=720); the last is patched in little-endian order as `D0 02 00 00`.

> **Exam tip** When you patch a value wider than one byte, slice the number into byte pairs and reverse them for little-endian. The register value `0x2D0` is written to memory as `D0 02 00 00`, never `02 D0 00 00` and never `00 00 02 D0`. Registers themselves have no byte order; only the memory copy does.

4.4 Phase 3 — a jump-table switch

Phase 3 parses `"%d %c %d"` into a selector integer, a character, and a second integer, requiring `sscanf` to match at least three items. The selector indexes an eight-entry jump table; a selector above 7 explodes. Each case fixes an expected character and an expected second integer, and a final comparison checks the parsed character against the case's character.

```
    cmp     [ebp+nmatched], 3
    jge     ok1
    call    explode_bomb
ok1:
    cmp     [ebp+sel], 7
    ja      explode                 ; unsigned: sel > 7 → boom
    jmp     ds:jumptable[sel*4]     ; indirect jump through the table
```

The unsigned compare (`ja`) guards both ends of the range at once, since a negative selector is read as a large unsigned value. The jump table is data embedded in the code, so it must be read as an array of addresses rather than decoded as instructions. The eight cases give eight valid `selector char number` triples:

Selector	Char	2nd int (hex)	Decimal
0	`q`	`0x309`	777
1	`b`	`0x0D6`	214
2	`b`	`0x2F3`	755
3	`k`	`0x0FB`	251
4	`o`	`0x0A0`	160
5	`t`	`0x1CA`	458
6	`v`	`0x30C`	780
7	`b`	`0x20C`	524

Any row is a correct answer; the simplest is **`0 q 777`**.

4.5 Phase 4 — a recursive function

Phase 4 reads one integer (requiring exactly one match and a value `≥ 1`), passes it to a recursive function, and requires the result to equal `55` (`0x37`). The function returns `1` for `n ≤ 1`, otherwise `f(n-1) + f(n-2)` — the Fibonacci sequence with `f(0) = f(1) = 1`:

```
f(0)=1  f(1)=1  f(2)=2  f(3)=3  f(4)=5  f(5)=8
f(6)=13 f(7)=21 f(8)=34 f(9)=55
```

`f(9) = 55` and `9 ≥ 1`, so the answer is **`9`**.

4.6 Phase 5 — a nibble-indexed cipher

Phase 5 requires the input to be exactly six characters long, then maps each character through a substitution table and compares the result to a target. Each output character is `cipher[input[i] & 0xF]`: the low four bits of each input byte index into the 16-character table `isrveawhobpnutfggiants`, and the decoded six characters must equal `giants`.

```
index: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
char:  i s r v e a w h o b  p  n  u  t  f  g
```

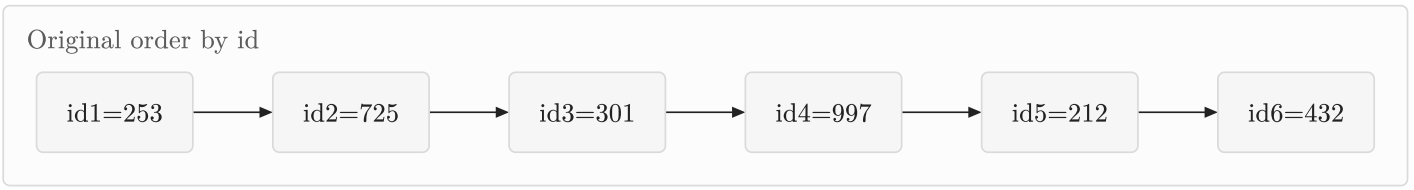
The target `giants` needs table indices `15, 0, 5, 11, 13, 1`. Any input bytes with those low nibbles work; choosing printable lowercase letters gives the nibbles `f 0 5 b d 1`, which are the characters `o p e k m a`. The answer is **`opekma`**.

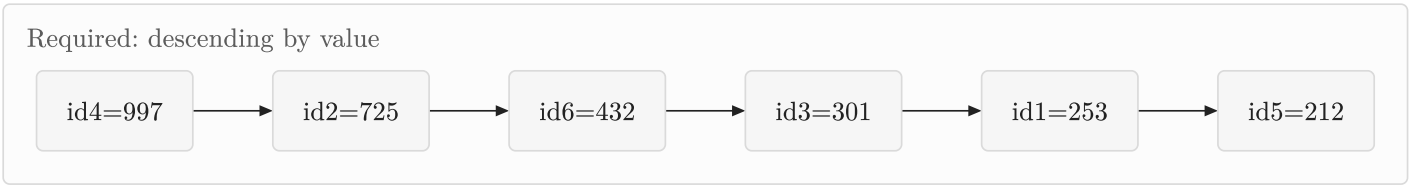
4.7 Phase 6 — reordering a linked list

Phase 6 reads six integers that must be a permutation of `1..6` (each in range and all distinct). It walks a global singly linked list of six nodes — each holding a value, an id, and a

`next` pointer — and uses the permutation to reorder the nodes, then verifies the reordered list is in **descending** order of value, exploding if any node's value is less than its successor's.

The node values, read from the data section, are `253, 725, 301, 997, 212, 432` for ids `1..6`. Sorting by descending value gives the id order `4, 2, 6, 3, 1, 5`, which is the permutation the phase requires, so the answer is **`4 2 6 3 1 5`**.

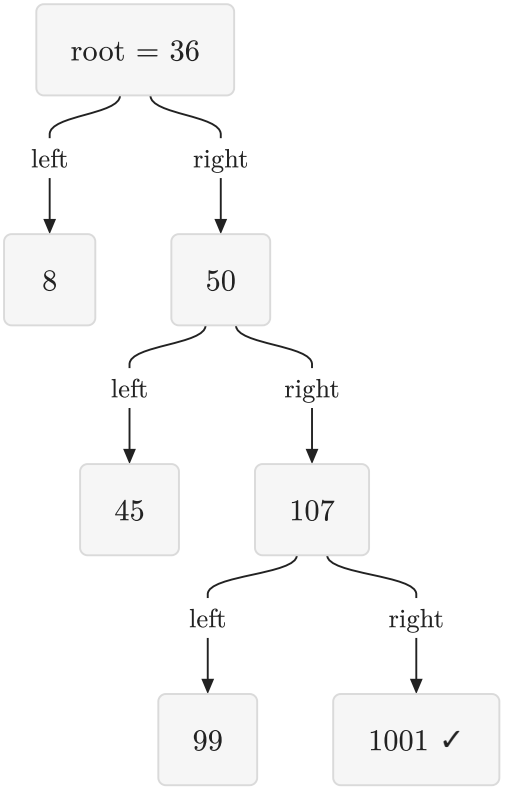




4.8 The secret phase

`phase_defused` runs after every phase but only acts once all six are solved. It then re-parses the stored **phase-4 input line** with the format `"%d %s"`. Phase 4 consumed only the leading integer, so a trailing word on that line survives unused; if that word equals `austinpowers`, the hidden `secret_phase` is invoked. Unlocking it means entering phase 4 as `9 austinpowers` instead of `9` — the `%d` still yields 9 and phase 4 passes, while the `%s` captures the trigger word.

`secret_phase` reads a number, requires it in `[1, 1001]`, and calls a recursive binary-search-tree lookup (`fun7`) that must return `7`. Each left step in the tree multiplies the result by 2, each right step multiplies by 2 and adds 1, and a match returns 0, so a required result pins down an exact path from the root. The value `7 = 2·(2·(2·0+1)+1)+1` decodes to right, right, right, then match. The tree's right spine is `36 → 50 → 107 → 1001`, so the secret-phase answer is **`1001`**.



---

5. Safe analysis of a live sample

The same tools apply to real malware, but launching an unknown executable under a debugger adds risk that must be managed first. Pressing *run* is the moment the machine enters a risk zone, and the debugger warns of unauthorized code execution for exactly this reason. The standard precautions are to work in a disposable virtual machine, take a snapshot beforehand and restore it afterward, and disconnect networking so the sample cannot reach a command-and-control server or spread.

A breakpoint on `main` assumes execution begins there, but code can run earlier. **TLS callbacks** are registered during startup and execute before `main`, and are a common place for malware to run setup or anti-analysis code ahead of any `main` breakpoint; packed samples likewise unpack and transfer control before a `main`-level breakpoint is reached. A hostile function may also never return, so `step over` is not a guarantee of regaining control — the function can terminate the thread, spawn another, or run anti-analysis tricks. A sample may detect the debugger and retaliate rather than reveal its logic.

> **Exam tip** An import such as `IsDebuggerPresent` is a lead to investigate, not proof of evasion — the normal runtime uses it too, in its error-reporting path. Read every suspicious import in context before concluding it is an active anti-debugging check. And remember that `step over` on real malware may never hand control back; keep a breakpoint set as a failsafe. One wrong step in dynamic analysis can force you to restart from the beginning.

---

6. A second target: a string-building ransomware

A small ransomware sample, built with a custom toolchain to resist reading, shows the same techniques on deliberately awkward code. Its `main` opens with a long run of `mov` instructions writing to the stack, which statically looks like meaningless integer moves. Those moves are a **stack-constructed wide string**: read four bytes at a time, the sequence spells a UTF-16LE string, each character followed by a `00` byte and the whole thing ending in a `00 00` terminator. The disassembler shows the region as `dd` integers precisely because of the interleaved zero bytes.

The string is recovered dynamically by breaking on the first `mov`, letting the moves run, and reading the destination buffer, where `b.r.i.e.f.c.a.s.e` appears. It can also be recovered statically by selecting the bytes and undefining them (`dd` → `db`) so IDA re-interprets the region as text. The program then calls `SHGetFolderPathW` with a `CSIDL` constant to resolve a per-user special folder (such as the desktop), whose path varies by machine and user, and feeds the result into string-manipulation APIs and `CreateFileW`.

> **Exam tip** Strings assembled on the stack one character at a time never appear in a `strings` scan, because they are built at runtime and live in no data section. Recover them by reading the buffer in the debugger, or statically by undefining the bytes so the disassembler re-reads them as a string. A `lea [ebp+local]` pushed before a call marks a local buffer that the callee will fill — annotate it (for example as `pszPath`) before moving on.